

Business Architecture Core Metamodel

Version: Initial Submission

OMG Document Number:

Normative reference: http://www.omg.org/spec/bacm_yyyymmdd

Machine readable file(s): http://www.omg.org/bacm_yyyymmdd

Normative: http://www.omg.org/spec/bacm_yyyymmdd/foo.xmi

Non-normative: http://www.omg.org/spec/bacm_yyyymmdd/non_normative/foo.xmi

Copyright © 2017, Business Architecture Guild
Copyright © 2017, Object Management Group, Inc.

USE OF SPECIFICATION - TERMS, CONDITIONS & NOTICES

The material in this document details an Object Management Group specification in accordance with the terms, conditions and notices set forth below. This document does not represent a commitment to implement any portion of this specification in any company's products. The information contained in this document is subject to change without notice.

LICENSES

The companies listed above have granted to the Object Management Group, Inc. (OMG) a nonexclusive, royalty-free, paid up, worldwide license to copy and distribute this document and to modify this document and distribute copies of the modified version. Each of the copyright holders listed above has agreed that no person shall be deemed to have infringed the copyright in the included material of any such copyright holder by reason of having used the specification set forth herein or having conformed any computer software to the specification.

Subject to all of the terms and conditions below, the owners of the copyright in this specification hereby grant you a fully-paid up, non-exclusive, nontransferable, perpetual, worldwide license (without the right to sublicense), to use this specification to create and distribute software and special purpose specifications that are based upon this specification, and to use, copy, and distribute this specification as provided under the Copyright Act; provided that: (1) both the copyright notice identified above and this permission notice appear on any copies of this specification; (2) the use of the specifications is for informational purposes and will not be copied or posted on any network computer or broadcast in any media and will not be otherwise resold or transferred for commercial purposes; and (3) no modifications are made to this specification. This limited permission automatically terminates without notice if you breach any of these terms or conditions. Upon termination, you

will destroy immediately any copies of the specifications in your possession or control.

PATENTS

The attention of adopters is directed to the possibility that compliance with or adoption of OMG specifications may require use of an invention covered by patent rights. OMG shall not be responsible for identifying patents for which a license may be required by any OMG specification, or for conducting legal inquiries into the legal validity or scope of those patents that are brought to its attention. OMG specifications are prospective and advisory only. Prospective users are responsible for protecting themselves against liability for infringement of patents.

GENERAL USE RESTRICTIONS

Any unauthorized use of this specification may violate copyright laws, trademark laws, and communications regulations and statutes. This document contains information which is protected by copyright. All Rights Reserved. No part of this work covered by copyright herein may be reproduced or used in any form or by any means--graphic, electronic, or mechanical, including photocopying, recording, taping, or information storage and retrieval systems--without permission of the copyright owner.

DISCLAIMER OF WARRANTY

WHILE THIS PUBLICATION IS BELIEVED TO BE ACCURATE, IT IS PROVIDED "AS IS" AND MAY CONTAIN ERRORS OR MISPRINTS. THE OBJECT MANAGEMENT GROUP AND THE COMPANIES LISTED ABOVE MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS PUBLICATION, INCLUDING BUT NOT LIMITED TO ANY WARRANTY OF TITLE OR OWNERSHIP, IMPLIED WARRANTY OF MERCHANTABILITY OR WARRANTY OF FITNESS FOR A PARTICULAR PURPOSE OR USE. IN NO EVENT SHALL THE OBJECT MANAGEMENT GROUP OR ANY OF THE COMPANIES LISTED ABOVE BE LIABLE FOR ERRORS CONTAINED HEREIN OR FOR DIRECT, INDIRECT, INCIDENTAL, SPECIAL, CONSEQUENTIAL, RELIANCE OR COVER DAMAGES, INCLUDING LOSS OF PROFITS, REVENUE, DATA OR USE, INCURRED BY ANY USER OR ANY THIRD PARTY IN CONNECTION WITH THE FURNISHING, PERFORMANCE, OR USE OF THIS MATERIAL, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

The entire risk as to the quality and performance of software developed using this specification is borne by you. This disclaimer of warranty constitutes an essential part of the license granted to you to use this specification.

RESTRICTED RIGHTS LEGEND

Use, duplication or disclosure by the U.S. Government is subject to the restrictions set forth in subparagraph (c) (1) (ii) of The Rights in Technical Data and Computer Software Clause at DFARS 252.227-7013 or in subparagraph (c)(1) and (2) of the Commercial Computer Software - Restricted Rights clauses at 48 C.F.R. 52.227-19 or as specified in 48 C.F.R. 227-7202-2 of the DoD F.A.R. Supplement and its successors, or as specified in 48 C.F.R. 12.212 of the Federal Acquisition Regulations and its successors, as applicable. The specification copyright owners are as indicated above and may be contacted through the Object Management Group, 109 Highland Avenue, Needham, MA 02494, U.S.A.

TRADEMARKS

IMM®, MDA®, Model Driven Architecture®, UML®, UML Cube logo®, OMG Logo®, CORBA® and

XMI® are registered trademarks of the Object Management Group, Inc., and Object Management Group™, OMG™, Unified Modeling Language™, Model Driven Architecture Logo™, Model Driven Architecture Diagram™, CORBA logos™, XMI Logo™, CWM™, CWM Logo™, IIOP™, MOF™, OMG Interface Definition Language (IDL)™, and OMG SysML™ are trademarks of the Object Management Group. All other products or company names mentioned are used for identification purposes only, and may be trademarks of their respective owners.

COMPLIANCE

The copyright holders listed above acknowledge that the Object Management Group (acting itself or through its designees) is and shall at all times be the sole entity that may authorize developers, suppliers and sellers of computer software to use certification marks, trademarks or other special designations to indicate compliance with these materials.

Software developed under the terms of this license may claim compliance or conformance with this specification if and only if the software compliance is of a nature fully matching the applicable compliance points as stated in the specification. Software developed only partially matching the applicable compliance points may claim only that the software was based on this specification, but may not claim compliance or conformance with this specification. In the event that testing suites are implemented or approved by Object Management Group, Inc., software developed using this specification may claim compliance or conformance with the specification only if the software satisfactorily completes the testing suites.

OMG's Issue Reporting Procedure

All OMG specifications are subject to continuous review and improvement. As part of this process we encourage readers to report any ambiguities, inconsistencies, or inaccuracies they may find by completing the Issue Reporting Form listed on the main web page <http://www.omg.org>, under Documents, Report a Bug/Issue (http://www.omg.org/report_issue.)

Table of Contents

1	Scope	1
2	Conformance	2
3	References	2
4	Terms and Definitions	3
5	Symbols	3
6	Additional Information	3
6.1	Changes to Adopted OMG Specifications [optional]	3
6.2	Acknowledgments	3
7	Specification	4
7.1	Overview of BACM	4
7.1.1	Core Domain Packages	4
7.1.2	Extended Domain Packages	4
7.1.3	Interdisciplinary Alignment Packages	5
7.1.4	Baseline Packages	5
7.2	BACM_Model	6
7.2.1	BACM_Model diagram	6
7.2.2	Classes:	8
7.3	Foundations	19
7.3.1	Foundations diagram	19
7.3.2	Class Elements diagram	21
7.3.3	Classes:	21
7.4	Capability	31
7.4.1	Capability diagram	31
7.4.2	Affects diagram	32
7.4.3	Classes:	33
7.5	Value	47
7.5.1	Value diagram	47
7.5.2	Classes:	49
7.6	Information	57

7.6.1	Information diagram	57
7.6.2	Classes:	58
7.7	Organization	64
7.7.1	Organization diagram	65
7.7.2	Classes:	66
7.8	BusinessObject	73
7.8.1	BusinessObject diagram.....	73
7.8.2	Classes:	74
7.9	Stakeholder.....	78
7.9.1	Stakeholder diagram	78
7.9.2	Classes:	80
7.10	Product	84
7.10.1	Product diagram	84
7.10.2	Classes:	85
7.11	Policy	90
7.11.1	Policy diagram.....	90
7.11.2	Classes:	92
7.12	Strategy	95
7.12.1	Strategy diagram.....	96
7.12.2	Classes:	97
7.13	Initiative	104
7.13.1	Initiative diagram.....	104
7.13.2	Classes:	105
7.14	Requirements.....	108
7.14.1	Requirements diagram	108
7.14.2	Classes:	109
7.15	Alignment.....	113
7.15.1	Alignment diagram	114
7.15.2	Classes:	114
8	Alignment Guidelines (non-normative).....	117
8.1	BACM Alignment with BMM	117
8.1.1	BMM diagram	117
8.1.2	Classes:	117
8.1.3	Assembler: Course of Action	118

8.1.4	Assembler: Objective.....	118
8.2	BACM Alignment with TOGAF Capability-based Planning	119
8.2.1	CapabilityIncrement diagram.....	119
8.2.2	Classes:	120
8.3	BACM Alignment with VDML	121
8.3.1	Introduction	121
8.3.2	Value Alignment	122
8.3.3	Capability Alignment.....	123
8.4	BACM Alignment with BPMN, CMMN, and DMN	125
8.4.1	Background	125
8.4.2	Approach to Touchpoint Mechanisms.....	126
8.4.3	Mappings from Assemblers to Corresponding Elements	128
8.4.4	Examples of Touchpoint Mechanism at Work per Different Methodologies.....	132
8.4.5	Conclusion	133
9	Annexes	134
9.1	Annex A: Glossary	135
9.2	Annex B: References	139

Preface

OMG

Founded in 1989, the Object Management Group, Inc. (OMG) is an open membership, not-for-profit computer industry standards consortium that produces and maintains computer industry specifications for interoperable, portable, and reusable enterprise applications in distributed, heterogeneous environments. Membership includes Information Technology vendors, end users, government agencies, and academia.

OMG member companies write, adopt, and maintain its specifications following a mature, open process. OMG's specifications implement the Model Driven Architecture® (MDA®), maximizing ROI through a full-lifecycle approach to enterprise integration that covers multiple operating systems, programming languages, middleware and networking infrastructures, and software development environments. OMG's specifications include: UML® (Unified Modeling Language™); CORBA® (Common Object Request Broker Architecture); CWM™ (Common Warehouse Metamodel); and industry-specific standards for dozens of vertical markets.

More information on the OMG is available at <http://www.omg.org/>.

OMG Specifications

As noted, OMG specifications address middleware, modeling and vertical domain frameworks. All OMG Specifications are available from the OMG website at:

<http://www.omg.org/spec>

Specifications are organized by the following categories:

Business Modeling Specifications

Middleware Specifications

- **CORBA/IIOP**
- **Data Distribution Services**
- **Specialized CORBA**

IDL/Language Mapping Specifications

Modeling and Metadata Specifications

- **UML, MOF, CWM, XMI**
- **UML Profile**

Modernization Specifications

Platform Independent Model (PIM), Platform Specific Model (PSM), Interface Specifications

- **CORBAServices**
- **CORBAFacilities**

CORBA Embedded Intelligence Specifications

CORBA Security Specifications

OMG Domain Specifications

Signal and Image Processing Specifications

All of OMG's formal specifications may be downloaded without charge from our website. (Products implementing OMG specifications are available from individual suppliers.) Copies of specifications, available in PostScript and PDF format, may be obtained from the Specifications Catalog cited above or by contacting the Object Management Group, Inc. at:

OMG Headquarters
109 Highland Avenue
Needham, MA 02494
USA
Tel: +1-781-444-0404
Fax: +1-781-444-0320
Email: pubs@omg.org

Certain OMG specifications are also available as ISO standards. Please consult <http://www.iso.org>

Typographical Conventions

The type styles shown below are used in this document to distinguish programming statements from ordinary English. However, these conventions are not used in tables or section headings where no distinction is necessary.

Times/Times New Roman - 10 pt.: Standard body text

Helvetica/Arial - 10 pt. Bold: OMG Interface Definition Language (OMG IDL) and syntax elements.

Courier - 10 pt. Bold: Programming language elements.

Helvetica/Arial - 10 pt: Exceptions

NOTE: Terms that appear in italics are defined in the glossary. Italic text also represents the name of a document, specification, or other publication.

Issues

The reader is encouraged to report any technical or editing issues/problems with this specification to http://www.omg.org/report_issue.htm.

1 Scope

The Business Architecture Core Metamodel (BACM) establishes a metamodel that represents what a business does, how it delivers stakeholder value, the information it uses, and its overall structure. The BACM further represents business strategy, policy, stakeholder engagement, product, and initiative perspectives and how these perspectives align with what the business does, value delivery, information, and business structure. In addition, the BACM facilitates alignment with various operating model, or interdisciplinary, perspectives as well as related standards.

The BACM has the following characteristics.

- Supports business architecture modeling as an integration of multiple perspectives to ensure that the business architecture is consistent and enables more effective management of business complexities.
- Provides views of the business architecture model that are at a level of abstraction appropriate for the understanding and analysis of business executives.
- Aligns to best practices and principles as articulated by “A Guide to the Business Architecture Body of Knowledge®” (BIZBOK® Guide).¹
- Provides models based on a standard that enables the exchange of models between different modeling tools to avoid vendor lock-in and promote competition for quality and continued improvement of modeling tools.
- Supports the development of consistent modeling techniques and methods to advance the business architecture discipline and support the training and certification of business architecture professionals.
- Facilitates the integration of multiple models of business, whether based on OMG standards or not.

A business architecture must have the capacity to effectively represent a business ecosystem, which is defined as “one or more legal entities, in whole or in part, that exist as an integrated community of individuals and assets, or aggregations thereof, interacting as a cohesive whole toward a common mission or purpose.”² The BACM is structured in such a way as to have the ability to fully represent a given organization’s business ecosystem.

The scope of business architecture is framed by fundamental business architecture focal points or “domains”. These domains form the basic building blocks of business architecture and provide the basis for establishing a wide variety of business abstractions that enable business ecosystem transparency. Business architecture domains align to the definition of business architecture as formally adopted by the Federation of Enterprise Architecture Professional Organizations (FEAPO)³ as follows.

“Business architecture represents holistic, multidimensional business views of: capabilities, end-to-end value delivery, information, and organizational structure; and the relationships among these business views and strategies, products, policies, initiatives, and stakeholders”.⁴

FEAPO is comprised of organizations such as Business Architecture Guild, Business Architecture Society, Data Management International, IEEE Computer Society, the International Federation for Information Processing International Professional Practice Partnership (IP3), International Institute of Business Analysis (IIBA), International Council on Systems Engineering, and other professional associations worldwide.⁵ The business architecture domain perspective leverages the FEAPO definition with capabilities, end-to-end value delivery, information, and organizational structure representing core domains and strategies, products, policies, initiatives, and stakeholders representing the extended domains.

Business architecture ties these perspectives together in various ways to more fully represent a business from multiple perspectives. Consider that a business is broken down into business units, each of which has certain capabilities. Capabilities enable value streams and require certain information. Similarly, strategies impact capabilities, products are enabled by capabilities, and initiatives impact a cross-section of capabilities and values streams. The BACM enables an organization to formally represent these and many other business domains and relationships, which in turn forms the basis for articulating a business architecture and leveraging that business architecture across a wide variety of business scenarios.

Business architecture is applied to many business scenarios where a scenario represents common situations where business architecture is applied to leverage, streamline, improve upon, or otherwise positively influence the final result and related investment. The following sample scenarios represent a cross-section of business situations or endeavors where business architecture has and continues to be applied in practice.⁶

- Investment Analysis
- Shift to Customer Centric Business Model
- Digital Transformation
- New Product/Service Rollout
- Merger & Acquisition
- Divestiture
- Audit Review
- Regulatory Compliance
- Business Performance Improvement
- Business Model Transformation
- Risk Management
- Strategy Shaping and Interpretation
- Operational Cost Reduction
- Joint Venture Deployment
- Globalization
- Business Capability Outsourcing
- Program Scope Determination
- IT Investment Analysis
- Cross-Industry Alignment, Collaboration
- Supply Chain Streamlining
- Customer Experience Optimization
- Change Management

It is important to note what is not defined in the BACM, but rather aligned to the BACM with the intent of establishing business focal points that enable and enhance a cross-section of related disciplines. The BACM, for example, does not represent a *business operating model*, where operating model is defined as “an abstract representation of how an organization operates across a range of domains in order to accomplish its function.”⁷ The BACM does, however, offer the ability to align with various operating model perspectives including business process management, case management, and decision management. Aligning operating models to business architecture provides substantive business value when strategy and related impacts are driven through business architecture perspectives versus the highly detailed, volatile operating model.

The BACM does not incorporate IT aspects of enterprise architecture that include data architecture, application architecture, and technical architecture.⁸ It does, however, establish a formal basis for aligning data, application, and technical architecture perspectives to business architecture, establishing the basis for business-driven, IT architecture alignment and transformation.

Finally, the BACM avoids veering into what would be considered “instances” of a business architecture that are best defined in a given reference model. For example, a transportation company would define assets, locations, and other business object-based perspectives in its business architecture. These business objects form the basis for capabilities, defined in a capability map, and information concepts, defined in an information map. The BACM does not incorporate these reference model instances into the metamodel because doing so would constrain an organization’s ability to frame and leverage capability and related business architecture domain mappings unique to their business model or models.

A summary of core and extended business architecture domain packages, interdisciplinary and standards alignment packages, and foundational packages are summarized in section 7 and, for non-normative alignments, section 8.

2 Conformance

The Conformance clause identifies which clauses of the specification are mandatory (or conditionally mandatory) and which are optional in order for an implementation to claim conformance to the specification.

Note: For conditionally mandatory clauses, the conditions must, of course, be specified.

3 References

3.1 Normative References

The following normative documents contain provisions which, through reference in this text, constitute provisions of this specification. For dated references, subsequent amendments to, or revisions of, any of these publications do not apply.

List of normative references.

3.2 Non-normative References

List of non-normative references.

4 Terms and Definitions

For the purposes of this specification, the following terms and definitions apply.

Term

See Annex A: Glossary of Terms.

5 Symbols

List of symbols/abbreviations.

6 Additional Information

6.1 Changes to Adopted OMG Specifications [optional]

Not applicable.

6.2 Acknowledgments

The following companies submitted this specification:

- * Armstrong Process Group, Inc.
- * Benchmark Consulting
- * Business Architecture Guild
- * Capsicum Business Architects Pty Ltd.
- * Holocentric Pty Ltd.
- * Tactical Strategy Group, Inc.

The following companies supported this specification:

- * BPM.com (BPM, Inc.)
- * Business Architecture Associates. Inc.
- * EnterpriseWeb, LLC
- * Federation of Enterprise Architecture Professional Organizations (FEAPO)
- * Model Driven Solutions
- * Planview, Inc.
- * S2E Consulting, Inc.
- * Thematrix Partners, LLC
- * The Software Revolution, Inc. (TSRI)

7 Specification

Section 7 provides details of the specification and includes a BACM overview package as well as the remaining normative models in the form of various packages.

7.1 Overview of BACM

BACM decomposes into a number of models or packages that represent core and extended business architecture domains as well as interrelated discipline and standards alignments. BACM packages are framed by the business architecture domains defined in section 1, each of which represents a specific, well-bounded business focal point that is used to establish formal abstractions for representing an organization’s business ecosystem. The following core and extended domain, interdisciplinary, and foundational packages are discussed in section 7.

7.1.1 Core Domain Packages

Core business architecture domains represent the business architecture baseline and include capability, value stream, information, and organization. Core domain packages, which incorporate relationships among the four core domains, are summarized as follows.

Capability Package: A capability is defined as “a particular ability or capacity that a business may possess or exchange to achieve a specific purpose or outcome”.⁹ A capability describes *what* a business does, not *how* it is done. The capability package defines capability, outcome, instance, behavior, and associations required for establishing and leveraging capabilities in a business architecture. The capability package also includes the affects diagram.

Value Package: The value package models value delivery to stakeholders through the concept of the value stream. A value stream is defined as “An end-to-end collection of activities that creates a result for a customer, who may be the ultimate customer or an internal end-user of the value stream.”¹⁰ The value package models value proposition, value item, value stream, and value item as well relationships to capability, capability outcome, and stakeholder.

Information Package: The information package is the centerpiece for the “way in which to represent business terms and semantics within the context of business architecture.”¹¹ The information map frames the basis for representing information and its ability to make explicit a business object.

Organization Package: The organization package represents the business structure of an organization. This package represents business units, which include partners, structure, collaborative teams, and accountability along with relationships to capability, policy, and goals.

7.1.2 Extended Domain Packages

Extended business architecture domains, which represent an expanded view of the business, include strategy, policy, product, initiative, and stakeholder. Extended domain packages establish relationships to the four core domains as well as to other extended domains as follows.

Stakeholder Package: Stakeholder is defined as “An internal or external individual or organization with a vested interest in achieving value through a particular outcome.”¹² Note here that stakeholders are formally categorized and represent a cross-section of internal and external individuals and organizations. The stakeholder package defines stakeholder, both triggering and participating, and customer segment, and their associations with value stream stage and capability outcome where capability outcomes contribute to value items within the context of a given value stream stage.

Product Package: Product is defined as “the overall experience provided by the combination of goods and services to satisfy the customer’s needs.”¹³ The concept of a product targets end customers and does not refer to a good or service exchanged internally. The product package defines product, product line, and entitlement, and their associations with business unit, capability, value proposition, and information concept.

Policy Package: Policy is defined as “a course or principle of action adopted or proposed by a government, party, business, or individual.”¹⁴ The policy package defines policy and its associations with stakeholder, business unit, capability, and information.

Strategy Package: Strategy is defined as “The pattern or plan that integrates an organization’s major goals, policies, and action sequences into a cohesive whole.”¹⁵ The strategy package defines and manages objectives, course of action, needs, and change within business architecture and the relationship to capability.

Initiative Package: Initiative is defined as “a course of action that is being executed or has been selected for execution.”¹⁶ Initiative in this context includes what many organizations call programs and projects. The initiative package defines the initiative and its associations with capability, value stream stage, value item, objective, course of action, and business unit.

7.1.3 Interdisciplinary Alignment Packages

Beyond the core and extended business architecture domains, the BACM defines packages that align related business disciplines, including requirements analysis, business process management, case management, and decision management. Process, case, and decision management alignment are addressed through alignment to formal OMG standards. In addition, related standards alignment include representing points of intersection with the Business Motivation Model (BMM) and The Open Group Architecture Framework™ (TOGAF™).

Requirements Package: Business requirement is defined as “A condition or capability needed by a stakeholder to solve a problem or achieve an objective.”¹⁷ The requirements package defines requirement and its associations to objective, capability, stakeholder, product, information, value stream stage, and value proposition. This requirements alignment package establishes the basis for refining, aligning, rationalizing, and reusing requirements across an organization.

The following interdisciplinary and standards alignment packages are contained in the section 8, the non-normative portion of the specification.

- Business Motivation Model (BMM) Alignment Package
- TOGAF Alignment Package
- Value Delivery Modeling Language™ (VDML™) Alignment Package
- Business Process Modeling Notation (BPMN) Alignment Package
- Case Management Modeling Notation (CMMN) Alignment Package
- Decision Management Notation (DMN) Alignment Package

7.1.4 Baseline Packages

Certain summary level, foundational, and related packages fill out the remainder of the BACM. These packages are defined in section 7 as follows.

- BACM Package: Provides an overall perspective of all packages and aligns core and extended domain packages, foundations and related packages, interdisciplinary packages, and additional alignment packages.
- Foundations Package: Establishes foundational elements, operating entity, category, and association to the Structured Metrics Metamodel (SMM)¹⁸. The foundations package includes the class elements diagram.
- Business Object Package: The business object package establishes the model for defining a (passive) business object, which is used as a basis for certain business architecture domains including capability and information.
- Alignment Package: The alignment package establishes the “assembler” which is used to associate elements of the BACM with elements of other modeling standards.

7.2 BACM_Model

The BACM package provides an overview of all models within the BACM.

7.2.1 BACM_Model diagram

Class diagram in package 'BACM_Model'

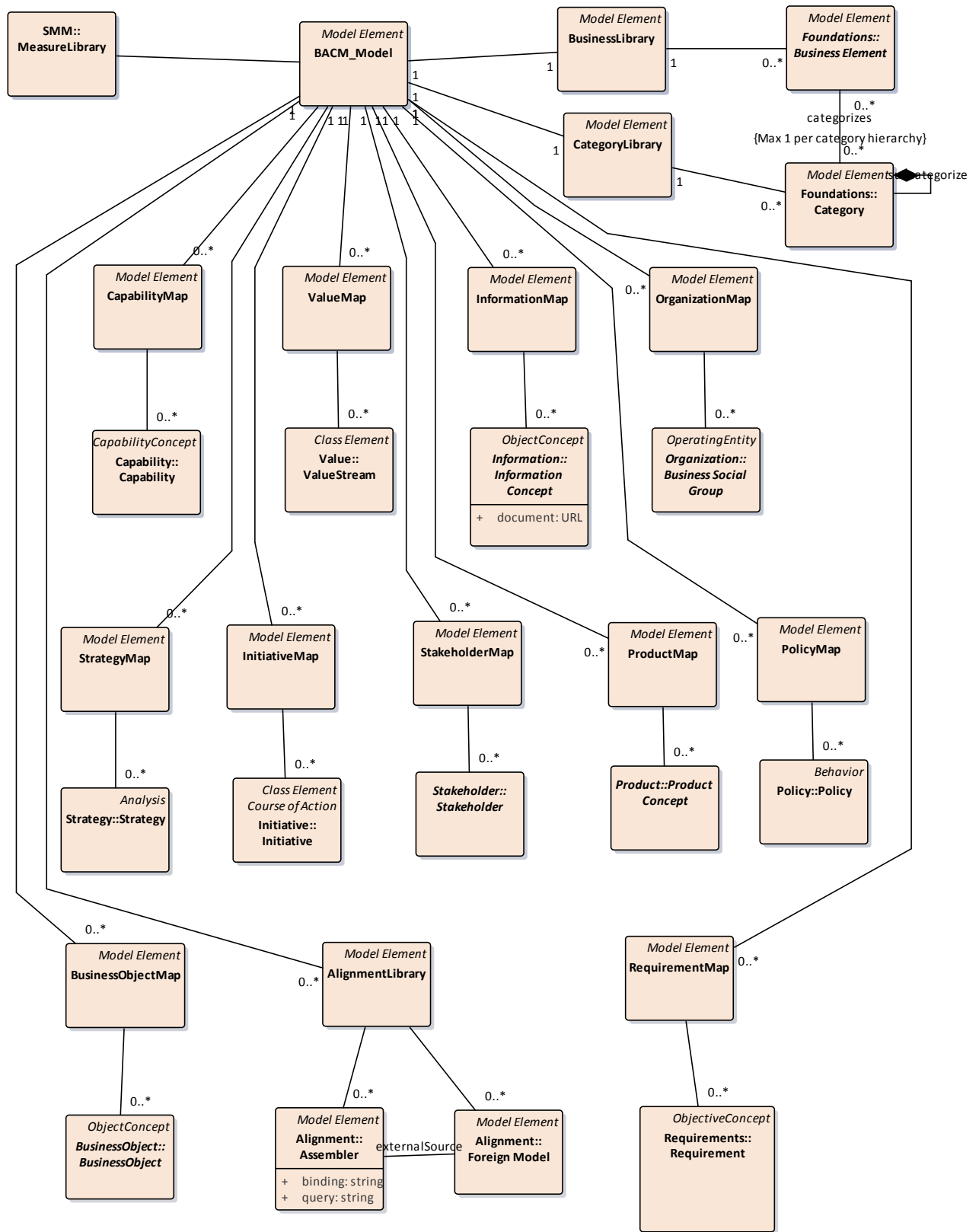


Figure 1: BACM_Model

The BACM_Model diagram shows the root element (BACM_Model) for a business architecture model. This root element has various containers, called "maps" that hold key elements associated with a particular perspective on the

business being modeled.

In addition, the `BACM_Model` element also has an iterable container, the `BusinessLibrary`, which permits sequential access to each business element of the model.

The `CategoryLibrary` is a container of `Category` hierarchies. The business architect is expected to define category hierarchies in accordance with the purpose and objectives of the modeling effort. The `CategoryLibrary` contains all of the `Category` elements defined in the model.

The `AlignmentLibrary` holds the defined alignments represented by `Assembler` elements that extract information from foreign models. The `AlignmentLibrary` also holds the endpoint definitions needed to access the contents of the foreign models.

`MeasureLibrary` is defined by the OMG Structured Metrics Meta-Model standard. This submission anticipates the use of Version 1.2 of the SMM (in process at this time) but is fully compatible with prior versions of SMM.

The `BACM_Model` may incorporate none or several SMM `MeasureLibraries` (see the current SMM specification at omg.org). SMM `MeasureLibraries` define `Measures` that can be associated with `SMM:Measurements` associated with `BACM::BusinessElements` (see `Foundations` package). The `BACM` anticipates the use of SMM 1.2 (in process for standardization) but is compatible with prior versions.

7.2.2 Classes:

7.2.2.1 Class `AlignmentLibrary`

in package 'BACM_Model'

The `AlignmentLibrary` holds the defined alignments with external models in an iterable container. An alignment is represented by an `Assembler`. An `Assembler` extracts information from a foreign model by means of a query. There are typically multiple `Assemblers` retrieving information from the same foreign model, so the information needed to connect to the foreign model is contained in the `ForeignModel` element that is shared by the several `Assembler` elements. The `AlignmentLibrary` holds the `ForeignModel` elements in an iterable container.

7.2.2.1.1 Superclasses

7.2.2.1.1.1 Class `Foundations::ModelElement`

7.2.2.1.2 Subclasses:

7.2.2.1.3 Associations:

7.2.2.1.3.1.1 Association: Type: `Association`

Source: `BACM_Model::AlignmentLibrary` **Role:** **Cardinality:**

Target: `Alignment::Assembler` **Role:** **Cardinality:** 0..*

7.2.2.1.3.1.1.1 *Constraints:*

7.2.2.1.3.1.1.2 Association: Type: `Association`

Source: `BACM_Model::AlignmentLibrary` **Role:** **Cardinality:**

Target: `Alignment::ForeignModel` **Role:** **Cardinality:** 0..*

7.2.2.1.3.1.2.1 *Constraints:*

7.2.2.1.3.1.3 Association: Type: `Association`

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::AlignmentLibrary **Role:** **Cardinality:** 0..*

7.2.2.1.3.1.3.1 *Constraints:*

7.2.2.1.4 Attributes:

7.2.2.1.5 Methods:

7.2.2.2 Class BACM_Model

in package 'BACM_Model'

7.2.2.2.1 Superclasses

7.2.2.2.1.1 Class Foundations::Model Element

7.2.2.2.2 Subclasses:

7.2.2.2.3 Associations:

7.2.2.2.3.1.1 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::BusinessLibrary **Role:** **Cardinality:** 1

7.2.2.2.3.1.1.1 *Constraints:*

The BACM_Model includes exactly one BusinessLibrary.

7.2.2.2.3.1.2 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:**

Target: SMM::MeasureLibrary **Role:** **Cardinality:**

7.2.2.2.3.1.2.1 *Constraints:*

7.2.2.2.3.1.3 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::CategoryLibrary **Role:** **Cardinality:** 1

7.2.2.2.3.1.3.1 *Constraints:*

7.2.2.2.3.1.4 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::AlignmentLibrary **Role:** **Cardinality:** 0..*

7.2.2.2.3.1.4.1 *Constraints:*

7.2.2.2.3.1.5 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::ValueMap **Role:** **Cardinality:** 0..*

7.2.2.2.3.1.5.1 *Constraints:*

7.2.2.2.3.1.6 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::CapabilityMap **Role:** **Cardinality:** 0..*

7.2.2.2.3.1.6.1 *Constraints:*

7.2.2.2.3.1.7 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::StrategyMap **Role:** **Cardinality:** 0..*

7.2.2.2.3.1.7.1 *Constraints:*

7.2.2.2.3.1.8 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::InformationMap **Role:** **Cardinality:** 0..*

7.2.2.2.3.1.8.1 *Constraints:*

7.2.2.2.3.1.9 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::PolicyMap **Role:** **Cardinality:** 0..*

7.2.2.2.3.1.9.1 *Constraints:*

7.2.2.2.3.1.10 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::StakeholderMap **Role:** **Cardinality:** 0..*

7.2.2.2.3.1.10.1 *Constraints:*

7.2.2.2.3.1.11 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::OrganizationMap **Role:** **Cardinality:** 0..*

7.2.2.2.3.1.11.1 *Constraints:*

7.2.2.2.3.1.12 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::RequirementMap **Role:** **Cardinality:** 0..*

7.2.2.2.3.1.12.1 *Constraints:*

7.2.2.2.3.1.13 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::BusinessObjectMap **Role:** **Cardinality:** 0..*

7.2.2.2.3.1.13.1 *Constraints:*

7.2.2.2.3.1.14 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::ProductMap **Role:** **Cardinality:** 0..*

7.2.2.2.3.1.14.1 *Constraints:*

7.2.2.2.3.1.15 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::InitiativeMap **Role:** **Cardinality:** 0..*

7.2.2.2.3.1.15.1 *Constraints:*

7.2.2.2.4 Attributes:

7.2.2.2.5 Methods:

7.2.2.3 Class BusinessLibrary

in package 'BACM_Model'

The BusinessLibrary provides an iterable collection of all BusinessElements defined in the model.

7.2.2.3.1 Superclasses

7.2.2.3.1.1 Class Foundations::Model Element

7.2.2.3.2 Subclasses:

7.2.2.3.3 Associations:

7.2.2.3.3.1.1 Association: Type:Association

Source: BACM_Model::BusinessLibrary **Role:** **Cardinality:** 1

Target: Foundations::Business Element **Role:** **Cardinality:** 0..*

7.2.2.3.3.1.1.1 *Constraints:*

7.2.2.3.3.1.2 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::BusinessLibrary **Role:** **Cardinality:** 1

7.2.2.3.3.1.2.1 *Constraints:*

The BACM_Model includes exactly one BusinessLibrary.

7.2.2.3.4 Attributes:

7.2.2.3.5 Methods:

7.2.2.4 Class BusinessObjectMap

in package 'BACM_Model'

The BusinessObjectMap contains the BusinessObjects of the business.

Each BusinessObject must appear in at least one BusinessObjectMap.

7.2.2.4.1 Superclasses

7.2.2.4.1.1 Class Foundations::Model Element

7.2.2.4.2 Subclasses:

7.2.2.4.3 Associations:

7.2.2.4.3.1.1 Association: Type:Association

Source: BACM_Model::BusinessObjectMap **Role:** **Cardinality:**

Target: BusinessObject::BusinessObject **Role:** **Cardinality:** 0..*

7.2.2.4.3.1.1.1 *Constraints:*

7.2.2.4.3.1.2 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::BusinessObjectMap **Role:** **Cardinality:** 0..*

7.2.2.4.3.1.2.1 *Constraints:*

7.2.2.4.4 Attributes:

7.2.2.4.5 Methods:

7.2.2.5 Class CapabilityMap

in package 'BACM_Model'

The CapabilityMap is a container of a complete set of Capabilities that model a business or a part of a business.

CapabilityMaps cannot be combined and should not be used to factor the architecture of a business; any factoring should be done by creating the appropriate Capability hierarchy.

A BACM_Model may incorporate multiple CapabilityMaps. These can be different versions of the business or different businesses.

Each Capability must appear in at least one CapabilityMap.

7.2.2.5.1 Superclasses

7.2.2.5.1.1 Class Foundations::Model Element

7.2.2.5.2 Subclasses:

7.2.2.5.3 Associations:

7.2.2.5.3.1.1 Association: Type:Association

Source: BACM_Model::CapabilityMap **Role:** **Cardinality:**

Target: Capability::Capability **Role:** **Cardinality:** 0..*

7.2.2.5.3.1.1.1 *Constraints:*

7.2.2.5.3.1.1.2 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::CapabilityMap **Role:** **Cardinality:** 0..*

7.2.2.5.3.1.2.1 *Constraints:*

7.2.2.5.4 Attributes:

7.2.2.5.5 Methods:

7.2.2.6 Class CategoryLibrary

in package 'BACM_Model'

Categories are hierarchies that define inheritable properties. They provide a way for the business architect to categorize BusinessElements with properties resulting from analysis, inspection or simulation.

The CategoryLibrary is an iterable container that holds the root element of every category hierarchy defined in a business architecture model.

7.2.2.6.1 Superclasses

7.2.2.6.1.1 **Class Foundations::Model Element**

7.2.2.6.2 Subclasses:

7.2.2.6.3 Associations:

7.2.2.6.3.1.1 Association: Type:Association

Source: BACM_Model::CategoryLibrary **Role:** **Cardinality:** 1

Target: Foundations::Category **Role:** **Cardinality:** 0..*

7.2.2.6.3.1.1.1 *Constraints:*

7.2.2.6.3.1.1.2 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::CategoryLibrary **Role:** **Cardinality:** 1

7.2.2.6.3.1.2.1 *Constraints:*

7.2.2.6.4 Attributes:

7.2.2.6.5 Methods:

7.2.2.7 Class InformationMap

in package 'BACM_Model'

The InformationMap is a container for the InformationConcepts defined for a business. There may be several InformationMaps in a BACM_Model, but InformationMaps should not be used to factor a business architecture model.

Each InformationConcept in a BACM_Model should appear in at least one InformationMap.

7.2.2.7.1 Superclasses

7.2.2.7.1.1 Class Foundations::Model Element

7.2.2.7.2 Subclasses:

7.2.2.7.3 Associations:

7.2.2.7.3.1.1 Association: Type:Association

Source: BACM_Model::InformationMap **Role:** **Cardinality:**

Target: Information::Information Concept **Role:** **Cardinality:** 0..*

7.2.2.7.3.1.1.1 *Constraints:*

7.2.2.7.3.1.2 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::InformationMap **Role:** **Cardinality:** 0..*

7.2.2.7.3.1.2.1 *Constraints:*

7.2.2.7.4 Attributes:

7.2.2.7.5 Methods:

7.2.2.8 Class InitiativeMap

in package 'BACM_Model'

The InitiativeMap contains the Initiatives of the business. The business architecture model may have several InitiativeMaps.

Each Initiative in a business architecture model must appear in at least one InitiativeMap

7.2.2.8.1 Superclasses

7.2.2.8.1.1 Class Foundations::Model Element

7.2.2.8.2 Subclasses:

7.2.2.8.3 Associations:

7.2.2.8.3.1.1 Association: Type:Association

Source: BACM_Model::InitiativeMap **Role:** **Cardinality:**

Target: Initiative::Initiative **Role:** **Cardinality:** 0..*

7.2.2.8.3.1.1.1 *Constraints:*

7.2.2.8.3.1.2 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::InitiativeMap **Role:** **Cardinality:** 0..*

7.2.2.8.3.1.2.1 *Constraints:*

7.2.2.8.4 Attributes:

7.2.2.8.5 Methods:

7.2.2.9 Class OrganizationMap

in package 'BACM_Model'

The OrganizationMap contains the BusinessSocialGroups of a business architecture model. There may be several OrganizationMaps, representing versions of a business or separate businesses.

Each BusinessSocialGroup in the business architecture model must appear in at least one OrganizationMap.

7.2.2.9.1 Superclasses

7.2.2.9.1.1 Class Foundations::Model Element

7.2.2.9.2 Subclasses:

7.2.2.9.3 Associations:

7.2.2.9.3.1.1 Association: Type:Association

Source: BACM_Model::OrganizationMap **Role:** **Cardinality:**

Target: Organization::Business Social Group **Role:** **Cardinality:** 0..*

7.2.2.9.3.1.1.1 *Constraints:*

7.2.2.9.3.1.2 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::OrganizationMap **Role:** **Cardinality:** 0..*

7.2.2.9.3.1.2.1 *Constraints:*

7.2.2.9.4 Attributes:

7.2.2.9.5 Methods:

7.2.2.10 Class PolicyMap

in package 'BACM_Model'

The PolicyMap contains the Policies of the business. There may be several PolicyMaps.

Each Policy must appear in at least one PolicyMap.

7.2.2.10.1 Superclasses

7.2.2.10.1.1 Class Foundations::Model Element

7.2.2.10.2 Subclasses:

7.2.2.10.3 Associations:

7.2.2.10.3.1.1 Association: Type:Association

Source: BACM_Model::PolicyMap **Role:** **Cardinality:**

Target: Policy::Policy **Role:** **Cardinality:** 0..*

7.2.2.10.3.1.1.1 *Constraints:*

7.2.2.10.3.1.2 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::PolicyMap **Role:** **Cardinality:** 0..*

7.2.2.10.3.1.2.1 *Constraints:*

7.2.2.10.4 Attributes:

7.2.2.10.5 Methods:

7.2.2.11 Class ProductMap

in package 'BACM_Model'

The ProductMap holds the ProductConcepts of the business. There may be more than one ProductMap in a business architecture model.

Each ProductConcept in a model must appear in at least one ProductMap of the model.

7.2.2.11.1 Superclasses

7.2.2.11.1.1 **Class Foundations::Model Element**

7.2.2.11.2 Subclasses:

7.2.2.11.3 Associations:

7.2.2.11.3.1.1 Association: Type:Association

Source: BACM_Model::ProductMap **Role:** **Cardinality:**

Target: Product::Product Concept **Role:** **Cardinality:** 0..*

7.2.2.11.3.1.1.1 *Constraints:*

7.2.2.11.3.1.2 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::ProductMap **Role:** **Cardinality:** 0..*

7.2.2.11.3.1.2.1 *Constraints:*

7.2.2.11.4 Attributes:

7.2.2.11.5 Methods:

7.2.2.12 Class RequirementMap

in package 'BACM_Model'

The RequirementMap contains all of the Requirements defined in a business architecture model.

7.2.2.12.1 Superclasses

7.2.2.12.1.1 Class Foundations::Model Element

7.2.2.12.2 Subclasses:

7.2.2.12.3 Associations:

7.2.2.12.3.1.1 Association: Type:Association

Source: BACM_Model::RequirementMap **Role:** **Cardinality:**

Target: Requirements::Requirement **Role:** **Cardinality:** 0..*

7.2.2.12.3.1.1.1 *Constraints:*

7.2.2.12.3.1.2 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::RequirementMap **Role:** **Cardinality:** 0..*

7.2.2.12.3.1.2.1 *Constraints:*

7.2.2.12.4 Attributes:

7.2.2.12.5 Methods:

7.2.2.13 Class StakeholderMap

in package 'BACM_Model'

The StakeholderMap contains the Stakeholders of a business. There may be several StakeholderMaps in a business architecture model.

Each Stakeholder must appear in at least one StakeholderMap.

7.2.2.13.1 Superclasses

7.2.2.13.1.1 Class Foundations::Model Element

7.2.2.13.2 Subclasses:

7.2.2.13.3 Associations:

7.2.2.13.3.1.1 Association: Type:Association

Source: BACM_Model::StakeholderMap **Role:** **Cardinality:**

Target: Stakeholder::Stakeholder **Role:** **Cardinality:** 0..*

7.2.2.13.3.1.1.1 *Constraints:*

7.2.2.13.3.1.2 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::StakeholderMap **Role:** **Cardinality:** 0..*

7.2.2.13.3.1.2.1 *Constraints:*

7.2.2.13.4 Attributes:

7.2.2.13.5 Methods:

7.2.2.14 Class StrategyMap

in package 'BACM_Model'

A StrategyMap contains the set of Strategies contemplated by a business. A strategy is said to be operational when an Initiative is created for it.

Each Strategy in a business architecture model must appear at least once in a StrategyMap.

7.2.2.14.1 Superclasses

7.2.2.14.1.1 Class Foundations::Model Element

7.2.2.14.2 Subclasses:

7.2.2.14.3 Associations:

7.2.2.14.3.1.1 Association: Type:Association

Source: BACM_Model::StrategyMap **Role:** **Cardinality:**

Target: Strategy::Strategy **Role:** **Cardinality:** 0..*

7.2.2.14.3.1.1.1 *Constraints:*

7.2.2.14.3.1.2 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::StrategyMap **Role:** **Cardinality:** 0..*

7.2.2.14.3.1.2.1 *Constraints:*

7.2.2.14.4 Attributes:

7.2.2.14.5 Methods:

7.2.2.15 Class ValueMap

in package 'BACM_Model'

The ValueMap holds all of the ValueStreams defined for a business. There may be more than one ValueMap associated with a BACM_Model, but ValueMaps should not be used to factor the business being modeled. ValueStreams can be categorized in lieu of placing them in separate ValueMaps.

Each ValueStream in a business architecture model must appear in at least one ValueMap.

7.2.2.15.1 Superclasses

7.2.2.15.1.1 Class Foundations::Model Element

7.2.2.15.2 Subclasses:

7.2.2.15.3 Associations:

7.2.2.15.3.1.1 Association: Type:Association

Source: BACM_Model::ValueMap **Role:** **Cardinality:**

Target: Value::ValueStream **Role:** **Cardinality:** 0..*

7.2.2.15.3.1.1.1 Constraints:

7.2.2.15.3.1.2 Association: Type:Association

Source: BACM_Model::BACM_Model **Role:** **Cardinality:** 1

Target: BACM_Model::ValueMap **Role:** **Cardinality:** 0..*

7.2.2.15.3.1.2.1 Constraints:

7.2.2.15.4 Attributes:

7.2.2.15.5 Methods:

7.3 Foundations

Package in package 'Model'

7.3.1 Foundations diagram

Class diagram in package 'Foundations'

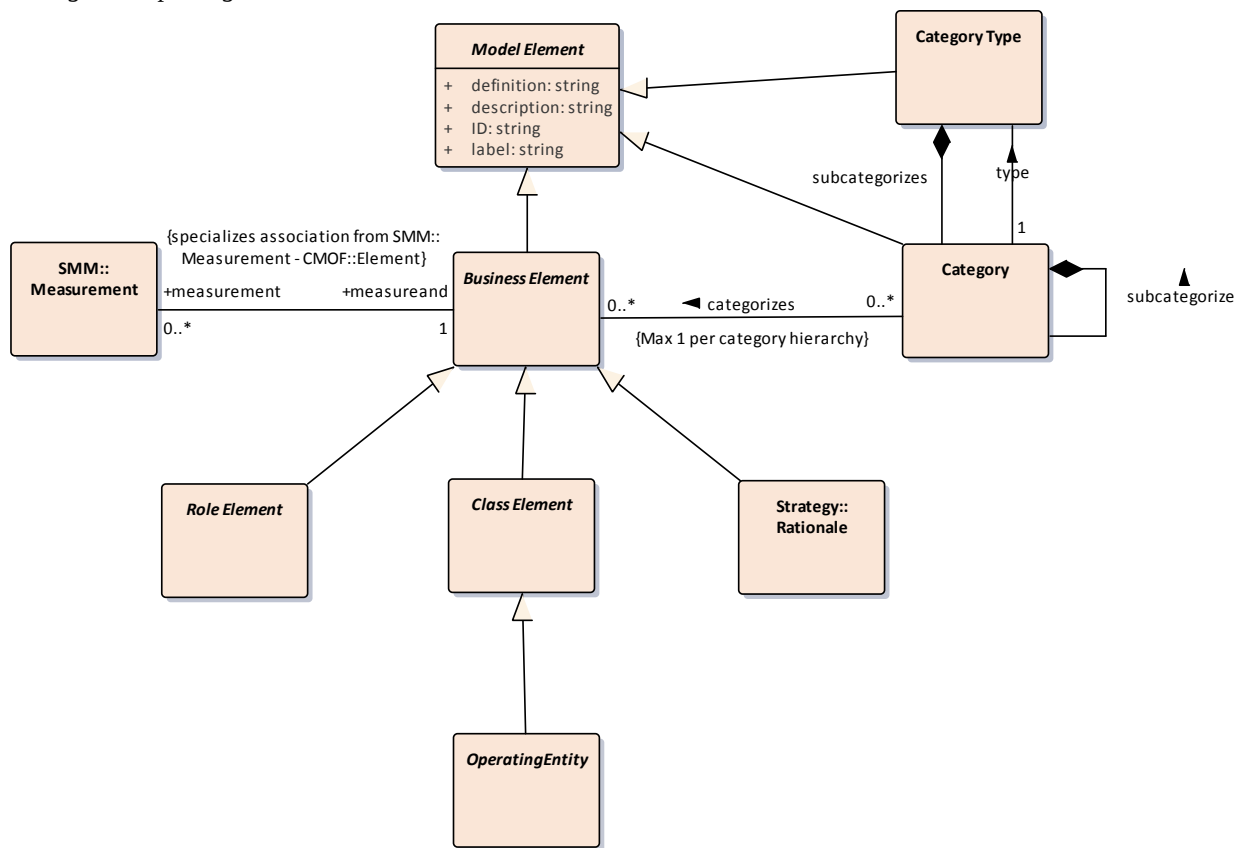


Figure 2: Foundations

The Foundations diagram presents the basic classes that are specialized or used in the business architecture metamodel. The base element in the specialization hierarchy is the `ModelElement`. This class has four attributes that are inherited by every other class in the metamodel.

The metamodel provides for the creation of any number of hierarchical structures that can be used to categorize `BusinessElement` model instances. A category hierarchy has a model instance of the `CategoryType` class as the root. The children are model instances of the `Category` class. Model instances of the `Category` class can also have children.

The metamodel does not provide an intrinsic semantic for a category hierarchy. However, the children of any category should be non-overlapping (disjoint). And, if a `BusinessElement` is associated with a `Category` model instance X, the `BusinessElement` should be assumed to also be associated with all parents of X.

Categories are intended for use by the business architect to mark business architecture `BusinessElements` in various ways during the construction and maintenance of the architecture model and its subsequent use in analysis projects. Categories are not intended to model features or aspects of the business or its strategies. For this reason, Categories specialize `ModelElement` rather than specializing `BusinessElement`.

`BusinessElement` is the abstract base class for all metamodel classes whose model elements are intended to model concepts, entities, structures and behaviors of the business.

`ClassElement` is the abstract base class for all metamodel classes intended to represent concepts, entities and behaviors of the business.

`RoleElement` is the abstract base class for all n-ary relationships that represent relationships in the structure and behavior of the business. `RoleElement` subclasses are defined with associations linking them to the `ClassElement` subclasses that participate in the relationship. It is often the case that `RoleElement` subclasses are labeled with a verb and their associations are labeled with linguistic structure category names such as "subject" and "object". The association names, together with the verb of the `RoleElement` specialization facilitate reading the portion of the diagram containing these elements into natural language prose. See the `Value` package for an example.

`Strategy::Rationale` is a `BusinessElement`, but not a `ClassElement` or a `RoleElement`. Conceptually, this class represents a potentially complex argument that supports a conclusion. It is used in the `Strategy` package to provide the business architect a way to capture the rationale of a strategy, specifically the rationale supporting the conclusion that an objective can be realized by following a course of action. The business architect can capture the rationale as a description or a definition attribute value in the business architecture model, or this element can be used as a proxy to an element in a separate, and more detailed model created by a separate tool.

The metamodel adopts the `OMG Structured Metrics Metamodel (SMM)` for associating measurements with `BusinessElements`.

The concepts, entities and behaviors of a business can often be distinguished by whether they are used to produce end products of the business, or whether they are used to guide the business and cause it to respond to changes in its environment. The former items are targets for strategy driven change. They are sometimes regarded as elements of the "operating model" of a business. The nature of the change, the expected outcomes of the change and the supporting rationale are represented by classes defined in the `Strategy` package of this metamodel. The `OperatingEntity` class is an abstract parent of all `ClassElements` in the metamodel that would be targets for strategy driven change.

Note that the metamodel is being expressed in UML. The actual metamodel is expressed in MOF and the UML model undergoes a transformation to produce the MOF model. The subclasses of the `RoleElement` class, together with their defined associations, are a UML pattern for representing n-ary MOF associations. MOF associations are also classifiers and can be specialized, carry attributes and methods, and participate in other associations.

Ordinary UML associations that are not part of a `RoleElement` association pattern are also translated into MOF associations.

Finally, MOF does not intrinsically provide inheritance at the model instance level. Consequently, the business architect would not be allowed to specify inheritance between model instances when creating a business architecture model. The ability to indicate inheritance is introduced for specific model elements by an association labeled "generalizes". The

instances of metamodel classes connected by this directional association should be assumed to be classes (i.e. they represent sets of individuals).

7.3.2 Class Elements diagram

Class diagram in package 'Foundations'

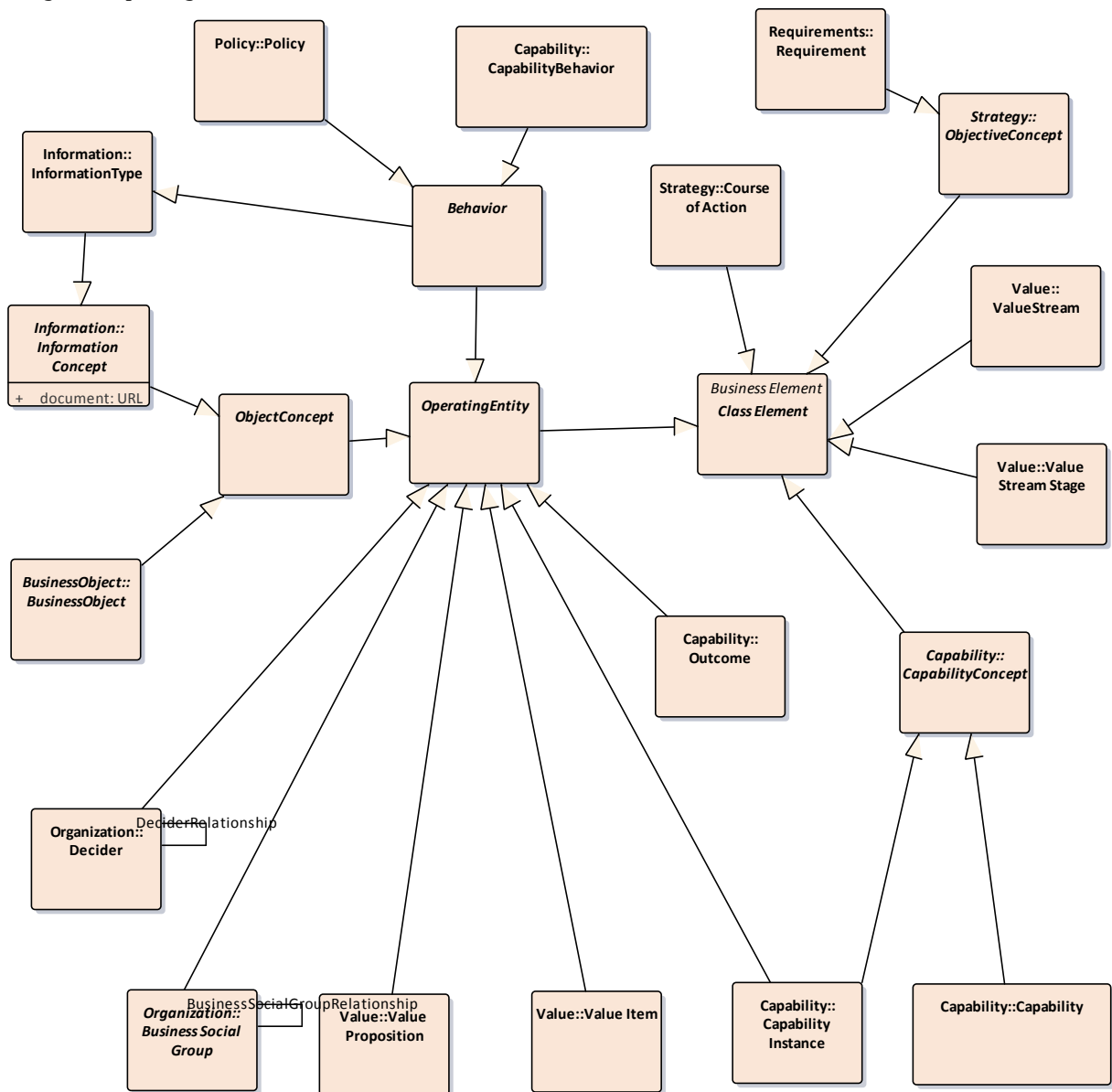


Figure 3: Class Elements

The ClassElements diagram documents the metamodel inheritance patterns for ClassElements. The OperatingEntity abstract class is the superclass for all classes in the model that are intended to represent business entities that are part of the operating model of the business. The significance of the operating model is that its entities are the targets for strategy driven change.

7.3.3 Classes:

7.3.3.1 Class Behavior

in package 'Foundations'

Behavior is an abstract class that unites Policy and CapabilityBehavior. Behaviors are controlled by Deciders and influenced by Policies that are controlled by Deciders.

Behaviors are also InformationTypes. This allows a Behavior to be expressed in a document that is a BusinessObject.

7.3.3.1.1 Superclasses

7.3.3.1.1.1 Class Foundations::OperatingEntity

7.3.3.1.1.2 Class Information::InformationType

7.3.3.1.2 Subclasses:

7.3.3.1.2.1 Class Capability::CapabilityBehavior

7.3.3.1.2.2 Class Policy::Policy

7.3.3.1.3 Associations:

7.3.3.1.3.1.1 Association:object Type:Association

Source: Organization::controls **Role:** **Cardinality:**

Target: Foundations::Behavior **Role:** **Cardinality:**

7.3.3.1.3.1.1.1 *Constraints:*

7.3.3.1.3.1.2 Association:object Type:Association

Source: Policy::influences **Role:** **Cardinality:**

Target: Foundations::Behavior **Role:** **Cardinality:**

7.3.3.1.3.1.2.1 *Constraints:*

7.3.3.1.4 Attributes:

7.3.3.1.5 Methods:

7.3.3.2 Class Business Element

in package 'Foundations'

The BusinessElement is an abstract superclass for all metamodel classes that are intended to model business entities, structures, concepts and behaviors.

7.3.3.2.1 Superclasses

7.3.3.2.1.1 Class Foundations::Model Element

7.3.3.2.2 Subclasses:

7.3.3.2.2.1 Class Foundations::Class Element

7.3.3.2.2.2 Class Strategy::Rationale

7.3.3.2.2.3 Class Foundations::Role Element

7.3.3.2.3 Associations:

7.3.3.2.3.1.1 Association:is about Type:Association

Source: BusinessObject::BusinessObject **Role:** **Cardinality:**

Target: Foundations::Business Element **Role:** **Cardinality:**

7.3.3.2.3.1.1.1 *Constraints:*

Constraint: Business Object makes explicit Information Concept is about Business Element Invariant

This "is about" association is a high level association that can be used to connect an information representing BusinessObject to the BusinessElement the information is about. When the model is developed to include InformationConcepts, for each instance of this association, there should be an instance of the association chain: BusinessObject makes explicit an InformationConcept that is about the BusinessElement.

7.3.3.2.3.1.2 Association:object Type:Association

Source: Information::is about **Role:** **Cardinality:**

Target: Foundations::Business Element **Role:** **Cardinality:**

7.3.3.2.3.1.2.1 *Constraints:*

7.3.3.2.3.1.3 Association:categorizes Type:Association

Source: Foundations::Category **Role:** **Cardinality:** 0..*

Target: Foundations::Business Element **Role:** **Cardinality:** 0..*

7.3.3.2.3.1.3.1 *Constraints:*

Constraint: Max 1 per category hierarchy Invariant

The "categorized" association links a Category model instance to the model instance of any BusinessElement.

7.3.3.2.3.1.4 Association: Type:Association

Source: SMM::Measurement **Role:** measurement **Cardinality:** 0..*

Target: Foundations::Business Element **Role:** measureand **Cardinality:** 1

7.3.3.2.3.1.4.1 *Constraints:*

Constraint: specializes association from SMM::Measurement - CMOF::Element Invariant

This association specializes the corresponding association from the SMM. ModelElement inherits from MOF Class, so BusinessElement is also a MOF Class.

7.3.3.2.3.1.5 Association: Type:Association

Source: BACM_Model::BusinessLibrary **Role:** **Cardinality:** 1

Target: Foundations::Business Element **Role:** **Cardinality:** 0..*

7.3.3.2.3.1.5.1 *Constraints:*

7.3.3.2.3.1.6 Association:localElement Type:Association

Source: Alignment::Assembler **Role:** **Cardinality:**

Target: Foundations::Business Element **Role:** **Cardinality:**

7.3.3.2.3.1.6.1 *Constraints:*

The localElement association links an Assembler model object to a model object instance of any concrete subclass of the BusinessElement class.

7.3.3.2.4 **Attributes:**

7.3.3.2.5 **Methods:**

7.3.3.3 **Class Category Type**

in package 'Foundations'

The CategoryType class represents the root of a categorization hierarchy. It is a specialization of ModelElement rather than BusinessElement because it is intended for use by the business architect in the process of creating and maintaining the business architecture model and/or performing analytic work based on the model. These categories should not be used to model business concepts or categories; classes of the BusinessObject and Information packages should be used instead.

7.3.3.3.1 **Superclasses**

7.3.3.3.1.1 **Class Foundations::Model Element**

7.3.3.3.2 **Subclasses:**

7.3.3.3.2.1 **Class Foundations::Category**

7.3.3.3.3 **Associations:**

7.3.3.3.3.1.1 Association:type Type:Association

Source: Foundations::Category **Role:** **Cardinality:** 1

Target: Foundations::Category Type **Role:** **Cardinality:**

7.3.3.3.3.1.1.1 *Constraints:*

7.3.3.3.3.1.2 Association:subcategorizes Type:Aggregation

Source: Foundations::Category **Role:** **Cardinality:**

Target: Foundations::Category Type **Role:** **Cardinality:**

7.3.3.3.3.1.2.1 *Constraints:*

7.3.3.3.4 **Attributes:**

7.3.3.3.5 **Methods:**

7.3.3.4 **Class ObjectConcept**

in package 'Foundations'

ObjectConcept is the abstract parent of InformationConcept and BusinessObject.

Note that InformationConcept represents intangibles, while BusinessObject represents tangibles. The term "business object" is often used to denote things that are important to a business and are acted upon by the business, without distinguishing whether they are tangible or intangible. However, creating a model requires the modeler to make a commitment as tangibles and intangibles have different semantic properties that are represented in the model by different attributes and associations,

7.3.3.4.1 Superclasses

7.3.3.4.1.1 Class **Foundations::OperatingEntity**

7.3.3.4.2 Subclasses:

7.3.3.4.2.1 Class **Information::Information Concept**

7.3.3.4.2.2 Class **BusinessObject::BusinessObject**

7.3.3.4.3 Associations:

7.3.3.4.3.1.1 Association:contains Type:Aggregation

Source: Foundations::ObjectConcept **Role:** **Cardinality:**

Target: Product::Product Concept **Role:** **Cardinality:**

7.3.3.4.3.1.1.1 *Constraints:*

7.3.3.4.3.1.2 Association:enables Type:Association

Source: Capability::Capability **Role:** **Cardinality:**

Target: Foundations::ObjectConcept **Role:** **Cardinality:**

7.3.3.4.3.1.2.1 *Constraints:*

Capability enables ObjectConcept represents the notion that the business must possess the capabilities needed to produce the components of the product and to satisfy any entitlements created by statute or contract.

This is a high level association that is consistent with Capabilities producing Outcomes that affect ObjectConcepts where there is a ValueItem for those Outcomes that addresses the purchaser's expectations as defined in the CustomerSegment.

7.3.3.4.3.1.3 Association:from Type:Association

Source: Capability::Derives **Role:** **Cardinality:**

Target: Foundations::ObjectConcept **Role:** **Cardinality:**

7.3.3.4.3.1.3.1 *Constraints:*

7.3.3.4.3.1.4 Association:from Type:Association

Source: Capability::Modifies **Role:** **Cardinality:**

Target: Foundations::ObjectConcept **Role:** **Cardinality:**

7.3.3.4.3.1.4.1 *Constraints:*

7.3.3.4.3.1.5 Association:values Type:Association

Source: Value::Value Item **Role:** **Cardinality:**

Target: Foundations::ObjectConcept **Role:** **Cardinality:**

7.3.3.4.3.1.5.1 *Constraints:*

This is a high level association representing the valuation of a BusinessObject or an InformationConcept by a ValueItem. In detail, this association implies that an Outcome valued by the same ValueItem affects the ObjectConcept in a manner that supports its valuation by the ValueItem.

7.3.3.4.3.1.6 Association:from Type:Association

Source: Capability::Assembled **Role:** **Cardinality:**

Target: Foundations::ObjectConcept **Role:** **Cardinality:**

7.3.3.4.3.1.6.1 *Constraints:*

7.3.3.4.4 Attributes:

7.3.3.4.5 Methods:

7.3.3.5 Class OperatingEntity

in package 'Foundations'

"Operating Entity" is an abstract parent class for a number of concrete metamodel classes that represent business concepts typically used to refer to business entities, concepts and activities that form the ongoing operations of the business. The ability to change is a key characteristic of human endeavors and is carried out by both executives and workers. Business change is modeled as a sequence of changes in an operating model - a configuration of the business that is assumed to be static between these changes.

NOTE: the concept of an operating model as a configuration of versions of operating entities is not currently defined in the metamodel.

7.3.3.5.1 Superclasses

7.3.3.5.1.1 Class Foundations::Class Element

7.3.3.5.2 Subclasses:

7.3.3.5.2.1 Class Organization::Decider

7.3.3.5.2.2 Class Organization::Business Social Group

7.3.3.5.2.3 Class Capability::Outcome

7.3.3.5.2.4 Class Foundations::Behavior

7.3.3.5.2.5 Class Capability::Capability Instance

7.3.3.5.2.6 Class Foundations::ObjectConcept

7.3.3.5.2.7 Class Value::Value Proposition

7.3.3.5.2.8 Class Value::Value Item

7.3.3.5.3 Associations:

7.3.3.5.3.1.1 Association:influences Type:Association

Source: Policy::Policy **Role:** **Cardinality:**

Target: Foundations::OperatingEntity **Role:** **Cardinality:**

7.3.3.5.3.1.1.1 *Constraints:*

While Policies directly influence behaviors, Policies are often specified as rules about the desired state of one or more OperatingEntities. In many cases, it is not necessary for the architect to directly model the behavior that will be influenced, but it is necessary to record the relationship between the Policy and the OperatingEntities whose state is described in the Policy. This "influences" association allows that relationship to be modeled.

If the influenced Behavior is later added to the model, it must be consistent with the associations to the OperatingEntities affected by or performing that Behavior.

7.3.3.5.3.1.2 Association:affects Type:Association

Source: Capability::CapabilityConcept **Role:** **Cardinality:**

Target: Foundations::OperatingEntity **Role:** **Cardinality:**

7.3.3.5.3.1.2.1 *Constraints:*

Constraint: produces Outcome that Affects OperatingEntity Invariant

Affects is a high level association used to associate a Capability with an OperatingEntity when the details of the Outcomes affecting the OperatingEntities are not yet described in the business architecture model.

These details must be compatible with the constraint of this association.

7.3.3.5.3.1.3 Association:before Type:Association

Source: Strategy::Change **Role:** **Cardinality:**

Target: Foundations::OperatingEntity **Role:** **Cardinality:**

7.3.3.5.3.1.3.1 *Constraints:*

7.3.3.5.3.1.4 Association:after Type:Association

Source: Strategy::Change **Role:** **Cardinality:**

Target: Foundations::OperatingEntity **Role:** **Cardinality:**

7.3.3.5.3.1.4.1 *Constraints:*

7.3.3.5.3.1.5 Association:object Type:Association

Source: Capability::Affects **Role:** **Cardinality:**

Target: Foundations::OperatingEntity **Role:** **Cardinality:**

7.3.3.5.3.1.5.1 *Constraints:*

7.3.3.5.4 Attributes:

7.3.3.5.5 Methods:

7.3.3.6 Class Category

in package 'Foundations'

The Category class represents subcategories in a category hierarchy. Subcategories are owned by their parent and should be assumed to be non-overlapping (disjoint). In addition, anything assumed to be the case for a subcategory should also be the case for the parent of that category.

7.3.3.6.1 Superclasses

7.3.3.6.1.1 Class Foundations::Model Element

7.3.3.6.2 Subclasses:

7.3.3.6.2.1 Class Foundations::Category

7.3.3.6.3 Associations:

7.3.3.6.3.1.1 Association:type Type:Association

Source: Foundations::Category **Role:** **Cardinality:** 1

Target: Foundations::Category Type **Role:** **Cardinality:**

7.3.3.6.3.1.1.1 *Constraints:*

7.3.3.6.3.1.2 Association:subcategorizes Type:Aggregation

Source: Foundations::Category **Role:** **Cardinality:**

Target: Foundations::Category **Role:** **Cardinality:**

7.3.3.6.3.1.2.1 *Constraints:*

7.3.3.6.3.1.3 Association:categorizes Type:Association

Source: Foundations::Category **Role:** **Cardinality:** 0..*

Target: Foundations::Business Element **Role:** **Cardinality:** 0..*

7.3.3.6.3.1.3.1 *Constraints:*

Constraint: Max 1 per category hierarchy Invariant

The "categorized" association links a Category model instance to the model instance of any BusinessElement.

7.3.3.6.3.1.4 Association:subcategorizes Type:Aggregation

Source: Foundations::Category **Role:** **Cardinality:**

Target: Foundations::Category Type **Role:** **Cardinality:**

7.3.3.6.3.1.4.1 *Constraints:*

7.3.3.6.3.1.5 Association:subcategorizes Type:Aggregation

Source: Foundations::Category **Role:** **Cardinality:**

Target: Foundations::Category **Role:** **Cardinality:**

7.3.3.6.3.1.5.1 *Constraints:*

7.3.3.6.3.1.6 Association: Type:Association

Source: BACM_Model::CategoryLibrary **Role:** **Cardinality:** 1

Target: Foundations::Category **Role:** **Cardinality:** 0..*

7.3.3.6.3.1.6.1 *Constraints:*

7.3.3.6.3.1.7 Association:stratifies Type:Association

Source: Capability::CapabilityConcept **Role:** **Cardinality:**

Target: Foundations::Category **Role:** **Cardinality:**

7.3.3.6.3.1.7.1 *Constraints:*

Constraint: specializes Foundations::categorizes Invariant

7.3.3.6.4 Attributes:

7.3.3.6.5 Methods:

7.3.3.7 Class Class Element

in package 'Foundations'

The ClassElement represents entities, concepts and behaviors of the business. It is the abstract parent of all the concrete metamodel classes representing these aspects of the business.

7.3.3.7.1 Superclasses

7.3.3.7.1.1 Class Foundations::Business Element

7.3.3.7.2 Subclasses:

7.3.3.7.2.1 Class Value::Value Stream Stage

7.3.3.7.2.2 Class Strategy::Course of Action

7.3.3.7.2.3 Class Foundations::OperatingEntity

7.3.3.7.2.4 Class Initiative::Initiative

7.3.3.7.2.5 Class Capability::CapabilityConcept

7.3.3.7.2.6 Class Strategy::ObjectiveConcept

7.3.3.7.2.7 Class Value::ValueStream

7.3.3.7.3 Associations:

7.3.3.7.4 Attributes:

7.3.3.7.5 Methods:

7.3.3.8 Class Model Element

in package 'Foundations'

ModelElement is the base abstract class for all elements of the business architecture metamodel. It provides each subclass with attributes that include a label, a unique ID and a description. An attribute is provided for a definition if the methodology supports or requires a formal definition such as an expression in a logic language.

7.3.3.8.1 Superclasses

7.3.3.8.2 Subclasses:

7.3.3.8.2.1 Class BACM_Model::StakeholderMap

7.3.3.8.2.2 Class Foundations::Category

7.3.3.8.2.3 Class BACM_Model::BACM_Model

7.3.3.8.2.4 Class BACM_Model::InitiativeMap

7.3.3.8.2.5 Class Alignment::Foreign Model

7.3.3.8.2.6 Class BACM_Model::StrategyMap

7.3.3.8.2.7 Class BACM_Model::CapabilityMap

7.3.3.8.2.8 Class BACM_Model::CategoryLibrary

- 7.3.3.8.2.9 **Class** **BACM_Model::InformationMap**
- 7.3.3.8.2.10 **Class** **BACM_Model::AlignmentLibrary**
- 7.3.3.8.2.11 **Class** **BACM_Model::RequirementMap**
- 7.3.3.8.2.12 **Class** **Alignment::Assembler**
- 7.3.3.8.2.13 **Class** **Foundations::Category Type**
- 7.3.3.8.2.14 **Class** **BACM_Model::ValueMap**
- 7.3.3.8.2.15 **Class** **Foundations::Business Element**
- 7.3.3.8.2.16 **Class** **BACM_Model::OrganizationMap**
- 7.3.3.8.2.17 **Class** **BACM_Model::PolicyMap**
- 7.3.3.8.2.18 **Class** **BACM_Model::ProductMap**
- 7.3.3.8.2.19 **Class** **BACM_Model::BusinessLibrary**
- 7.3.3.8.2.20 **Class** **BACM_Model::BusinessObjectMap**

7.3.3.8.3 Associations:

7.3.3.8.4 Attributes:

7.3.3.8.4.1.1 Attribute: Model Element.definition Type: string

This attribute holds a formal definition of the model element, typically in a logic language, when required by the architecture methodology.

7.3.3.8.4.1.2 Attribute: Model Element.description Type: string

This attribute is used to hold a description or informal definition of the model element.

7.3.3.8.4.1.3 Attribute: Model Element.ID Type: string

This attribute holds a unique identifier of the model element. To the extent possible, tools implementing this metamodel should strive to retain this ID over changes to the model in order to facilitate differential comparisons of two or more models where the models are derived from each other via a change process. The metamodel does not specify whether the ID is unique to a model, to a set of versions of a model or any other scope of uniqueness.

7.3.3.8.4.1.4 Attribute: Model Element.label Type: string

This attribute holds a label for the model element. The label does not need to be unique.

7.3.3.8.5 Methods:

7.3.3.9 Class Role Element

in package 'Foundations'

A RoleElement defines a role relationship between two or more ClassElement classes in the metamodel. Each subclass of the RoleElement class has a particular configuration of associations that define the role relationships between the RoleElement and the ClassElements that are the components of the relationship.

It is necessary sometimes to allow the business architect to define particular types of relationships whose roles cannot be statically defined by the metamodel. See the BusinessObject and Information packages for examples of this alternative method of defining relationships.

7.3.3.9.1 Superclasses

7.3.3.9.1.1 Class Foundations::Business Element

7.3.3.9.2 Subclasses:

7.3.3.9.2.1 Class Strategy::expected to result in

7.3.3.9.2.2 Class Value::Adds

7.3.3.9.2.3 Class Information::InformationRelationRole

7.3.3.9.2.4 Class BusinessObject::BusinessObjectRelationRole

7.3.3.9.2.5 Class Organization::accountable

7.3.3.9.2.6 Class Capability::Affects

7.3.3.9.2.7 Class Strategy::Change

7.3.3.9.2.8 Class Policy::influences

7.3.3.9.2.9 Class Information::is about

7.3.3.9.2.10 Class Organization::controls

7.3.3.9.2.11 Class Organization::staffs

7.3.3.9.2.12 Class Capability::Produce

7.3.3.9.2.13 Class Strategy::needs

7.3.3.9.2.14 Class Capability::Need

7.3.3.9.2.15 Class Capability::Characterizes

7.3.3.9.2.16 Class Capability::Realizes

7.3.3.9.3 Associations:

7.3.3.9.4 Attributes:

7.3.3.9.5 Methods:

7.4 Capability

Package in package 'Model'

7.4.1 Capability diagram

Class diagram in package 'Capability'

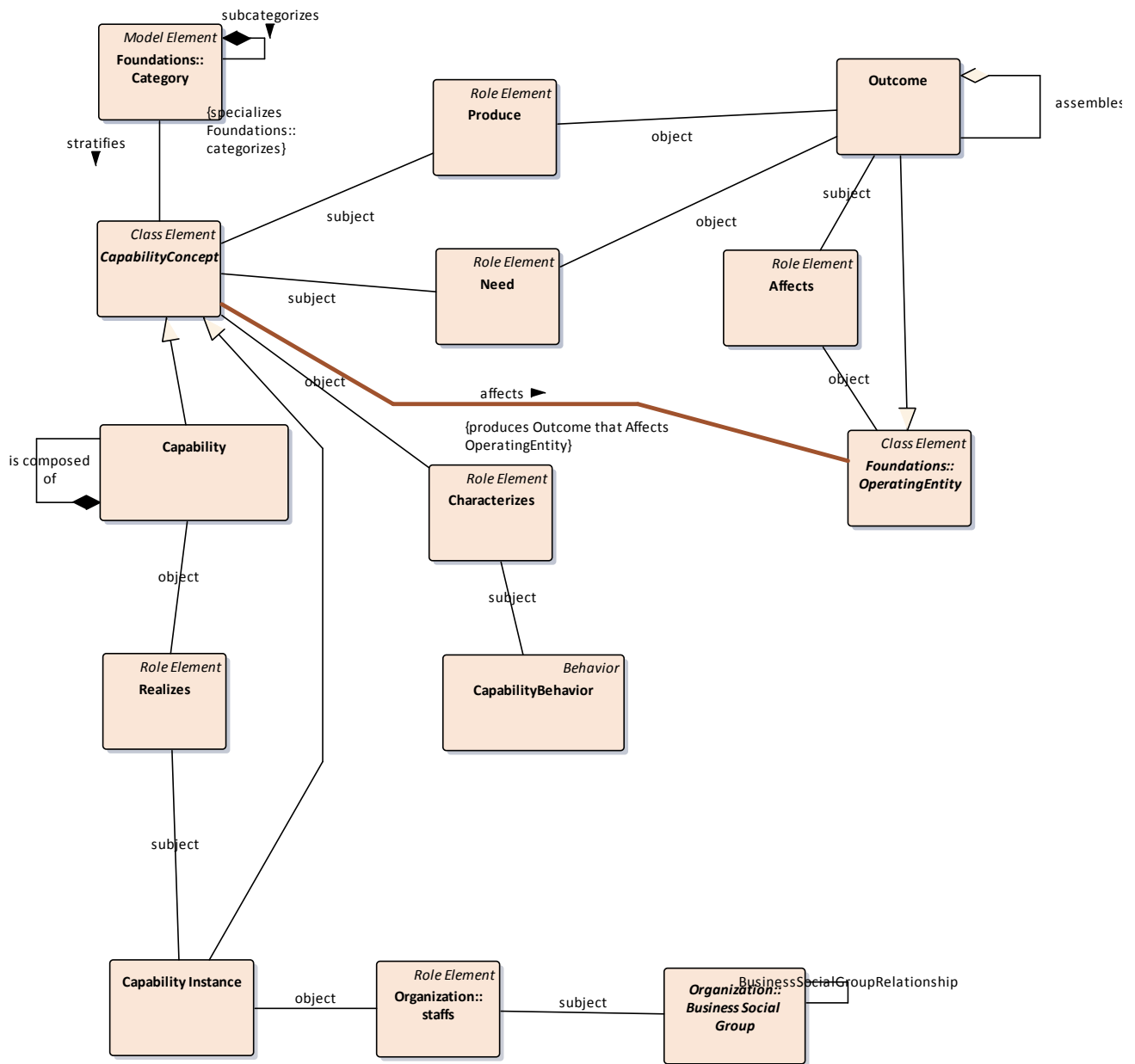


Figure 4: Capability

Capability Instance inherits from Capability so can produce and need outcomes. The Capability Instance is needed because a Capability can be defined only once in a Capability Map. However, a business may have or plan to have several instances of that capability. The design intent is to allow specialization of Capability Instances, Outcomes and Business Objects in order to add specificity to the Capability Map for both reference modeling and for individual business models.

7.4.2 Affects diagram

Class diagram in package 'Capability'

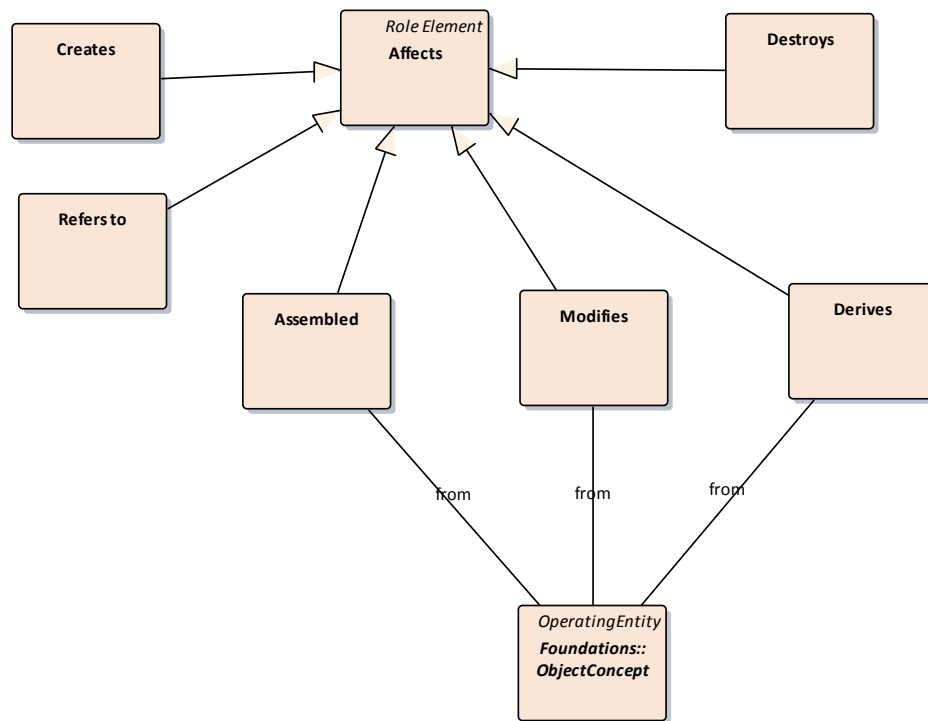


Figure 5: Affects

This diagram shows some useful specializations of the "affects" class. Three of these specializations ("Assembled", "Modifies", and "Derives") allow the representation of the source OperatingEntity that is transformed by the activity represented by these specializations.

7.4.3 Classes:

7.4.3.1 Class Affects

in package 'Capability'

Affects associates a business object with an outcome and indicates that the outcome has affected the state of the business object, e.g. by creating a business object, modifying a business object, changing an association in which the business object participates, or changing the environment in such a way that the interpretation of the business object is changed, or destroying the business object.

7.4.3.1.1 Superclasses

7.4.3.1.1.1 Class Foundations::Role Element

7.4.3.1.2 Subclasses:

7.4.3.1.2.1 Class Capability::Assembled

7.4.3.1.2.2 Class Capability::Draft::CapabilitySpecification

7.4.3.1.2.3 Class Capability::Destroys

7.4.3.1.2.4 Class Capability::Modifies

7.4.3.1.2.5 Class Capability::Derives

7.4.3.1.2.6 Class Capability::Creates

7.4.3.1.2.7 Class Capability::Refers to

7.4.3.1.3 Associations:

7.4.3.1.3.1.1 Association:object Type:Association

Source: Capability::Affects **Role:** **Cardinality:**

Target: Foundations::OperatingEntity **Role:** **Cardinality:**

7.4.3.1.3.1.1.1 *Constraints:*

7.4.3.1.3.1.2 Association:subject Type:Association

Source: Capability::Outcome **Role:** **Cardinality:**

Target: Capability::Affects **Role:** **Cardinality:**

7.4.3.1.3.1.2.1 *Constraints:*

7.4.3.1.4 Attributes:

7.4.3.1.5 Methods:

7.4.3.2 Class Assembled

in package 'Capability'

7.4.3.2.1 Superclasses

7.4.3.2.1.1 Class Capability::Affects

7.4.3.2.2 Subclasses:

7.4.3.2.3 Associations:

7.4.3.2.3.1.1 Association:from Type:Association

Source: Capability::Assembled **Role:** **Cardinality:**

Target: Foundations::ObjectConcept **Role:** **Cardinality:**

7.4.3.2.3.1.1.1 *Constraints:*

7.4.3.3 Attributes:

7.4.3.4 Methods:

7.4.3.5 Class Capability

in package 'Capability'

“A particular ability or capacity that a business may possess or exchange to achieve a specific purpose or outcome”, U. Homann¹⁹

7.4.3.6 Superclasses

7.4.3.6.1.1 Class Capability::CapabilityConcept

7.4.3.7 Subclasses:

7.4.3.7.1.1 Class Capability::Capability

7.4.3.8 Associations:

7.4.3.8.1.1.1 Association:enables Type:Association

Source: Capability::Capability **Role:** **Cardinality:**

Target: Product::Product Concept **Role:** **Cardinality:**

7.4.3.8.1.1.1.1 *Constraints:*

This association is a high level abstraction of the "enables" associations of the BusinessObjects and/or InformationConcepts contained in the Product or ProductLine with the Capabilities that produce them. The associations is valuable for aggregating and presenting information about the performance of Capabilities that enable the components of a product. It can also be used when the detailed components of a Product or ProductLine are not yet modeled.

7.4.3.8.1.1.2 Association:is composed of Type:Aggregation

Source: Capability::Capability **Role:** **Cardinality:**

Target: Capability::Capability **Role:** **Cardinality:**

7.4.3.8.1.1.2.1 *Constraints:*

7.4.3.8.1.1.3 Association:enables Type:Association

Source: Capability::Capability **Role:** **Cardinality:**

Target: Foundations::ObjectConcept **Role:** **Cardinality:**

7.4.3.8.1.1.3.1 *Constraints:*

Capability enables ObjectConcept represents the notion that the business must possess the capabilities needed to produce the components of the product and to satisfy any entitlements created by statute or contract.

This is a high level association that is consistent with Capabilities producing Outcomes that affect ObjectConcepts where there is a ValueItem for those Outcomes that addresses the purchaser's expectations as defined in the CustomerSegment.

7.4.3.8.1.1.4 Association:enables Type:Association

Source: Capability::Capability **Role:** **Cardinality:**

Target: Value::Value Stream Stage **Role:** **Cardinality:**

7.4.3.8.1.1.4.1 *Constraints:*

Capabilities enable a value stream stage in that the activities associated with the capability create outcomes that contribute to the value created by the value stream stage. This association models the concept of a capability enabling a value stream stage.

7.4.3.8.1.1.5 Association:object Type:Association

Source: Capability::Realizes **Role:** **Cardinality:**

Target: Capability::Capability **Role:** **Cardinality:**

7.4.3.8.1.1.5.1 *Constraints:*

7.4.3.8.1.1.6 Association:is composed of Type:Aggregation

Source: Capability::Capability **Role:** **Cardinality:**

Target: Capability::Capability **Role:** **Cardinality:**

7.4.3.8.1.1.6.1 *Constraints:*

7.4.3.8.1.1.7 Association:influences Type:Association

Source: Policy::Policy **Role:** **Cardinality:**

Target: Capability::Capability **Role:** **Cardinality:**

7.4.3.8.1.1.7.1 *Constraints:*

This influences association is an abstraction of the details of a Policy influencing an OperatingEntity, typically a CapabilityBehavior or a CapabilityInstance. Capability is not considered an operating entity because it is an abstract categorization of activities performed by a business and cannot be directly influenced by a policy. The association allows the architect to model the concept that a Policy influences behavior of a Capability without having to model the details of the behavior.

7.4.3.8.1.1.8 Association:impacts Type:Association

Source: Initiative::Initiative **Role:** **Cardinality:**

Target: Capability::Capability **Role:** **Cardinality:**

7.4.3.8.1.1.8.1 *Constraints:*

The **impacts** association generalizes the concept that the Initiative has an Objective to make **changes** in a CapabilityInstance or CapabilityBehavior associated with the target Capability. This association must be consistent with the details when present in a model.

7.4.3.8.1.1.9 Association:requires Type:Association

Source: Initiative::Initiative **Role:** **Cardinality:**

Target: Capability::Capability **Role:** **Cardinality:**

7.4.3.8.1.1.9.1 *Constraints:*

The **requires** association models the notion that a specific Capability is needed in order to execute the Initiative. This association is used in Initiative planning to establish requirements.

7.4.3.8.1.1.10 Association: Type:Association

Source: BACM_Model::CapabilityMap **Role:** **Cardinality:**

Target: Capability::Capability **Role:** **Cardinality:** 0..*

7.4.3.8.1.1.10.1 *Constraints:*

7.4.3.8.1.1.11 Association:supportCapability Type:Association

Source: Information::InformationType **Role:** **Cardinality:**

Target: Capability::Capability **Role:** **Cardinality:**

7.4.3.8.1.1.11.1 *Constraints:*

7.4.3.9 Attributes:

7.4.3.10 Methods:

7.4.3.11 Class Capability Instance

in package 'Capability'

Capability Instance is a specific realization of a capability as it exists or is envisioned to exist in the context of a given business unit, value stage stream, or other situational context.

The CapabilityInstance represents a bundle of real or virtual resources that have the ability to perform the Capability that is realized by the CapabilityInstance.

7.4.3.12 Superclasses

7.4.3.12.1.1 Class Capability::CapabilityConcept

7.4.3.12.1.2 Class Foundations::OperatingEntity

7.4.3.13 Subclasses:

7.4.3.14 Associations:

7.4.3.14.1.1.1 Association:subject Type:Association

Source: Capability::Realizes **Role:** **Cardinality:**

Target: Capability::Capability Instance **Role:** **Cardinality:**

7.4.3.14.1.1.1.1 *Constraints:*

7.4.3.14.1.1.2 Association:after Type:Association

Source: TOGAF::CapabilityIncrement **Role:** **Cardinality:**

Target: Capability::Capability Instance **Role:** **Cardinality:**

7.4.3.14.1.1.2.1 *Constraints:*

7.4.3.14.1.1.3 Association:object Type:Association

Source: Organization::staffs **Role:** **Cardinality:**

Target: Capability::Capability Instance **Role:** **Cardinality:**

7.4.3.14.1.1.3.1 *Constraints:*

7.4.3.14.1.1.4 Association:before Type:Association

Source: TOGAF::CapabilityIncrement **Role:** **Cardinality:**

Target: Capability::Capability Instance **Role:** **Cardinality:**

7.4.3.14.1.1.4.1 *Constraints:*

7.4.3.15 Attributes:

7.4.3.16 Methods:

7.4.3.17 Class CapabilityBehavior

in package 'Capability'

The CapabilityBehavior class represents behavior specifications such as procedures, process definitions, policies and norms. It models the "how" associated with Capability.

The behaviors represented by CapabilityBehavior can be either real or planned. Capabilities and CapabilityInstances can have behaviors that are represented by CapabilityBehaviors.

7.4.3.18 Superclasses

7.4.3.18.1.1 Class Foundations::Behavior

7.4.3.19 Subclasses:

7.4.3.20 Associations:

7.4.3.20.1.1.1 Association:after Type:Association

Source: Capability::CapabilityBehavior **Role:** **Cardinality:**

Target: TOGAF::CapabilityIncrement **Role:** **Cardinality:**

7.4.3.20.1.1.1.1 *Constraints:*

7.4.3.20.1.1.2 Association:subject_2 Type:Association

Source: Capability::Draft::specifies **Role:** **Cardinality:**

Target: Capability::CapabilityBehavior **Role:** **Cardinality:**

7.4.3.20.1.1.2.1 *Constraints:*

7.4.3.20.1.1.3 Association:subject Type:Association

Source: Capability::Characterizes **Role:** **Cardinality:**

Target: Capability::CapabilityBehavior **Role:** **Cardinality:**

7.4.3.20.1.1.3.1 *Constraints:*

7.4.3.20.1.1.4 Association: Type:Association

Source: Requirements::Requirement **Role:** **Cardinality:**

Target: Capability::CapabilityBehavior **Role:** **Cardinality:**

7.4.3.20.1.1.4.1 *Constraints:*

7.4.3.20.1.1.5 Association:before Type:Association

Source: TOGAF::CapabilityIncrement **Role:** **Cardinality:**

Target: Capability::CapabilityBehavior **Role:** **Cardinality:**

7.4.3.20.1.1.5.1 *Constraints:*

7.4.3.21 Attributes:

7.4.3.22 Methods:

7.4.3.23 Class CapabilityConcept

in package 'Capability'

CapabilityConcept is an abstract class that generalizes the concept that Capabilities need Outcomes and the concept that

Capabilities produce Outcomes, that Capabilities have varieties of specific behaviors (the CapabilityCharacteristic), and that Capabilities affect OperatingEntities.
This class is specialized in the concrete classes Capability and CapabilityInstance.

7.4.3.23.1 Superclasses

7.4.3.23.1.1 Class Foundations::Class Element

7.4.3.23.2 Subclasses:

7.4.3.23.2.1 Class Capability::Capability

7.4.3.23.2.2 Class Capability::Capability Instance

7.4.3.23.3 Associations:

7.4.3.23.3.1.1 Association:subject Type:Association

Source: Capability::CapabilityConcept **Role:** **Cardinality:**

Target: Capability::Need **Role:** **Cardinality:**

7.4.3.23.3.1.1.1 Constraints:

7.4.3.23.3.1.2 Association:before Type:Association

Source: Capability::CapabilityConcept **Role:** **Cardinality:**

Target: TOGAF::CapabilityIncrement **Role:** **Cardinality:**

7.4.3.23.3.1.2.1 Constraints:

7.4.3.23.3.1.3 Association:subject_1 Type:Association

Source: Capability::CapabilityConcept **Role:** **Cardinality:**

Target: Capability::Draft::specifies **Role:** **Cardinality:**

7.4.3.23.3.1.3.1 Constraints:

7.4.3.23.3.1.4 Association:affects Type:Association

Source: Capability::CapabilityConcept **Role:** **Cardinality:**

Target: Foundations::OperatingEntity **Role:** **Cardinality:**

7.4.3.23.3.1.4.1 Constraints:

Constraint: produces Outcome that Affects OperatingEntity Invariant

Affects is a high level association used to associate a Capability with an OperatingEntity when the details of the Outcomes affecting the OperatingEntities are not yet described in the business architecture model.

These details must be compatible with the constraint of this association.

7.4.3.23.3.1.5 Association:object Type:Association

Source: Capability::CapabilityConcept **Role:** **Cardinality:**

Target: Capability::Characterizes **Role:** **Cardinality:**

7.4.3.23.3.1.5.1 *Constraints:*

7.4.3.23.3.1.6 Association:stratifies Type:Association

Source: Capability::CapabilityConcept **Role:** **Cardinality:**

Target: Foundations::Category **Role:** **Cardinality:**

7.4.3.23.3.1.6.1 *Constraints:*

Constraint: specializes Foundations::categorizes Invariant

7.4.3.23.3.1.7 Association:subject Type:Association

Source: Capability::CapabilityConcept **Role:** **Cardinality:**

Target: Capability::Produce **Role:** **Cardinality:**

7.4.3.23.3.1.7.1 *Constraints:*

Capabilities may result in an outcome taking place or happening.

7.4.3.23.3.1.8 Association:contributes to Type:Association

Source: Stakeholder::Participating Stakeholder **Role:** **Cardinality:**

Target: Capability::CapabilityConcept **Role:** **Cardinality:**

7.4.3.23.3.1.8.1 *Constraints:*

This association abstracts the notion that a ParticipatingStakeholder contributes to Outcomes of a Capability that enables the ValueStreamStage. It is intended for use when Outcomes have not yet been defined in the Capability map or in presentations when the Outcomes of the Capability are obvious.

7.4.3.23.3.1.9 Association:after Type:Association

Source: TOGAF::CapabilityIncrement **Role:** **Cardinality:**

Target: Capability::CapabilityConcept **Role:** **Cardinality:**

7.4.3.23.3.1.9.1 *Constraints:*

7.4.3.24 Attributes:

7.4.3.25 Methods:

7.4.3.26 Class Characterizes

in package 'Capability'

The "Characterizes" class is the core element in a binary association pattern linking CapabilityConcept and CapabilityCharacteristic. This linkage represents the association of a behavior to either a Capability or a CapabilityInstance. A CapabilityCharacteristic would be associated with a Capability during strategic planning to define an intended new behavior for the Capability. During Initiative planning, the CapabilityCharacteristic would be associated with a CapabilityInstance that defines the resources expected to be needed by the CapabilityInstance when implemented.

Open Issue:

The concept represented by this association is not simple. In practice, the skills identified in the resources of the CapabilityInstance would need to be linked to the roles specified in the behavior descriptions of the CapabilityCharacteristic.

7.4.3.27 Superclasses

7.4.3.27.1.1 Class Foundations::Role Element

7.4.3.28 Subclasses:

7.4.3.29 Associations:

7.4.3.29.1.1.1 Association:subject Type:Association

Source: Capability::Characterizes **Role:** **Cardinality:**

Target: Capability::CapabilityBehavior **Role:** **Cardinality:**

7.4.3.29.1.1.1.1 *Constraints:*

7.4.3.29.1.1.2 Association:object Type:Association

Source: Capability::CapabilityConcept **Role:** **Cardinality:**

Target: Capability::Characterizes **Role:** **Cardinality:**

7.4.3.29.1.1.2.1 *Constraints:*

7.4.3.30 Attributes:

7.4.3.31 Methods:

7.4.3.32 Class Creates

in package 'Capability'

7.4.3.33 Superclasses

7.4.3.33.1.1 Class Capability::Affects

7.4.3.34 Subclasses:

7.4.3.35 Associations:

7.4.3.36 Attributes:

7.4.3.37 Methods:

7.4.3.38 Class Derives

in package 'Capability'

7.4.3.39 Superclasses

7.4.3.39.1.1 Class Capability::Affects

7.4.3.40 Subclasses:

7.4.3.41 Associations:

7.4.3.41.1.1.1 Association:from Type:Association

Source: Capability::Derives **Role:** **Cardinality:**

Target: Foundations::ObjectConcept **Role:** **Cardinality:**

7.4.3.41.1.1.1.1 *Constraints:*

7.4.3.42 Attributes:

7.4.3.43 Methods:

7.4.3.44 Class Destroys
in package 'Capability'

7.4.3.45 Superclasses

7.4.3.45.1.1 Class Capability::Affects

7.4.3.46 Subclasses:

7.4.3.47 Associations:

7.4.3.48 Attributes:

7.4.3.49 Methods:

7.4.3.50 Class Modifies
in package 'Capability'

7.4.3.51 Superclasses

7.4.3.51.1.1 Class Capability::Affects

7.4.3.52 Subclasses:

7.4.3.53 Associations:

7.4.3.53.1.1.1 Association:from Type:Association

Source: Capability::Modifies **Role:** **Cardinality:**

Target: Foundations::ObjectConcept **Role:** **Cardinality:**

7.4.3.53.1.1.1.1 *Constraints:*

7.4.3.54 Attributes:

7.4.3.55 Methods:

7.4.3.56 Class Need
in package 'Capability'

NeedOutcome is a RoleElement that is the head of an association pattern linking Capability and Outcome where the Capability may not be able to be executed unless the Outcome is present.

7.4.3.56.1 Superclasses

7.4.3.56.1.1 Class Foundations::Role Element

7.4.3.56.2 Subclasses:

7.4.3.56.3 Associations:

7.4.3.56.3.1.1 Association:object Type:Association

Source: Capability::Need **Role:** **Cardinality:**

Target: Capability::Outcome **Role:** **Cardinality:**

7.4.3.56.3.1.1.1 *Constraints:*

7.4.3.56.3.1.2 Association:subject Type:Association

Source: Capability::CapabilityConcept **Role:** **Cardinality:**

Target: Capability::Need **Role:** **Cardinality:**

7.4.3.56.3.1.2.1 *Constraints:*

7.4.3.56.4 Attributes:

7.4.3.56.5 Methods:

7.4.3.57 Class Outcome

in package 'Capability'

An end result or final product that is a consequence of an event, action, or a series of events/actions. Because a capability is generic, outcomes are similarly generic in that they represent a class of specific outcomes of a particular nature. In the context of a grocery business, a generic capability might be the displaying of produce. The outcome of this capability would be that the produce is displayed. The produce is the generic business object, whose prior outcome state was that the produce was held in a refrigerated space.

7.4.3.58 Superclasses

7.4.3.58.1.1 Class Foundations::OperatingEntity

7.4.3.58.2 Subclasses:

7.4.3.58.2.1 Class Capability::Outcome

7.4.3.58.3 Associations:

7.4.3.58.3.1.1 Association:after Type:Association

Source: Capability::Outcome **Role:** **Cardinality:**

Target: TOGAF::CapabilityIncrement **Role:** **Cardinality:**

7.4.3.58.3.1.1.1 *Constraints:*

7.4.3.58.3.1.2 Association:assembles Type:Aggregation

Source: Capability::Outcome **Role:** **Cardinality:**

Target: Capability::Outcome **Role:** **Cardinality:**

7.4.3.58.3.1.2.1 *Constraints:*

Outcomes may be decomposed subject to the constraint that the Outcomes in a decomposition may not be temporally inconsistent with each other

7.4.3.58.3.1.3 Association:subject Type:Association

Source: Capability::Outcome **Role:** **Cardinality:**

Target: Capability::Affects **Role:** **Cardinality:**

7.4.3.58.3.1.3.1 *Constraints:*

7.4.3.58.3.1.4 Association:contributes to Type:Association

Source: Stakeholder::Participating Stakeholder **Role:** **Cardinality:**

Target: Capability::Outcome **Role:** **Cardinality:**

7.4.3.58.3.1.4.1 *Constraints:*

This association captures the concept that a ParticipatingStakeholder contributes to the Outcomes of Capabilities that support the ValueStreamStage.

7.4.3.58.3.1.5 Association:object Type:Association

Source: Capability::Produce **Role:** **Cardinality:**

Target: Capability::Outcome **Role:** **Cardinality:**

7.4.3.58.3.1.5.1 *Constraints:*

A ProduceOutcome represents the occurrence of an outcome.

7.4.3.58.3.1.6 Association:values Type:Association

Source: Value::Value Item **Role:** **Cardinality:**

Target: Capability::Outcome **Role:** **Cardinality:**

7.4.3.58.3.1.6.1 *Constraints:*

This association represents one of the primary links between the outcomes of activities of the business and the value perceived for those outcomes by a would-be customer.

7.4.3.58.3.1.7 Association:object Type:Association

Source: Capability::Need **Role:** **Cardinality:**

Target: Capability::Outcome **Role:** **Cardinality:**

7.4.3.58.3.1.7.1 *Constraints:*

7.4.3.58.3.1.8 Association:before Type:Association

Source: TOGAF::CapabilityIncrement **Role:** **Cardinality:**

Target: Capability::Outcome **Role:** **Cardinality:**

7.4.3.58.3.1.8.1 *Constraints:*

7.4.3.58.3.1.9 Association:has entry condition of Type:Association

Source: Value::Value Stream Stage **Role:** **Cardinality:**

Target: Capability::Outcome **Role:** **Cardinality:**

7.4.3.58.3.1.9.1 *Constraints:*

An entry condition is a condition that must hold in order for the work of a ValueStreamStage to begin. Such a condition must be satisfied by an Outcome from a Capability that supports a preceding ValueStreamStage in the same ValueStream.

In the capability map, a Capability may need an Outcome (e.g. the availability of a part to configure into an assembly) that is produced by another Capability (e.g. Inventory Management). The metamodel defines the Outcome satisfying the entry condition as an Outcome needed by a Capability that enables the ValueStreamStage. This definition of the association between an entry condition and an Outcome simplifies the metamodel, compared with the initial definition, because it avoids a reference to a preceding ValueStreamStage in order to select the Capability that produces the Outcome.

This association is a high level association intended to be used in value/capability cross mapping when the capability map may not be complete. It is also used to simplify presentations intended to create understanding and drive decision making.

7.4.3.58.3.1.10 Association:has exit condition of Type:Association

Source: Value::Value Stream Stage **Role:** **Cardinality:**

Target: Capability::Outcome **Role:** **Cardinality:**

7.4.3.58.3.1.10.1 *Constraints:*

An exit condition is a condition that must be achieved in order for the ValueStreamStage to have finished its work. The exit condition is fulfilled by an Outcome of some Capability that supports the ValueStreamStage. This association represents the linkage between the Outcome and the ValueStreamStage whose exit condition the Outcome satisfies.

7.4.3.58.3.1.11 Association:assembles Type:Aggregation

Source: Capability::Outcome **Role:** **Cardinality:**

Target: Capability::Outcome **Role:** **Cardinality:**

7.4.3.58.3.1.11.1 *Constraints:*

Outcomes may be decomposed subject to the constraint that the Outcomes in a decomposition may not be temporally inconsistent with each other

7.4.3.58.3.1.12 Association:subject_3 Type:Association

Source: Capability::Draft::specifies **Role:** **Cardinality:**

Target: Capability::Outcome **Role:** **Cardinality:**

7.4.3.58.3.1.12.1 *Constraints:*

7.4.3.58.4 **Attributes:**

7.4.3.58.5 **Methods:**

7.4.3.59 **Class Produce**

in package 'Capability'

ProduceOutcome is a RoleElement that is part of a pattern that models a relationship between a Capability and an

Outcome that can result from performing the Capability.

This relationship has arity 2 and connects a capability to an outcome produced by the capability. The relationship is modeled as a class to allow it to have attributes and participate in other relationships.

7.4.3.59.1 Superclasses

7.4.3.59.1.1 Class Foundations::Role Element

7.4.3.59.2 Subclasses:

7.4.3.59.3 Associations:

7.4.3.59.3.1.1 Association:object Type:Association

Source: Capability::Produce **Role:** **Cardinality:**

Target: Capability::Outcome **Role:** **Cardinality:**

7.4.3.59.3.1.1.1 *Constraints:*

A ProduceOutcome represents the occurrence of an outcome.

7.4.3.59.3.1.2 Association:has exit condition of Type:Association

Source: Value::Value Stream Stage **Role:** **Cardinality:**

Target: Capability::Produce **Role:** **Cardinality:**

7.4.3.59.3.1.2.1 *Constraints:*

7.4.3.59.3.1.3 Association:subject Type:Association

Source: Capability::CapabilityConcept **Role:** **Cardinality:**

Target: Capability::Produce **Role:** **Cardinality:**

7.4.3.59.3.1.3.1 *Constraints:*

Capabilities may result in an outcome taking place or happening.

7.4.3.59.4 Attributes:

7.4.3.59.5 Methods:

7.4.3.60 Class Realizes

in package 'Capability'

Specifies realization of - this role element associates a capability instance that realizes a capability in the context of a business.

7.4.3.60.1 Superclasses

7.4.3.60.1.1 Class Foundations::Role Element

7.4.3.60.2 Subclasses:

7.4.3.60.3 Associations:

7.4.3.60.3.1.1 Association:object Type:Association

Source: Capability::Realizes **Role:** **Cardinality:**

Target: Capability::Capability **Role:** **Cardinality:**

7.4.3.60.3.1.1.1 *Constraints:*

7.4.3.60.3.1.2 Association:subject Type:Association

Source: Capability::Realizes **Role:** **Cardinality:**

Target: Capability::Capability Instance **Role:** **Cardinality:**

7.4.3.60.3.1.2.1 *Constraints:*

7.4.3.60.3.1.3 Association:staffs Type:Association

Source: Capability::Realizes **Role:** **Cardinality:**

Target: Draft::Organization_2::Organization **Role:** **Cardinality:**

7.4.3.60.3.1.3.1 *Constraints:*

7.4.3.60.4 Attributes:

7.4.3.60.5 Methods:

7.4.3.61 Class Refers to *in package 'Capability'*

A specialization of "Affects" representing a non-transformational, non-destructive use of an OperatingEntity by a Capability.

7.4.3.61.1 Superclasses

7.4.3.61.1.1 Class Capability::Affects

7.4.3.61.2 Subclasses:

7.4.3.61.3 Associations:

7.4.3.61.4 Attributes:

7.4.3.61.5 Methods:

7.5 Value

Package in package 'Model'

7.5.1 Value diagram

Class diagram in package 'Value'

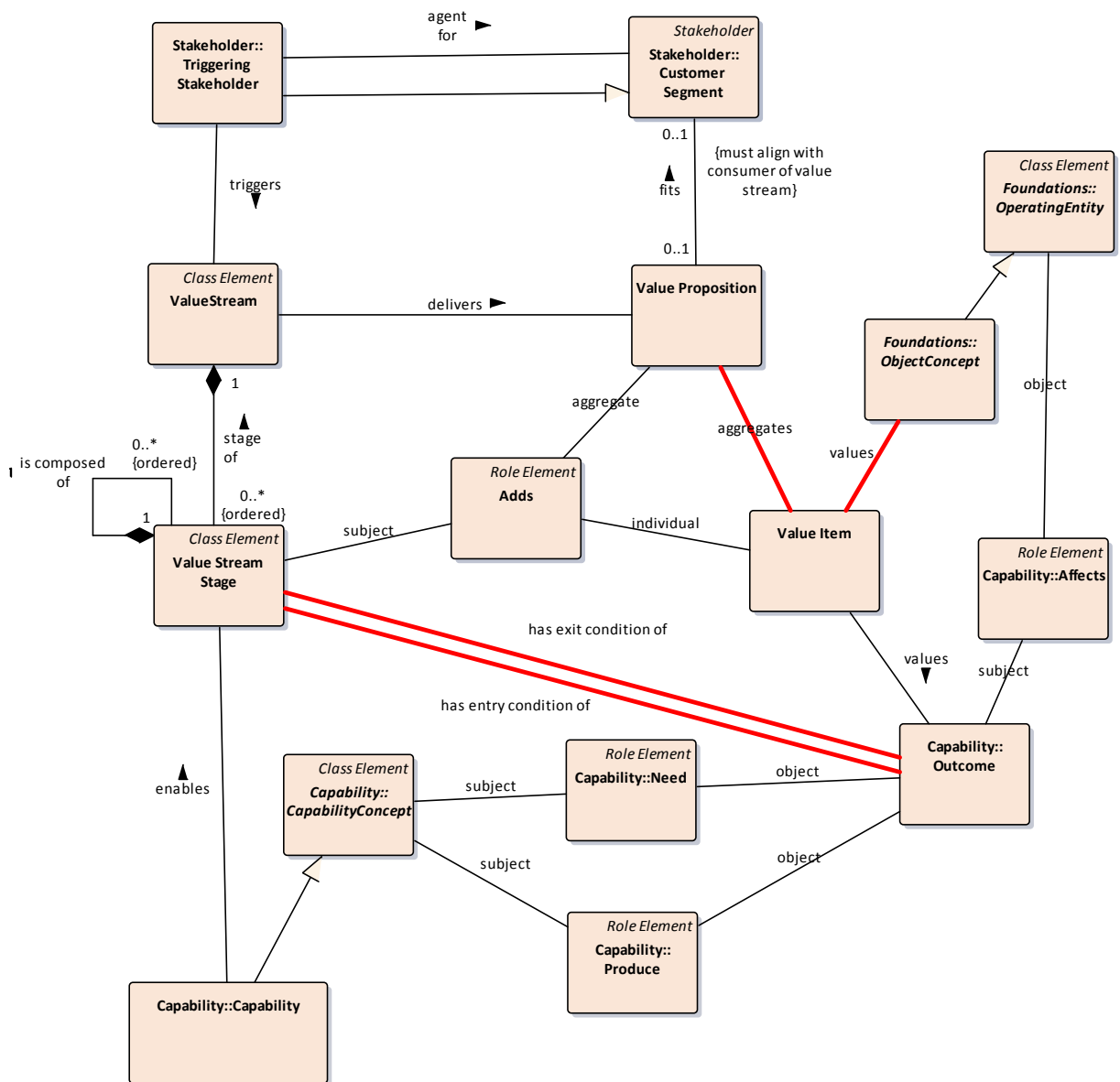


Figure 6: Value

Value is generated for a customer segment by a value stream. The value stream consists of a sequence of value stages such that each stage adds some value items to the value proposition of the value stream. The value proposition represents the value the business intends to provide, and usually does not completely satisfy the value requirements represented by the customer segment. For analysis, a value proposition may be matched with several customer segments in order to determine the degree to which the value proposition satisfies the value requirements of the customer segment.

It is sometimes the case that a product is purchased by one person and another person realizes the benefits of the product. Examples include toys purchased by adults for children, and life and medical insurance policies. To model this, a triggering stakeholder may be distinct from a customer segment. In this case, the "agent for" association establishes the link between the triggering stakeholder who makes the purchase and the customer segment who realizes the value. The value proposition must be matched with the value requirements of both the triggering stakeholder and the customer segment. In the normal case, the triggering stakeholder and the customer segment are the same.

Value stream stages also aggregate capabilities that are material to creating outcomes that provide the value indicated by the value items that make up the value proposition. The metamodel separates these outcomes from the value they provide because the same outcome may have different value perceptions to different customer segments.

In the process of creating a business architecture, many details may not yet be known. The metamodel provides some high level associations that can be used when details are not known. These associations are shown as thick red lines in the diagram. For example, a value proposition may value a product. As the business architecture model is detailed, the product contains a business object produced as the outcome of a capability that supports a value stream stage. The chain of model elements and associations used to model the fact in this statement represent the detail underlying the "value" association linking the value proposition to the product. The abstract associations shown as thick red lines have constraints that define the details supporting them. Tool vendors are expected to check these constraints.

7.5.2 Classes:

7.5.2.1 Class Adds

in package 'Value'

Adds is a class that represents a 3-ary association. The association models the addition of a ValueItem to a ValueProposition by the Capabilities of a ValueStreamStage.

The three legs of the association are:

subject - the actor that causes the addition;

individual - the ValueItem that is created by the subject and added to the aggregate

aggregate - the ValueProposition to which the ValueItem is added by the ValueStreamStage.

7.5.2.1.1 Superclasses

7.5.2.1.1.1 Class Foundations::Role Element

7.5.2.1.2 Subclasses:

7.5.2.1.3 Associations:

7.5.2.1.3.1.1 Association:aggregate Type:Association

Source: Value::Adds **Role:** **Cardinality:**

Target: Value::Value Proposition **Role:** **Cardinality:**

7.5.2.1.3.1.1.1 *Constraints:*

7.5.2.1.3.1.2 Association:individual Type:Association

Source: Value::Adds **Role:** **Cardinality:**

Target: Value::Value Item **Role:** **Cardinality:**

7.5.2.1.3.1.2.1 *Constraints:*

7.5.2.1.3.1.3 Association:subject Type:Association

Source: Value::Value Stream Stage **Role:** **Cardinality:**

Target: Value::Adds **Role:** **Cardinality:**

7.5.2.1.3.1.3.1 *Constraints:*

7.5.2.1.4 Attributes:

7.5.2.1.5 Methods:

7.5.2.2 Class Value Item

in package 'Value'

The judgement of worth, made by an individual or organization, attached to something tangible or intangible and attained in the course of a particular interaction with one or more parties (BIZBOK® Guide v6.5, Section 2.4)

In a value stream, a type of value map, value is defined by a value item, which is an abstraction that represents how a stakeholder receives value from the outcomes of one or more capabilities associated with a value stream stage.

(BIZBOK® Guide v6.5, Section 2.4)

The value item is an assessment of the value of an outcome produced by a value stream stage in the context of a Consumer Segment. The characteristics of the Consumer Segment determine the characteristics of the Value Item.

7.5.2.2.1 Superclasses

7.5.2.2.1.1 Class Foundations::OperatingEntity

7.5.2.2.2 Subclasses:

7.5.2.2.3 Associations:

7.5.2.2.3.1.1 Association:values Type:Association

Source: Value::Value Item **Role:** **Cardinality:**

Target: Capability::Outcome **Role:** **Cardinality:**

7.5.2.2.3.1.1.1 Constraints:

This association represents one of the primary links between the outcomes of activities of the business and the value perceived for those outcomes by a would-be customer.

7.5.2.2.3.1.2 Association:values Type:Association

Source: Value::Value Item **Role:** **Cardinality:**

Target: Foundations::ObjectConcept **Role:** **Cardinality:**

7.5.2.2.3.1.2.1 Constraints:

This is a high level association representing the valuation of a BusinessObject or an InformationConcept by a ValueItem. In detail, this association implies that an Outcome valued by the same ValueItem affects the ObjectConcept in a manner that supports its valuation by the ValueItem.

7.5.2.2.3.1.3 Association:delivers Type:Association

Source: Initiative::Initiative **Role:** **Cardinality:**

Target: Value::Value Item **Role:** **Cardinality:**

7.5.2.2.3.1.3.1 Constraints:

The **delivers** association models the concept that an Objective of the Initiative delivers a new version of a ValueItem (i.e. the ValueItem is the result **after** the **change** implied by the Objective).

7.5.2.2.3.1.4 Association:individual Type:Association

Source: Value::Adds **Role:** **Cardinality:**

Target: Value::Value Item **Role:** **Cardinality:**

7.5.2.2.3.1.4.1 Constraints:

7.5.2.2.3.1.5 Association:aggregates Type:Association

Source: Value::Value Proposition **Role:** **Cardinality:**

Target: Value::Value Item **Role:** **Cardinality:**

7.5.2.2.3.1.5.1 *Constraints:*

This association is an abstraction of the concept that a ValueStreamStage adds a ValueItem to a ValueProposition. It can be used to simplify presentations where the linkage to the ValueStreamStage is not material.

7.5.2.2.4 Attributes:

7.5.2.2.5 Methods:

7.5.2.3 Class Value Proposition

in package 'Value'

7.5.2.3.1 Superclasses

7.5.2.3.1.1 **Class Foundations::OperatingEntity**

7.5.2.3.2 Subclasses:

7.5.2.3.3 Associations:

7.5.2.3.3.1.1 Association:fits Type:Association

Source: Value::Value Proposition **Role:** **Cardinality:** 0..1

Target: Stakeholder::Customer Segment **Role:** **Cardinality:** 0..1

7.5.2.3.3.1.1.1 *Constraints:*

Constraint: must align with consumer of value stream Invariant

7.5.2.3.3.1.2 Association:guides Type:Association

Source: Value::Value Proposition **Role:** **Cardinality:**

Target: Requirements::Requirement **Role:** **Cardinality:**

7.5.2.3.3.1.2.1 *Constraints:*

A Requirement is intended to (indirectly) affect a ValueProposition in the context of achieving a strategy Objective. The Requirement is said to be "guided" by the new ValueProposition.

BIZBOK Reference:

Per Section: Benefits of BA and Requirements Alignment

Requirement > links to > Business (Value) Proposition

7.5.2.3.3.1.3 Association:aggregates Type:Association

Source: Value::Value Proposition **Role:** **Cardinality:**

Target: Value::Value Item **Role:** **Cardinality:**

7.5.2.3.3.1.3.1 *Constraints:*

This association is an abstraction of the concept that a ValueStreamStage adds a ValueItem to a ValueProposition. It can be used to simplify presentations where the linkage to the ValueStreamStage is not material.

7.5.2.3.3.1.4 Association:delivers Type:Association

Source: Value::ValueStream **Role:** **Cardinality:**

Target: Value::Value Proposition **Role:** **Cardinality:**

7.5.2.3.3.1.4.1 *Constraints:*

7.5.2.3.3.1.5 Association:aggregate Type:Association

Source: Value::Adds **Role:** **Cardinality:**

Target: Value::Value Proposition **Role:** **Cardinality:**

7.5.2.3.3.1.5.1 *Constraints:*

7.5.2.3.3.1.6 Association:values Type:Association

Source: Product::Product Concept **Role:** **Cardinality:**

Target: Value::Value Proposition **Role:** **Cardinality:**

7.5.2.3.3.1.6.1 *Constraints:*

This is a high level association linking a value proposition to a product. If details are added to the business architecture, they must satisfy this association's constraint that the ValueItems added to the ValueProposition must value the Product or ProductLine.

7.5.2.3.4 Attributes:

7.5.2.3.5 Methods:

7.5.2.4 Class Value Stream Stage

in package 'Value'

A distinct, Identifiable phase or step within a value stream that has a unique name, entrance criteria, exit criteria, and identifiable participating stakeholders(s). (BIZBOK® Guide v6.5, Section 2.4)

A distinct, identifiable phase or step within a ValueStream that produces some ValueItems.

For example, each value stream stage creates value for one or more stakeholders as it moves from left to right. While one can argue that a value stream to authorize a driver's license for an applicant is most valuable at the end state, consider that there is value in allowing the application to pass the vision exam. The applicant is informed that he or she has the visual ability to get a license and has additionally learned that his or her vision is not impaired. (BIZBOK® Guide v6.5, Section 2.4)

Value stream stages are owned by a value stream or a value stream stage and are not shared. It will often be the case that capabilities will support several value streams. The trade audit capability in a bank supports both the bank customer's value stream and the bank shareholder's value stream by reducing mistakes and fraud. The business architect must model this situation by sharing capabilities between value stream stages.

Because capabilities are often associated with significant business objects or concepts and represent compositions of more detailed capabilities, it is possible that the same capability may support several stages in a value stream. For example, a case management capability in an insurance business may support multiple value stream stages in the claims processing value stream. This usually occurs when the capability has not been decomposed into sub-capabilities to represent the different activities that occur in different value stream stages. Consequently, when a capability "supports" a value stream stage it is implied that at least one sub-capability of that capability also supports the value stream stage.

NOTE: an instance of ValueStreamStage cannot be simultaneously parented by both a ValueStream and another ValueStreamStage.

7.5.2.4.1 Superclasses

7.5.2.4.1.1 Class Foundations::Class Element

7.5.2.4.2 Subclasses:

7.5.2.4.2.1 Class Value::Value Stream Stage

7.5.2.4.3 Associations:

7.5.2.4.3.1.1 Association:bounds Type:Association

Source: Value::Value Stream Stage **Role:** **Cardinality:**

Target: Requirements::Requirement **Role:** **Cardinality:**

7.5.2.4.3.1.1.1 Constraints:

It should be rare that a Requirement affects changes in Outcomes associated with multiple ValueStreamStages of a ValueStream. The Requirement should be considered for splitting when this situation occurs. The "bounds" association allows the modeler to track the occurrence of such situations.

BIZBOK® Guide v6.5, Section 3.8 Reference:

Per Section: Principle of BA and Requirements Alignment

Requirements > bounded by > Value Stream Stage

7.5.2.4.3.1.2 Association:has exit condition of Type:Association

Source: Value::Value Stream Stage **Role:** **Cardinality:**

Target: Capability::Produce **Role:** **Cardinality:**

7.5.2.4.3.1.2.1 Constraints:

7.5.2.4.3.1.3 Association:has entry condition of Type:Association

Source: Value::Value Stream Stage **Role:** **Cardinality:**

Target: Capability::Outcome **Role:** **Cardinality:**

7.5.2.4.3.1.3.1 Constraints:

An entry condition is a condition that must hold in order for the work of a ValueStreamStage to begin. Such a condition must be satisfied by an Outcome from a Capability that supports a preceding ValueStreamStage in the same ValueStream.

In the capability map, a Capability may need an Outcome (e.g. the availability of a part to configure into an assembly) that is produced by another Capability (e.g. Inventory Management) . The metamodel defines the Outcome satisfying the entry condition as an Outcome needed by a Capability that enables the ValueStreamStage. This definition of the association between an entry condition and an Outcome simplifies the metamodel, compared with the initial definition, because it avoids a reference to a preceding ValueStreamStage in order to select the Capability that produces the Outcome.

This association is a high level association intended to be used in value/capability cross mapping when the capability

map may not be complete. It is also used to simplify presentations intended to create understanding and drive decision making.

7.5.2.4.3.1.4 Association:is composed of Type:Aggregation

Source: Value::Value Stream Stage **Role:** **Cardinality:** 0..*

Target: Value::Value Stream Stage **Role:** **Cardinality:** 1

7.5.2.4.3.1.4.1 *Constraints:*

This association decomposes a value stream stage into an ordered sequence of value stream stages.

7.5.2.4.3.1.5 Association:subject Type:Association

Source: Value::Value Stream Stage **Role:** **Cardinality:**

Target: Value::Adds **Role:** **Cardinality:**

7.5.2.4.3.1.5.1 *Constraints:*

7.5.2.4.3.1.6 Association:relies on Type:Association

Source: Value::Value Stream Stage **Role:** **Cardinality:**

Target: Product::Product Concept **Role:** **Cardinality:**

7.5.2.4.3.1.6.1 *Constraints:*

This association is a high level abstraction of the "enables" association between the ProductConcept and the Capabilities that "support" the ValueStreamStage. A ProductConcept relies on a ValueStreamStage when the ProductConcept contains ObjectConcepts that are enabled by Capabilities that support the ValueStreamStage

7.5.2.4.3.1.7 Association:has exit condition of Type:Association

Source: Value::Value Stream Stage **Role:** **Cardinality:**

Target: Capability::Outcome **Role:** **Cardinality:**

7.5.2.4.3.1.7.1 *Constraints:*

An exit condition is a condition that must be achieved in order for the ValueStreamStage to have finished its work. The exit condition is fulfilled by an Outcome of some Capability that supports the ValueStreamStage. This association represents the linkage between the Outcome and the ValueStreamStage whose exit condition the Outcome satisfies.

7.5.2.4.3.1.8 Association:stage of Type:Aggregation

Source: Value::Value Stream Stage **Role:** **Cardinality:** 0..*

Target: Value::ValueStream **Role:** **Cardinality:** 1

7.5.2.4.3.1.8.1 *Constraints:*

This composition association decomposes a value stream into value stream stages. The value stream orders the stages.

7.5.2.4.3.1.9 Association:contributes to Type:Association

Source: Stakeholder::Participating Stakeholder **Role:** **Cardinality:**

Target: Value::Value Stream Stage **Role:** **Cardinality:**

7.5.2.4.3.1.9.1 *Constraints:*

This "contributes to" high level association links a participating stakeholder to a value stream stage. It should be used to capture a significant contributing stakeholder to a value stream stage when there is no capability map or it is not yet defined. The constraint of this association defines compatible expressions of details involving the capabilities and their support for the value stream stage,

7.5.2.4.3.1.10 Association:impacts Type:Association

Source: Initiative::Initiative **Role:** **Cardinality:**

Target: Value::Value Stream Stage **Role:** **Cardinality:**

7.5.2.4.3.1.10.1 *Constraints:*

The **impacts** association generalizes the concept that the Initiative achieves an Objective of making a **change** to a ValueItem that is produced in a ValueStream (see Strategy and Value packages). Such an association instance must be consistent with model details when such details are present in the model.

7.5.2.4.3.1.11 Association:is composed of Type:Aggregation

Source: Value::Value Stream Stage **Role:** **Cardinality:** 0..*

Target: Value::Value Stream Stage **Role:** **Cardinality:** 1

7.5.2.4.3.1.11.1 *Constraints:*

This association decomposes a value stream stage into an ordered sequence of value stream stages.

7.5.2.4.3.1.12 Association:enables Type:Association

Source: Capability::Capability **Role:** **Cardinality:**

Target: Value::Value Stream Stage **Role:** **Cardinality:**

7.5.2.4.3.1.12.1 *Constraints:*

Capabilities enable a value stream stage in that the activities associated with the capability create outcomes that contribute to the value created by the value stream stage. This association models the concept of a capability enabling a value stream stage.

7.5.2.4.4 **Attributes:**

7.5.2.4.5 **Methods:**

7.5.2.5 **Class ValueStream**

in package 'Value'

An end-to-end collection of activities that create a result for a customer, who may be the ultimate customer or an internal end-user of the value stream.²⁰

A value stream has one clear goal, to satisfy or to delight the customer.²¹

Value streams strive to achieve the triggering stakeholder's value proposition as stated in the name of the value stream. (BIZBOK® Guide v6.5, Section 2.4)

A value stream is associated with ("delivers") some value propositions, one for each customer segment that is taken to be a stakeholder for the value stream.

Value streams may be decomposed into ordered sequences of value stream stages, and value stream stages may also be decomposed into ordered sequences of stages. This allows the architect to control the complexity of value stream

diagrams. The metamodel does not limit the levels of decomposition, but good practice is that decomposition of value streams should be limited to 2 levels. Note that a value stream stage is not a value stream and is not associated with a triggering stakeholder or a value proposition. Note also that a value stream is not supported by capabilities.

7.5.2.5.1 Superclasses

7.5.2.5.1.1 Class Foundations::Class Element

7.5.2.5.2 Subclasses:

7.5.2.5.2.1 Class Value::Value Stream Stage

7.5.2.5.3 Associations:

7.5.2.5.3.1.1 Association:delivers Type:Association

Source: Value::ValueStream **Role:** **Cardinality:**

Target: Value::Value Proposition **Role:** **Cardinality:**

7.5.2.5.3.1.1.1 Constraints:

7.5.2.5.3.1.2 Association:establishes scope for Type:Association

Source: Value::ValueStream **Role:** **Cardinality:**

Target: Requirements::Requirement **Role:** **Cardinality:**

7.5.2.5.3.1.2.1 Constraints:

A Requirement is intended to assist in fulfilling a strategic Objective. Typically, the Objective is framed in terms of affecting ValuePropositions associated with ValueStreams. Thus, the requirement has a scope established by one or more ValueStreams.

BIZBOK® Guide v6.5, Section 3.8 Reference:

Per Section: Benefits of BA and Requirements Alignment

Value stream > establishes scope for > requirement

Value Stream > clarifies how value is delivered by > requirement

Value streams > frame the context for > requirements

Value stream > enables reuse of > requirements

Per Section: Requirements Alignment with Value Stream

Requirement > is contextualized by > Value Stream

Per Section: BA and Requirements Alignment in Practice

Requirement > links to > Value Stream

7.5.2.5.3.1.3 Association:triggers Type:Association

Source: Value::ValueStream **Role:** **Cardinality:**

Target: Stakeholder::Triggering Stakeholder **Role:** **Cardinality:**

7.5.2.5.3.1.3.1 Constraints:

The "triggers" association links a stakeholder to the ValueStream that is triggered by the stakeholder. A stakeholder may

trigger the value stream directly (e.g. by asking to purchase an item) or indirectly (e.g. as in a loan default, where the triggering stakeholder establishes a rule or policy for triggering the value stream). The latter situation is called "triggering by proxy" in the BIZBOK® Guide.

7.5.2.5.3.1.4 Association: Type:Association

Source: BACM_Model::ValueMap **Role:** **Cardinality:**

Target: Value::ValueStream **Role:** **Cardinality:** 0..*

7.5.2.5.3.1.4.1 Constraints:

7.5.2.5.3.1.5 Association:relies on Type:Association

Source: Product::Product Concept **Role:** **Cardinality:**

Target: Value::ValueStream **Role:** **Cardinality:**

7.5.2.5.3.1.5.1 Constraints:

7.5.2.5.3.1.6 Association:triggers Type:Association

Source: Stakeholder::Stakeholder **Role:** **Cardinality:**

Target: Value::ValueStream **Role:** **Cardinality:**

7.5.2.5.3.1.6.1 Constraints:

7.5.2.5.3.1.7 Association:stage of Type:Aggregation

Source: Value::Value Stream Stage **Role:** **Cardinality:** 0..*

Target: Value::ValueStream **Role:** **Cardinality:** 1

7.5.2.5.3.1.7.1 Constraints:

This composition association decomposes a value stream into value stream stages. The value stream orders the stages.

7.5.2.5.4 Attributes:

7.5.2.5.5 Methods:

7.6 Information

Package in package 'Model'

7.6.1 Information diagram

Class diagram in package 'Information'

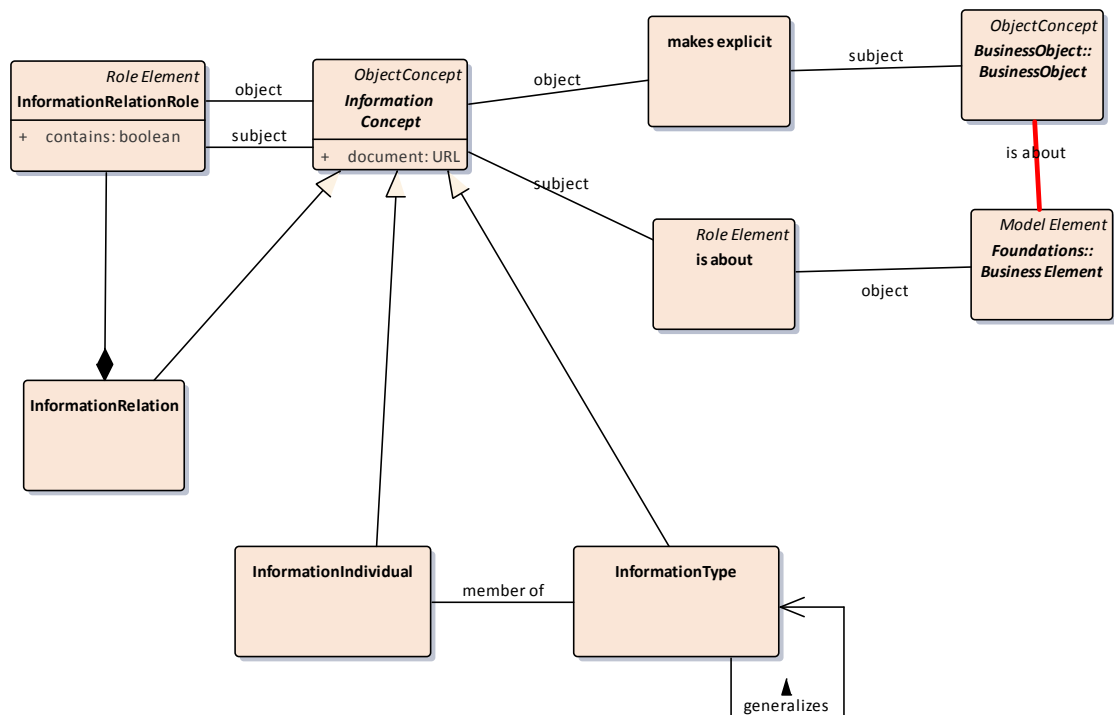


Figure 7: Information

This diagram describes the classes used to model information in a business architecture. Information is used to make decisions, a mental process. Consequently, information is treated as a tacit (mental) entity in this metamodel.

Information may also have an explicit (physical) representation in a business. Such a representation could be printed document or an electronic dataset containing visual or electronic symbols that can be interpreted by humans to transfer information.

This transfer is known to be imperfect because: 1) the explicit representation is only a part of the tacit information; 2) the writer chooses the symbols and syntactic arrangements of them in a way that is to some extent personal; 3) the reader perceives these symbols and syntactic arrangements in a way that is also partially personal.

The same information may be made explicit in many different forms. And, not all of the information that is used in a business is ever made explicit.

The metamodel provides InformationConcepts to model tacit information and BusinessObjects to model explicit representations of information. These model classes are connected by a relationship pattern modeled by instantiating the "makes explicit" class and the "subject" and "object" associations that are part of the pattern and indicate the directionality of the linkage.

Information is usually about some aspect of the business such as the location of parts in a warehouse, dollar volumes of sales transactions, amount of a price increase by a competitor. This is intended to be modeled by instantiating the "is about" association pattern, consisting of the "is about" class and the directional linking associations "subject" and "object". The association pattern links the InformationConcept to the BusinessElement the information is about.

7.6.2 Classes:

7.6.2.1 Class Information Concept in package 'Information'

InformationConcept is an abstract parent for the three concrete information classes that are used to model information types, information individuals and information relationships. It provides associations that are common to all three concrete subclasses:

- That a BusinessObject may make an InformationConcept explicit (tangible);
- The an InformationConcept is about a BusinessElement;
- That InformationConcepts may be linked by an InformationRelation.

7.6.2.1.1 Superclasses

7.6.2.1.1.1 Class Foundations::ObjectConcept

7.6.2.1.2 Subclasses:

7.6.2.1.2.1 Class Product::Entitlement

7.6.2.1.2.2 Class Information::InformationRelation

7.6.2.1.2.3 Class Product::Product Concept

7.6.2.1.2.4 Class Information::InformationType

7.6.2.1.2.5 Class Information::InformationIndividual

7.6.2.1.3 Associations:

7.6.2.1.3.1.1 Association:improves categorization of Type:Association

Source: Information::Information Concept **Role:** **Cardinality:**

Target: Requirements::Requirement **Role:** **Cardinality:**

7.6.2.1.3.1.1.1 Constraints:

BIZBOK Reference:

Per Section: Benefits of BA and Requirements Alignment

information > establishes scope for > requirement

Information > improves categorization of > requirements

Per Section: Requirements Alignment with Information

Requirement > links to > Information (Concepts)

7.6.2.1.3.1.2 Association: Type:Association

Source: BACM_Model::InformationMap **Role:** **Cardinality:**

Target: Information::Information Concept **Role:** **Cardinality:** 0..*

7.6.2.1.3.1.2.1 Constraints:

7.6.2.1.3.1.3 Association:subject Type:Association

Source: Information::is about **Role:** **Cardinality:**

Target: Information::Information Concept **Role:** **Cardinality:**

7.6.2.1.3.1.3.1 Constraints:

7.6.2.1.3.1.4 Association:object Type:Association

Source: Information::InformationRelationRole **Role:** **Cardinality:**

Target: Information::Information Concept **Role:** **Cardinality:**

7.6.2.1.3.1.4.1 *Constraints:*

7.6.2.1.3.1.5 Association:object Type:Association

Source: Information::makes explicit **Role:** **Cardinality:**

Target: Information::Information Concept **Role:** **Cardinality:**

7.6.2.1.3.1.5.1 *Constraints:*

7.6.2.1.3.1.6 Association:subject Type:Association

Source: Information::InformationRelationRole **Role:** **Cardinality:**

Target: Information::Information Concept **Role:** **Cardinality:**

7.6.2.1.3.1.6.1 *Constraints:*

7.6.2.1.4 Attributes:

7.6.2.1.4.1.1 Attribute: Information Concept.document Type: URL

This attribute of the InformationConcept provides one or more URLs that should reference electronic documents which make the InformationConcept explicit. It is semantically equivalent to instanting the "makes explicit" association pattern, but does not require instanting a BusinessObject to model the electronic document. It should be used when the architect does not see a justification to model the document as a BusinessObject.

7.6.2.1.5 Methods:

7.6.2.2 Class InformationIndividual

in package 'Information'

An InformationIndividual identifies a particular BusinessElement. It is effectively an information key for the corresponding BusinessElement.

InformationIndividuals can be members of an InformationType. This allows the InformationIndividuals to be constrained to identifying selected categories of BusinessElements without having to create individuals for the BusinessElements. For example, a business may have facilities at four locations. The concept of a facility is modeled as a BusinessObjectType and corresponding a InformationType is about the facility. Four InformationIndividuals (e.g. Paris, London, Beijing, and Singapore) are members of the corresponding InformationType

7.6.2.2.1 Superclasses

7.6.2.2.1.1 Class Information::Information Concept

7.6.2.2.2 Subclasses:

7.6.2.2.3 Associations:

7.6.2.2.3.1.1 Association:member of Type:Association

Source: Information::InformationType **Role:** **Cardinality:**

Target: Information::InformationIndividual **Role:** **Cardinality:**

7.6.2.2.3.1.1.1 *Constraints:*

7.6.2.2.4 Attributes:

7.6.2.2.5 Methods:

7.6.2.3 Class InformationRelation

in package 'Information'

InformationRelation and InformationRelationRole that it composes is a pattern that is intended to represent a relationship of variable arity between information concepts. It should be instantiated as an association with an arity determined by the architect. The InformationRelationshipRole class is instantiated to provide role names for each of the links in the association.

7.6.2.3.1 Superclasses

7.6.2.3.1.1 Class Information::Information Concept

7.6.2.3.2 Subclasses:

7.6.2.3.2.1 Class Information::InformationRelationRole

7.6.2.3.3 Associations:

7.6.2.3.3.1.1 Association: Type:Aggregation

Source: Information::InformationRelationRole **Role:** **Cardinality:**

Target: Information::InformationRelation **Role:** **Cardinality:**

7.6.2.3.3.1.1.1 *Constraints:*

7.6.2.3.4 Attributes:

7.6.2.3.5 Methods:

7.6.2.4 Class InformationRelationRole

in package 'Information'

The InformationRelationshipRole element provides a unique identity to each of the links in a variable arity association instanced from the pattern consisting of the InformationRelationship class, the InformationRelationshipRole class and the "subject" and "object" associations.

This class can be marked to indicate whether the role has the containment property (i.e. the object InformationConcept has no existence independent of the existence of the parent InformationConcept).

7.6.2.4.1 Superclasses

7.6.2.4.1.1 Class Foundations::Role Element

7.6.2.4.2 Subclasses:

7.6.2.4.2.1 Class Policy::relatesTo

7.6.2.4.3 Associations:

7.6.2.4.3.1.1 Association:object Type:Association

Source: Information::InformationRelationRole **Role:** **Cardinality:**

Target: Information::Information Concept **Role:** **Cardinality:**

7.6.2.4.3.1.1.1 *Constraints:*

7.6.2.4.3.1.2 Association:subject Type:Association

Source: Information::InformationRelationRole **Role:** **Cardinality:**

Target: Information::Information Concept **Role:** **Cardinality:**

7.6.2.4.3.1.2.1 *Constraints:*

7.6.2.4.3.1.3 Association: Type:Aggregation

Source: Information::InformationRelationRole **Role:** **Cardinality:**

Target: Information::InformationRelation **Role:** **Cardinality:**

7.6.2.4.3.1.3.1 *Constraints:*

7.6.2.4.4 Attributes:

7.6.2.4.4.1.1 Attribute: InformationRelationRole.contains Type: boolean

Indicates that the subject InformationConcept contains the object InformationConcept when true.

7.6.2.4.5 Methods:

7.6.2.5 Class InformationType

in package 'Information'

The InformationType class is intended to represent categories (sets) of information things. In English, such things are denoted by the absence of an article and/or the use of the plural. For example, the phrase "monthly sales by region" denotes a category of information consisting of sales amounts in a default currency for an unspecified month paired with an identifier for a sales region.

7.6.2.5.1 Superclasses

7.6.2.5.1.1 **Class Information::Information Concept**

7.6.2.5.2 Subclasses:

7.6.2.5.2.1 **Class Foundations::Behavior**

7.6.2.5.3 Associations:

7.6.2.5.3.1.1 Association:member of Type:Association

Source: Information::InformationType **Role:** **Cardinality:**

Target: Information::InformationIndividual **Role:** **Cardinality:**

7.6.2.5.3.1.1.1 *Constraints:*

7.6.2.5.3.1.2 Association:supportCapability Type:Association

Source: Information::InformationType **Role:** **Cardinality:**

Target: Capability::Capability **Role:** **Cardinality:**

7.6.2.5.3.1.2.1 *Constraints:*

7.6.2.5.3.1.3 Association:generalizes Type:Association

Source: Information::InformationType **Role:** **Cardinality:**

Target: Information::InformationType **Role:** **Cardinality:**

7.6.2.5.3.1.3.1 *Constraints:*

InformationTypes can be arranged in directed acyclic graphs (DAGs) such that all of the individuals that are members of an InformationType are also members of the supertypes of that InformationType.

The "specializes" association, when instantiated in a model, carries the property described above. In UML notation, this association at the model/instance level would be typically represented as an association with an open triangle at the end specified by the navigation direction of this meta-association.

7.6.2.5.3.1.4 Association:generalizes Type:Association

Source: Information::InformationType **Role:** **Cardinality:**

Target: Information::InformationType **Role:** **Cardinality:**

7.6.2.5.3.1.4.1 *Constraints:*

InformationTypes can be arranged in directed acyclic graphs (DAGs) such that all of the individuals that are members of an InformationType are also members of the supertypes of that InformationType.

The "specializes" association, when instantiated in a model, carries the property described above. In UML notation, this association at the model/instance level would be typically represented as an association with an open triangle at the end specified by the navigation direction of this meta-association.

7.6.2.5.4 Attributes:

7.6.2.5.5 Methods:

7.6.2.6 Class is about

in package 'Information'

InformationConcepts are often "about" other business entities that are represented as BusinessElements in an instance model. This class is the core element in a pattern representing that relationship between a BusinessElement and InformationConcepts that are about that BusinessElement. For example, a value of a financial instrument is an InformationConcept that is "about" a BusinessObject representing the type of financial instrument.

7.6.2.6.1 Superclasses

7.6.2.6.1.1 Class Foundations::Role Element

7.6.2.6.2 Subclasses:

7.6.2.6.3 Associations:

7.6.2.6.3.1.1 Association:subject Type:Association

Source: Information::is about **Role:** **Cardinality:**

Target: Information::Information Concept **Role:** **Cardinality:**

7.6.2.6.3.1.1.1 *Constraints:*

7.6.2.6.3.1.2 Association:object Type:Association

Source: Information::is about **Role:** **Cardinality:**

Target: Foundations::Business Element **Role:** **Cardinality:**

7.6.2.6.3.1.2.1 *Constraints:*

7.6.2.6.4 Attributes:

7.6.2.6.5 Methods:

7.6.2.7 Class makes explicit
in package 'Information'

The makes explicit class is the core element in a binary relationship pattern linking an InformationConcept and a BusinessObject.

Information can be made explicit by encoding it in visual or other symbols used to convey information. Information that has been encoded in this way is modeled by creating an InformationConcept instance to represent the information and a BusinessObject to represent the encoding of that information (e.g. as a printed report or an electronic data set). These model elements are linked by an instance of the "makes explicit" class and the subject and object linking associations.

7.6.2.7.1 Superclasses

7.6.2.7.2 Subclasses:

7.6.2.7.3 Associations:

7.6.2.7.3.1.1 Association:object Type:Association

Source: Information::makes explicit **Role:** **Cardinality:**

Target: Information::Information Concept **Role:** **Cardinality:**

7.6.2.7.3.1.1.1 *Constraints:*

7.6.2.7.3.1.2 Association:subject Type:Association

Source: Information::makes explicit **Role:** **Cardinality:**

Target: BusinessObject::BusinessObject **Role:** **Cardinality:**

7.6.2.7.3.1.2.1 *Constraints:*

7.6.2.7.4 Attributes:

7.6.2.7.5 Methods:

7.7 Organization

Package in package 'Model'

7.7.1 Organization diagram

Class diagram in package 'Organization'

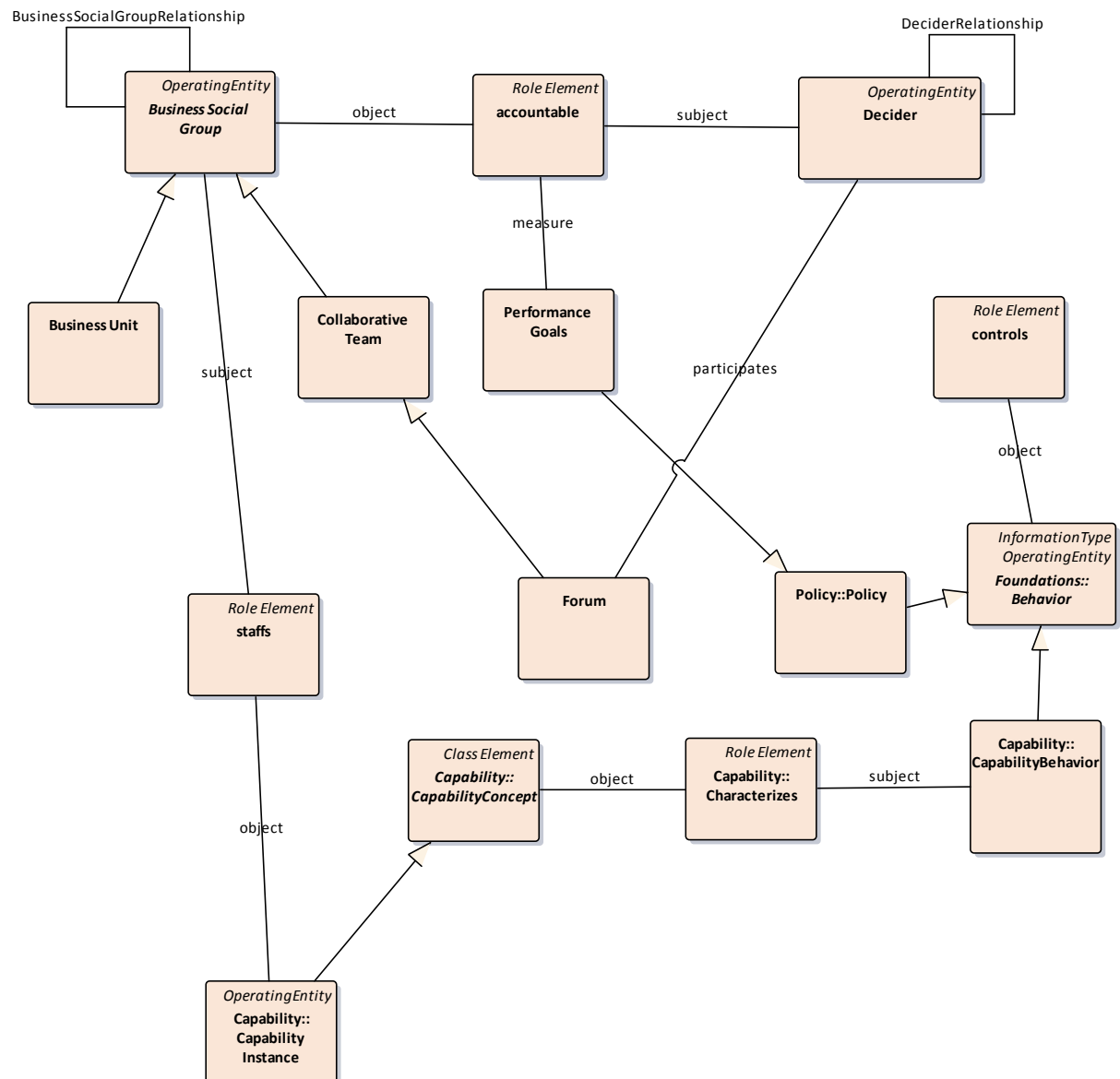


Figure 8: Organization

The Organization diagram displays the primary classes and relationships of the organization map. The organization package distinguishes groups and decision makers, but these concepts are not disjoint; a given individual may occupy both.

The decision makers are represented by the Decider class. This class has three primary relationships:

1. Control - this represents the control a decision maker has over how work is performed; it is modeled by a relationship pattern headed by the "controls" RoleElement. This relationship involves the Behaviors that define the work to be performed by way of procedures, policies and norms.
2. Accountability - this represents the accountability the decision maker has for achieving established performance measures. The accountability concept is modeled by the relationship pattern headed by the "accountable" RoleElement.

3. Participation in decision making Forums used to resolve issues requiring tradeoffs between different areas of decision maker control and accountability for the benefit of the business as a whole.

The BusinessSocialGroup represents the human resources that have been organized into a team for some purpose. A typical purpose is to staff a CapabilityInstance. CapabilityInstances may have defined behaviors, represented by CapabilityBehaviors.

Deciders can be related to each other. Typical relationships are subordination and collaboration.

BusinessSocialGroups can also be related to each other in a variety of ways, including composition, collaboration and friendship. The architect should be careful when using this association to maintain consistency with other model details such as when distinct BusinessSocialGroups staff CapabilityInstances whose Behaviors cause them to act on the same BusinessObject or when an Outcome produced by one group is needed by the other. Such cases are effectively collaborations that are required by business operations and not collaborations established by choice of the groups involved.

7.7.2 Classes:

7.7.2.1 Class Business Social Group

in package 'Organization'

A Business Social Group is a group of one or more individual persons who collaborate to achieve a specific outcome. A group may be defined to consist of a single individual as in the case of a singular executive such as a CEO. The group may exist or be projected to exist as the result of a strategy driven business transformation. When a group is defined but the individuals who belong to that group are not yet assigned or known, the group defines a role.

7.7.2.1.1 Superclasses

7.7.2.1.1.1 Class Foundations::OperatingEntity

7.7.2.1.2 Subclasses:

7.7.2.1.2.1 Class Draft::Organization_2::CollaborativeTeam

7.7.2.1.2.2 Class Organization::External Unit

7.7.2.1.2.3 Class Organization::Collaborative Team

7.7.2.1.2.4 Class Draft::Organization_2::BusinessUnit

7.7.2.1.2.5 Class Organization::Worker

7.7.2.1.2.6 Class Organization::Business Unit

7.7.2.1.2.7 Class Stakeholder::Stakeholder

7.7.2.1.3 Associations:

7.7.2.1.3.1.1 Association:subject Type:Association

Source: Organization::Business Social Group **Role:** **Cardinality:**

Target: Organization::staffs **Role:** **Cardinality:**

7.7.2.1.3.1.1.1 Constraints:

7.7.2.1.3.1.2 Association:BusinessSocialGroupRelationship Type:Association

Source: Organization::Business Social Group **Role:** **Cardinality:**

Target: Organization::Business Social Group **Role:** **Cardinality:**

7.7.2.1.3.1.2.1 Constraints:

The BusinessSocialGroupRelationship represents associations of various types between BusinessSocialGroups. This association may be specialized to represent particular types of relationships such as organizational composition, formal and informal coordination and so forth.

7.7.2.1.3.1.3 Association:object Type:Association

Source: Organization::accountable **Role:** **Cardinality:**

Target: Organization::Business Social Group **Role:** **Cardinality:**

7.7.2.1.3.1.3.1 Constraints:

7.7.2.1.3.1.4 Association: Type:Association

Source: BACM_Model::OrganizationMap **Role:** **Cardinality:**

Target: Organization::Business Social Group **Role:** **Cardinality:** 0..*

7.7.2.1.3.1.4.1 Constraints:

7.7.2.1.3.1.5 Association:BusinessSocialGroupRelationship Type:Association

Source: Organization::Business Social Group **Role:** **Cardinality:**

Target: Organization::Business Social Group **Role:** **Cardinality:**

7.7.2.1.3.1.5.1 Constraints:

The BusinessSocialGroupRelationship represents associations of various types between BusinessSocialGroups. This association may be specialized to represent particular types of relationships such as organizational composition, formal and informal coordination and so forth.

7.7.2.1.4 Attributes:

7.7.2.1.5 Methods:

7.7.2.2 Class Business Unit

in package 'Organization'

The BusinessUnit class represents a group that performs work of the business and is part of the business. This class should be disjoint from the ExternalUnit class. Note that neither the BusinessUnit class nor the ExternalUnit class is disjoint from any of the other specializations of BusinessSocialGroup.

7.7.2.2.1 Superclasses

7.7.2.2.1.1 Class Organization::Business Social Group

7.7.2.2.2 Subclasses:

7.7.2.2.2.1 Class Stakeholder::Participating Stakeholder

7.7.2.2.3 Associations:

7.7.2.2.3.1.1 Association:executes Type:Association

Source: Organization::Business Unit **Role:** **Cardinality:**

Target: Initiative::Initiative **Role:** **Cardinality:**

7.7.2.2.3.1.1.1 *Constraints:*

7.7.2.2.3.1.2 Association:sponsors Type:Association

Source: Organization::Business Unit **Role:** **Cardinality:**

Target: Initiative::Initiative **Role:** **Cardinality:**

7.7.2.2.3.1.2.1 *Constraints:*

7.7.2.2.3.1.3 Association:impacts Type:Association

Source: Organization::Business Unit **Role:** **Cardinality:**

Target: Initiative::Initiative **Role:** **Cardinality:**

7.7.2.2.3.1.3.1 *Constraints:*

The **impacts** association generalizes the concept that an Objective of the Initiative is to make a change in a Business Unit. Such associations must be consistent with details when details are added to the model.

7.7.2.2.3.1.4 Association:owns Type:Association

Source: Organization::Business Unit **Role:** **Cardinality:**

Target: Product::Product Concept **Role:** **Cardinality:**

7.7.2.2.3.1.4.1 *Constraints:*

This metamodel association represents the concept that a BusinessUnit owns (has responsibility for) a Product or a ProductLine.

7.7.2.2.3.1.5 Association:adopts Type:Association

Source: Policy::Policy **Role:** **Cardinality:**

Target: Organization::Business Unit **Role:** **Cardinality:**

7.7.2.2.3.1.5.1 *Constraints:*

A BusinessUnit may adopt a Policy. Typically this means that the Decider with responsibility for the BusinessUnit has decided that the Policy will influence the behavior of the BusinessUnit. This association represents the high level concept of both the decision to adopt the policy and the associated notion that the Policy influences the BusinessUnit. It would make little sense for a BusinessUnit to adopt a Policy that had no influence whatever on its behaviors.

7.7.2.2.4 Attributes:

7.7.2.2.5 Methods:

7.7.2.3 Class Collaborative Team

in package 'Organization'

CollaborativeTeam is a specialization of BusinessSocialGroup that is organized to address a specified set of issues and is disbanded once its work is completed.

7.7.2.3.1 Superclasses

7.7.2.3.1.1 Class Organization::Business Social Group

7.7.2.3.2 Subclasses:

7.7.2.3.2.1 Class **Organization::Forum**

7.7.2.3.3 Associations:

7.7.2.3.4 Attributes:

7.7.2.3.5 Methods:

7.7.2.4 Class Decider

in package 'Organization'

The Decider class represents a role in the organization that involves decision making and leadership of a group.

7.7.2.4.1 Superclasses

7.7.2.4.1.1 Class **Foundations::OperatingEntity**

7.7.2.4.2 Subclasses:

7.7.2.4.3 Associations:

7.7.2.4.3.1.1 Association:subject Type:Association

Source: Organization::Decider **Role:** **Cardinality:**

Target: Organization::accountable **Role:** **Cardinality:**

7.7.2.4.3.1.1.1 Constraints:

7.7.2.4.3.1.2 Association:DeciderRelationship Type:Association

Source: Organization::Decider **Role:** **Cardinality:**

Target: Organization::Decider **Role:** **Cardinality:**

7.7.2.4.3.1.2.1 Constraints:

The DeciderRelationship represents associations between Deciders. Typical DeciderRelationship types are subordination and collaboration.

7.7.2.4.3.1.3 Association:participates Type:Association

Source: Organization::Decider **Role:** **Cardinality:**

Target: Organization::Forum **Role:** **Cardinality:**

7.7.2.4.3.1.3.1 Constraints:

7.7.2.4.3.1.4 Association:DeciderRelationship Type:Association

Source: Organization::Decider **Role:** **Cardinality:**

Target: Organization::Decider **Role:** **Cardinality:**

7.7.2.4.3.1.4.1 Constraints:

The DeciderRelationship represents associations between Deciders. Typical DeciderRelationship types are subordination and collaboration.

7.7.2.4.4 Attributes:

7.7.2.4.5 Methods:

7.7.2.5 Class External Unit

in package 'Organization'

The ExternalUnit class represents a group that performs or is projected to perform work required by the business. This group is not part of the business and the work is performed under contract. The ExternalUnit class and the BusinessUnit class should be disjoint as the first is not a part of the business while the second is.

7.7.2.5.1 Superclasses

7.7.2.5.1.1 Class Organization::Business Social Group

7.7.2.5.2 Subclasses:

7.7.2.5.3 Associations:

7.7.2.5.4 Attributes:

7.7.2.5.5 Methods:

7.7.2.6 Class Forum

in package 'Organization'

A Forum is a type of CollaborativeTeam that is composed of business decision makers. It is typically used to resolve issues requiring tradeoffs among two or more areas of decision maker responsibility.

7.7.2.6.1 Superclasses

7.7.2.6.1.1 Class Organization::Collaborative Team

7.7.2.6.2 Subclasses:

7.7.2.6.3 Associations:

7.7.2.6.3.1.1 Association:participates Type:Association

Source: Organization::Decider **Role:** **Cardinality:**

Target: Organization::Forum **Role:** **Cardinality:**

7.7.2.6.3.1.1.1 *Constraints:*

7.7.2.6.4 Attributes:

7.7.2.6.5 Methods:

7.7.2.7 Class Performance Goals

in package 'Organization'

Performance goals are an objective for a set of measures that are used to manage the operations of a business and to audit progress of initiatives. PerformanceGoals are a kind of Policy and can be set by Deciders with the intent of controlling organization behavior.

NOTE: this class may be eliminated in favor of a measure defined in SMM.

7.7.2.7.1 Superclasses

7.7.2.7.1.1 Class Policy::Policy

7.7.2.7.2 Subclasses:

7.7.2.7.3 Associations:

7.7.2.7.3.1.1 Association:measure Type:Association

Source: Organization::accountable **Role:** **Cardinality:**

Target: Organization::Performance Goals **Role:** **Cardinality:**

7.7.2.7.3.1.1.1 Constraints:

7.7.2.7.4 Attributes:

7.7.2.7.5 Methods:

7.7.2.8 Class Worker

in package 'Organization'

The Worker class represents roles in the organization that primarily perform work. Typically the extensions of Worker and Executive will be disjoint, but this is not required by the metamodel.

7.7.2.8.1 Superclasses

7.7.2.8.1.1 Class Organization::Business Social Group

7.7.2.8.2 Subclasses:

7.7.2.8.3 Associations:

7.7.2.8.4 Attributes:

7.7.2.8.5 Methods:

7.7.2.9 Class accountable

in package 'Organization'

The Executive role is held accountable for the work performed by the Non-executive roles. This RoleElement is the core class of a 3-ary relationship linking together the Executive, the Non-executive and the PerformanceGoals.

7.7.2.9.1 Superclasses

7.7.2.9.1.1 Class Foundations::Role Element

7.7.2.9.2 Subclasses:

7.7.2.9.3 Associations:

7.7.2.9.3.1.1 Association:object Type:Association

Source: Organization::accountable **Role:** **Cardinality:**

Target: Organization::Business Social Group **Role:** **Cardinality:**

7.7.2.9.3.1.1.1 *Constraints:*

7.7.2.9.3.1.2 Association:measure Type:Association

Source: Organization::accountable **Role:** **Cardinality:**

Target: Organization::Performance Goals **Role:** **Cardinality:**

7.7.2.9.3.1.2.1 *Constraints:*

7.7.2.9.3.1.3 Association:subject Type:Association

Source: Organization::Decider **Role:** **Cardinality:**

Target: Organization::accountable **Role:** **Cardinality:**

7.7.2.9.3.1.3.1 *Constraints:*

7.7.2.9.4 Attributes:

7.7.2.9.5 Methods:

7.7.2.10 Class controls

in package 'Organization'

The "controls" class represents the ability an Executive (decision maker) has to change job assignments and work descriptions.

7.7.2.10.1 Superclasses

7.7.2.10.1.1 Class Foundations::Role Element

7.7.2.10.2 Subclasses:

7.7.2.10.3 Associations:

7.7.2.10.3.1.1 Association:object Type:Association

Source: Organization::controls **Role:** **Cardinality:**

Target: Foundations::Behavior **Role:** **Cardinality:**

7.7.2.10.3.1.1.1 *Constraints:*

7.7.2.10.3.1.2 Association:subject Type:Association

Source: Stakeholder::Participating Stakeholder **Role:** **Cardinality:**

Target: Organization::controls **Role:** **Cardinality:**

7.7.2.10.3.1.2.1 *Constraints:*

7.7.2.10.4 Attributes:

7.7.2.10.5 Methods:

7.7.2.11 Class staffs

in package 'Organization'

The staffs class is the core element in a binary association pattern linking BusinessSocialGroup with CapabilityInstance. It represents the concept that the BusinessSocialGroup provides the human resources for the CapabilityInstance. Note that a BusinessSocialGroup may provide human resources for several CapabilityInstances. At present, the metamodel does not provide representation for other kinds of resources (plant, utilities, automation, and information technology).

7.7.2.11.1 Superclasses

7.7.2.11.1.1 Class Foundations::Role Element

7.7.2.11.2 Subclasses:

7.7.2.11.3 Associations:

7.7.2.11.3.1.1 Association:object Type:Association

Source: Organization::staffs **Role:** **Cardinality:**

Target: Capability::Capability Instance **Role:** **Cardinality:**

7.7.2.11.3.1.1.1 *Constraints:*

7.7.2.11.3.1.2 Association:subject Type:Association

Source: Organization::Business Social Group **Role:** **Cardinality:**

Target: Organization::staffs **Role:** **Cardinality:**

7.7.2.11.3.1.2.1 *Constraints:*

7.7.2.11.4 Attributes:

7.7.2.11.5 Methods:

7.8 BusinessObject

Package in package 'Model'

7.8.1 BusinessObject diagram

Class diagram in package 'BusinessObject'

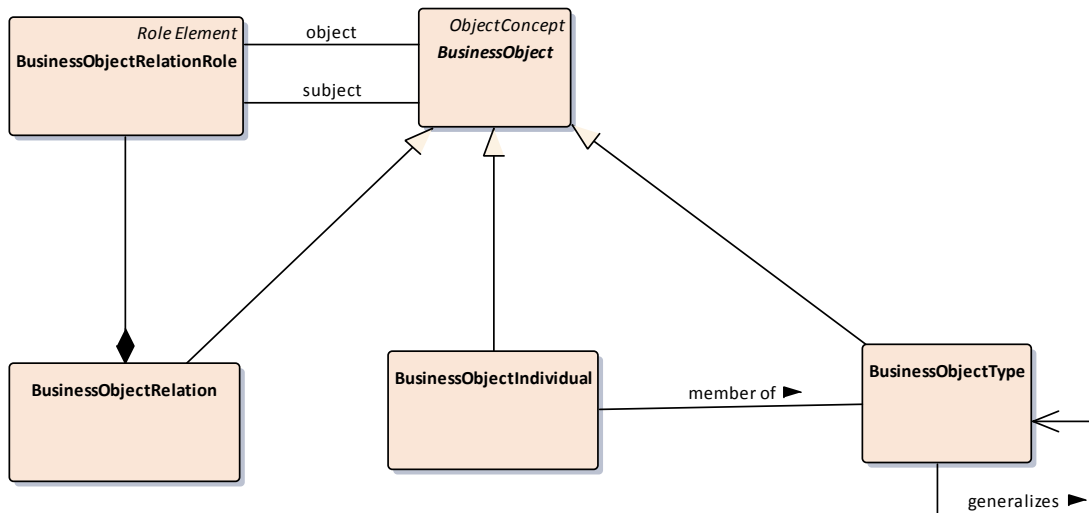


Figure 9: BusinessObject

BusinessObjects are passive, explicit entities in the business being modeled. The three concrete specializations of the BusinessObject supertypes are:

1. The BusinessObjectIndividual - a class that represents identifiable, individual explicit business entities such as a factory identified by its unique location or an executive identified by his/her unique role or title.
2. The BusinessObjectType - a class that represents a set of explicit business entities. It is not necessary to define the individual entities in a model instance of the BusinessObjectType; it typically suffices to define the type (e.g. iPhone 6, employee, assembly robot) without identifying the identifiable entities of the type.
3. The BusinessObjectRelation - a class that represents a business entity that is an aggregation of other business entities. This class can be used to represent physical assemblies; it can also be used to represent explicit entities that are connected together by causality or transformation, such as an email message sent via the internet.

The BusinessObjectRelationRole class represents an identifiable role that connects a BusinessObject to a BusinessObjectRelation. The model instances of this class need not be labeled when the contained BusinessObject is unique in the BusinessObjectRelation (e.g. the screen of an iPhone). They must be named when more than one model instance of a BusinessObject is contained (e.g. the front and rear facing cameras in an iPhone).

BusinessObjectType model instances may be arranged in a directed acyclic graph (DAG) with the "generalizes" association. The meaning of this association is that if an individual is a member of a BusinessObjectType model instance B and A generalizes B, then the individual is also a member of B. This association may be thought of as creating a subset relationship between the set represented by B and the set represented by A.

7.8.2 Classes:

7.8.2.1 Class BusinessObjectIndividual

in package 'BusinessObject'

A BusinessObjectIndividual model instance should represent an identifiable, explicit business entity such as a factory, a specific retail store or an identified executive. Typically there will be just a few of these model instances in a business architecture model. Most explicit business entities will be modeled as instances of the BusinessObjectType.

7.8.2.1.1 Superclasses

7.8.2.1.1.1 Class BusinessObject::BusinessObject

7.8.2.1.2 Subclasses:

7.8.2.1.3 Associations:

7.8.2.1.3.1.1 Association:member of Type:Association

Source: BusinessObject::BusinessObjectType **Role:** **Cardinality:**

Target: BusinessObject::BusinessObjectIndividual **Role:** **Cardinality:**

7.8.2.1.3.1.1.1 *Constraints:*

7.8.2.1.4 Attributes:

7.8.2.1.5 Methods:

7.8.2.2 Class BusinessObjectRelation

in package 'BusinessObject'

The BusinessObjectRelation class represents a business entity that is composed of identifiable parts. Most commonly this will be a physical assembly, but model instances of the class should also be used to represent aggregations of business entities that are causally or otherwise connected (e.g. an email sent over the internet, communicating radio stations). The model instance of this class is connected to the components through the BusinessObjectRelationRole class and the associations tied to it. This class and its associations form a pattern that represents an n-ary relation. The roles in the relation are identified by the model instances of the BusinessObjectRelationRole class. The same type of business entity may be included in the relation multiple times, but with different model instances of the BusinessObjectRelationRole class.

The BusinessObjectRelation model instance is effectively a type if some or all of its components are types. The BusinessObjectRelation model instance is an individual only if all of its components are in individuals.

7.8.2.2.1 Superclasses

7.8.2.2.1.1 Class BusinessObject::BusinessObject

7.8.2.2.2 Subclasses:

7.8.2.2.2.1 Class BusinessObject::BusinessObjectRelationRole

7.8.2.2.3 Associations:

7.8.2.2.3.1.1 Association: Type:Aggregation

Source: BusinessObject::BusinessObjectRelationRole **Role:** **Cardinality:**

Target: BusinessObject::BusinessObjectRelation **Role:** **Cardinality:**

7.8.2.2.3.1.1.1 *Constraints:*

7.8.2.2.4 Attributes:

7.8.2.2.5 Methods:

7.8.2.3 Class BusinessObjectRelationRole

in package 'BusinessObject'

The BusinessObjectRelationRole class is the core element in a pattern. This pattern represents the roles in an assembly of business entities that form a distinct entity. Structurally, the class and its subject and object associations is an individual relationship in an n-ary association identified by the BusinessObjectRelation.

The model instances of this class belong to the model instance of the BusinessObjectRelation class. These model instances have a model lifetime coincident with that of the model instance of the BusinessObjectRelation class.

7.8.2.3.1 Superclasses

7.8.2.3.1.1 Class Foundations::Role Element

7.8.2.3.2 Subclasses:

7.8.2.3.3 Associations:

7.8.2.3.3.1.1 Association:object Type:Association

Source: BusinessObject::BusinessObjectRelationRole **Role:** **Cardinality:**

Target: BusinessObject::BusinessObject **Role:** **Cardinality:**

7.8.2.3.3.1.1.1 *Constraints:*

7.8.2.3.3.1.2 Association:subject Type:Association

Source: BusinessObject::BusinessObjectRelationRole **Role:** **Cardinality:**

Target: BusinessObject::BusinessObject **Role:** **Cardinality:**

7.8.2.3.3.1.2.1 *Constraints:*

7.8.2.3.3.1.3 Association: Type:Aggregation

Source: BusinessObject::BusinessObjectRelationRole **Role:** **Cardinality:**

Target: BusinessObject::BusinessObjectRelation **Role:** **Cardinality:**

7.8.2.3.3.1.3.1 *Constraints:*

7.8.2.3.4 Attributes:

7.8.2.3.5 Methods:

7.8.2.4 Class BusinessObjectType

in package 'BusinessObject'

The BusinessObjectType class is intended to represent the concept of a set of explicit business entities that are all similar in some way such as the set of products of a particular type or the set of parts of a particular type.

Many other classes in the business architecture metamodel (e.g. CapabilityInstance, BusinessSocialGroup) could be thought of as BusinessObjects. While this is correct, the metamodel separates these concepts. Consequently, the business architect must make a conscious decision about whether to represent a business entity type as a BusinessObjectType or as one of the other classes in the metamodel. A significant criterion for the decision to model a business entity type as a BusinessObjectType is to consider the importance of the business entity type in the value streams of the business. If the business entity type plays a passive or supporting role in these value streams, then it is likely best modeled as a BusinessObjectType model instance.

7.8.2.4.1 Superclasses

7.8.2.4.1.1 Class BusinessObject::BusinessObject

7.8.2.4.2 Subclasses:

7.8.2.4.3 Associations:

7.8.2.4.3.1.1 Association:member of Type:Association

Source: BusinessObject::BusinessObjectType **Role:** **Cardinality:**

Target: BusinessObject::BusinessObjectIndividual **Role:** **Cardinality:**

7.8.2.4.3.1.1.1 *Constraints:*

7.8.2.4.3.1.2 Association:generalizes Type:Association

Source: BusinessObject::BusinessObjectType **Role:** **Cardinality:**

Target: BusinessObject::BusinessObjectType **Role:** **Cardinality:**

7.8.2.4.3.1.2.1 *Constraints:*

7.8.2.4.3.1.3 Association:generalizes Type:Association

Source: BusinessObject::BusinessObjectType **Role:** **Cardinality:**

Target: BusinessObject::BusinessObjectType **Role:** **Cardinality:**

7.8.2.4.3.1.3.1 *Constraints:*

7.8.2.4.4 Attributes:

7.8.2.4.5 Methods:

7.8.2.5 Class BusinessObject

in package 'BusinessObject'

BusinessObject is an abstract meta-class intended to represent explicit entities in the business. An explicit entity exists for a period of time and may be a physical entity or an electronic entity (electronic entities are physical). A

BusinessObject is not an action or an event.

BusinessObjects typically represent entities that are changeable. Business entities may lose a distinct identity during a change process. For example, a steel billet may be melted and cast into a set of axe heads. Not all entities have a distinct identity; e.g. a liter of water.

7.8.2.5.1 Superclasses

7.8.2.5.1.1 Class Foundations::ObjectConcept

7.8.2.5.2 Subclasses:

7.8.2.5.2.1 Class BusinessObject::BusinessObjectType

7.8.2.5.2.2 Class BusinessObject::BusinessObjectRelation

7.8.2.5.2.3 Class BusinessObject::BusinessObjectIndividual

7.8.2.5.3 Associations:

7.8.2.5.3.1.1 Association:is about Type:Association

Source: BusinessObject::BusinessObject **Role:** **Cardinality:**

Target: Foundations::Business Element **Role:** **Cardinality:**

7.8.2.5.3.1.1.1 *Constraints:*

Constraint: Business Object makes explicit Information Concept is about Business Element Invariant

This "is about" association is a high level association that can be used to connect an information representing BusinessObject to the BusinessElement the information is about. When the model is developed to include InformationConcepts, for each instance of this association, there should be an instance of the association chain: BusinessObject makes explicit an InformationConcept that is about the BusinessElement.

7.8.2.5.3.1.2 Association:object Type:Association

Source: BusinessObject::BusinessObjectRelationRole **Role:** **Cardinality:**

Target: BusinessObject::BusinessObject **Role:** **Cardinality:**

7.8.2.5.3.1.2.1 *Constraints:*

7.8.2.5.3.1.3 Association:subject Type:Association

Source: BusinessObject::BusinessObjectRelationRole **Role:** **Cardinality:**

Target: BusinessObject::BusinessObject **Role:** **Cardinality:**

7.8.2.5.3.1.3.1 *Constraints:*

7.8.2.5.3.1.4 Association:subject Type:Association

Source: Information::makes explicit **Role:** **Cardinality:**

Target: BusinessObject::BusinessObject **Role:** **Cardinality:**

7.8.2.5.3.1.4.1 *Constraints:*

7.8.2.5.3.1.5 Association: Type:Association

Source: BACM_Model::BusinessObjectMap **Role:** **Cardinality:**

Target: BusinessObject::BusinessObject **Role:** **Cardinality:** 0..*

7.8.2.5.3.1.5.1 *Constraints:*

7.8.2.5.4 Attributes:

7.8.2.5.5 Methods:

7.9 Stakeholder

Package in package 'Model'

7.9.1 Stakeholder diagram

Class diagram in package 'Stakeholder'

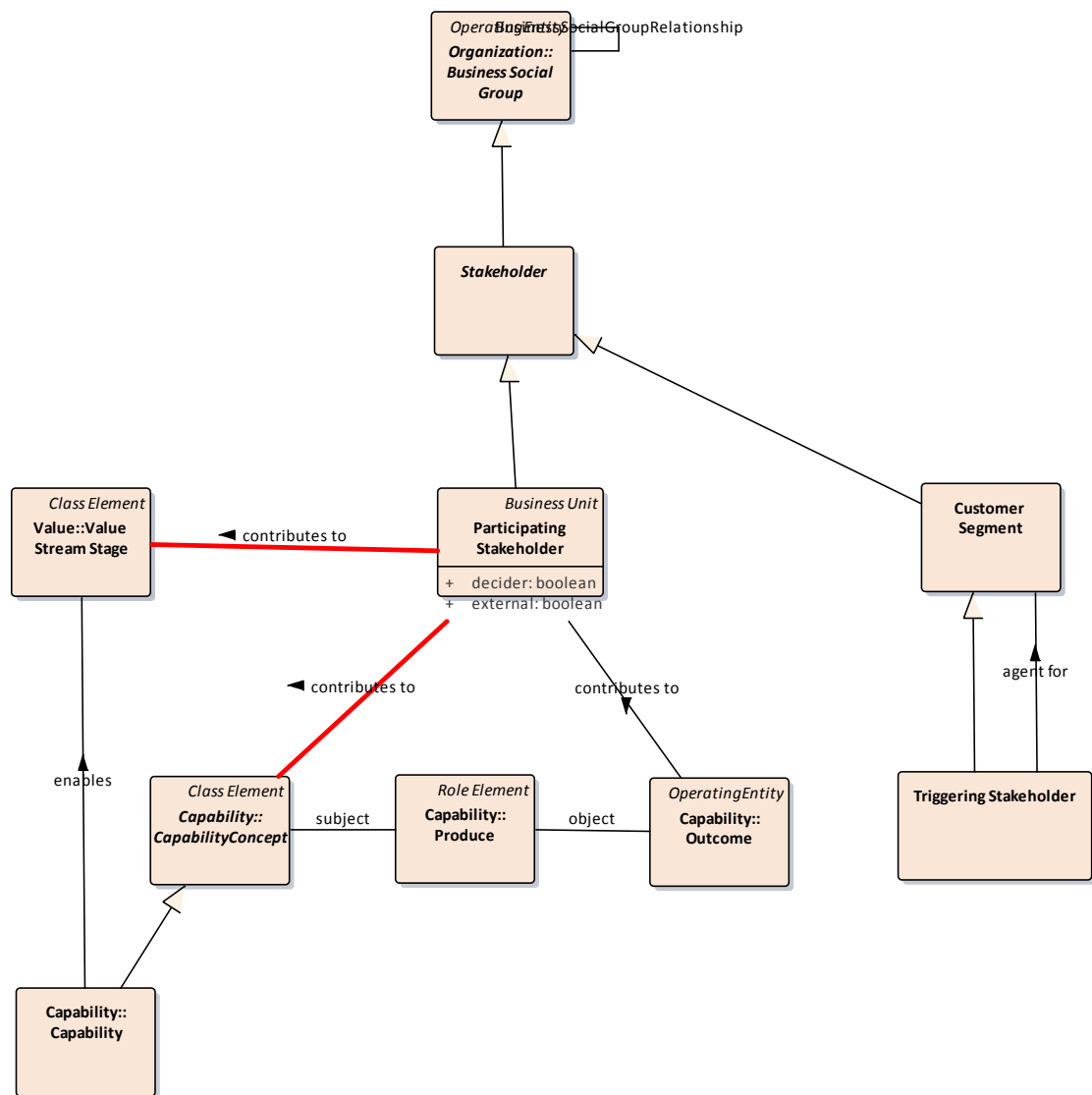


Figure 10: Stakeholder

A stakeholder is an individual or group (represented by the BusinessSocialGroup class) and inherits the characteristics of this class.

The Stakeholder diagram represents the two primary stakeholder concepts:

1. CustomerSegment - a stakeholder who receives value created by a value stream;
2. Participating stakeholder - a stakeholder who participates in the value creating activities of a ValueStreamStage.

In the case of the ParticipatingStakeholder, the diagram shows how this concept is defined in terms of metamodel associations and target classes. The base definition has the ParticipatingStakeholder contributing to some Outcomes produced by a Capability. Practically, these Outcomes will be substantial contributors to the value produced in the corresponding value streams.

Two high level abstractions of the "contributes to" association associate the ParticipatingStakeholder with a Capability (supporting the ValueStreamStage) and also with the ValueStreamStage. These associations are intended for use when the details of the capability map are not complete or when they are unimportant for the clarity of a presentation.

7.9.2 Classes:

7.9.2.1 Class Customer Segment

in package 'Stakeholder'

Consumer segment is a description of attributes and values which characterize the purchase threshold function with respect to a particular type of product or service. It is also a set of potential consumers for the type of product or service.

7.9.2.1.1 Superclasses

7.9.2.1.1.1 Class Stakeholder::Stakeholder

7.9.2.1.2 Subclasses:

7.9.2.1.2.1 Class Stakeholder::Triggering Stakeholder

7.9.2.1.3 Associations:

7.9.2.1.3.1.1 Association:agent for Type:Association

Source: Stakeholder::Triggering Stakeholder **Role:** **Cardinality:**

Target: Stakeholder::Customer Segment **Role:** **Cardinality:**

7.9.2.1.3.1.1.1 Constraints:

A triggering Stakeholder may either be a Consumer Segment or act as an agent for a Consumer Segment

7.9.2.1.3.1.2 Association:fits Type:Association

Source: Value::Value Proposition **Role:** **Cardinality:** 0..1

Target: Stakeholder::Customer Segment **Role:** **Cardinality:** 0..1

7.9.2.1.3.1.2.1 Constraints:

Constraint: must align with consumer of value stream Invariant

7.9.2.1.4 Attributes:

7.9.2.1.5 Methods:

7.9.2.2 Class Participating Stakeholder

in package 'Stakeholder'

A participating stakeholder has a defined role or responsibility within a value stream stage by contributing to the outcomes of the capabilities associated with that value stream stage.

An individual or group with decision making responsibility for a Capability that supports a ValueStreamStage is a ParticipatingStakeholder. This aspect of the definition of ParticipatingStakeholder cannot be directly shown in the metamodel, but it can be expressed as a constraint. The notion of a group with decision making responsibility is represented in the metamodel by the Executive class in the Organization package.

Likewise, an individual or group that performs important activities associated with a capability is a ParticipatingStakeholder. This notion of an individual or group with responsibility for performing activities is represented in the metamodel by the Worker class.

7.9.2.2.1 Superclasses

7.9.2.2.1.1 Class Stakeholder::Stakeholder

7.9.2.2.1.2 Class Organization::Business Unit

7.9.2.2.2 Subclasses:

7.9.2.2.3 Associations:

7.9.2.2.3.1.1 Association:contributes to Type:Association

Source: Stakeholder::Participating Stakeholder **Role:** **Cardinality:**

Target: Capability::CapabilityConcept **Role:** **Cardinality:**

7.9.2.2.3.1.1.1 *Constraints:*

This association abstracts the notion that a ParticipatingStakeholder contributes to Outcomes of a Capability that enables the ValueStreamStage. It is intended for use when Outcomes have not yet been defined in the Capability map or in presentations when the Outcomes of the Capability are obvious.

7.9.2.2.3.1.2 Association:sets Type:Association

Source: Stakeholder::Participating Stakeholder **Role:** **Cardinality:**

Target: Policy::Policy **Role:** **Cardinality:**

7.9.2.2.3.1.2.1 *Constraints:*

The "sets" association linking Decider to Policy is a more specific representation of the concept that a "Decider controls behavior" for the case where the Behavior being controlled is a Policy.

7.9.2.2.3.1.3 Association:contributes to Type:Association

Source: Stakeholder::Participating Stakeholder **Role:** **Cardinality:**

Target: Value::Value Stream Stage **Role:** **Cardinality:**

7.9.2.2.3.1.3.1 *Constraints:*

This "contributes to" high level association links a participating stakeholder to a value stream stage. It should be used to capture a significant contributing stakeholder to a value stream stage when there is no capability map or it is not yet defined. The constraint of this association defines compatible expressions of details involving the capabilities and their support for the value stream stage,

7.9.2.2.3.1.4 Association:subject Type:Association

Source: Stakeholder::Participating Stakeholder **Role:** **Cardinality:**

Target: Organization::controls **Role:** **Cardinality:**

7.9.2.2.3.1.4.1 *Constraints:*

7.9.2.2.3.1.5 Association:contributes to Type:Association

Source: Stakeholder::Participating Stakeholder **Role:** **Cardinality:**

Target: Capability::Outcome **Role:** **Cardinality:**

7.9.2.2.3.1.5.1 *Constraints:*

This association captures the concept that a ParticipatingStakeholder contributes to the Outcomes of Capabilities that support the ValueStreamStage.

7.9.2.2.4 Attributes:

7.9.2.2.4.1.1 Attribute: Participating Stakeholder.decider Type: boolean

7.9.2.2.4.1.2 Attribute: Participating Stakeholder.external Type: boolean

7.9.2.2.5 Methods:

7.9.2.3 Class Stakeholder

in package 'Stakeholder'

An internal or external individual or organization with a vested interest in achieving value through a particular outcome.

7.9.2.3.1 Superclasses

7.9.2.3.1.1 **Class Organization::Business Social Group**

7.9.2.3.2 Subclasses:

7.9.2.3.2.1 **Class Stakeholder::Customer Segment**

7.9.2.3.2.2 **Class Stakeholder::Participating Stakeholder**

7.9.2.3.3 Associations:

7.9.2.3.3.1.1 Association:triggers Type:Association

Source: Stakeholder::Stakeholder **Role:** **Cardinality:**

Target: Value::ValueStream **Role:** **Cardinality:**

7.9.2.3.3.1.1.1 *Constraints:*

7.9.2.3.3.1.2 Association:delivers value to Type:Association

Source: Requirements::Requirement **Role:** **Cardinality:**

Target: Stakeholder::Stakeholder **Role:** **Cardinality:**

7.9.2.3.3.1.2.1 *Constraints:*

BIZBOK® Guide, Section 3.8 Reference:

Per Section: Benefits of BA and Requirements Alignment

Requirement > delivers value to > stakeholder

Per Section: Principle of BA and Requirements Alignment

Requirements > based on clearly articulated > stakeholder perception

Per Section: BA and Requirements Alignment

Requirement > links to > stakeholder

7.9.2.3.3.1.3 Association: Type:Association

Source: BACM_Model::StakeholderMap **Role:** **Cardinality:**

Target: Stakeholder::Stakeholder **Role:** **Cardinality:** 0..*

7.9.2.3.3.1.3.1 *Constraints:*

7.9.2.3.4 Attributes:

7.9.2.3.5 Methods:

7.9.2.4 Class Triggering Stakeholder

in package 'Stakeholder'

A Value Stream begins with a stakeholder triggering the first stage of the value stream and ends when a product or service, notification, a degree of satisfaction, or other communication is delivered back to that stakeholder (BIZBOK® Guide v6.5, Stakeholder Mapping, Section 2.8)

A triggering stakeholder is defined as a category of stakeholder that initiates a value stream for the purpose of achieving a stated value proposition. (BIZBOK® Guide v6.5, Stakeholder Mapping, Section 2.8)

A triggering stakeholder may be distinct from a customer segment in order to model situations where one person purchases a product intended for use by another. The "agent for" association links these classes and represents the agency relationship existing between the two classes of stakeholders.

The triggering stakeholder specializes the customer segment. Normally the architect will create an instance of the triggering stakeholder, representing that the triggering stakeholder and the customer segment are the same,

The triggering stakeholder may trigger the value stream directly (e.g. by asking to purchase an item) or indirectly (e.g. by establishing a policy or rule that responds automatically to an event such as a loan default). The latter is called "triggering by proxy" (see BIZBOK® Guide v6.5, Value Mapping, Section 2.4).

7.9.2.4.1 Superclasses

7.9.2.4.1.1 Class Stakeholder::Customer Segment

7.9.2.4.2 Subclasses:

7.9.2.4.3 Associations:

7.9.2.4.3.1.1 Association:agent for Type:Association

Source: Stakeholder::Triggering Stakeholder **Role:** **Cardinality:**

Target: Stakeholder::Customer Segment **Role:** **Cardinality:**

7.9.2.4.3.1.1.1 *Constraints:*

A triggering Stakeholder may either be a Consumer Segment or act as an agent for a Consumer Segment

7.9.2.4.3.1.2 Association:triggers Type:Association

Source: Value::ValueStream **Role:** **Cardinality:**

Target: Stakeholder::Triggering Stakeholder **Role:** **Cardinality:**

7.9.2.4.3.1.2.1 *Constraints:*

The "triggers" association links a stakeholder to the ValueStream that is triggered by the stakeholder. A stakeholder may trigger the value stream directly (e.g. by asking to purchase an item) or indirectly (e.g. as in a loan default, where the triggering stakeholder establishes a rule or policy for triggering the value stream). The latter situation is called "triggering by proxy" (see BIZBOK® Guide v6.5, Value Mapping, Section 2.4).

7.9.2.4.4 Attributes:

7.9.2.4.5 Methods:

7.10 Product

Package in package 'Model'

7.10.1 Product diagram

Class diagram in package 'Product'

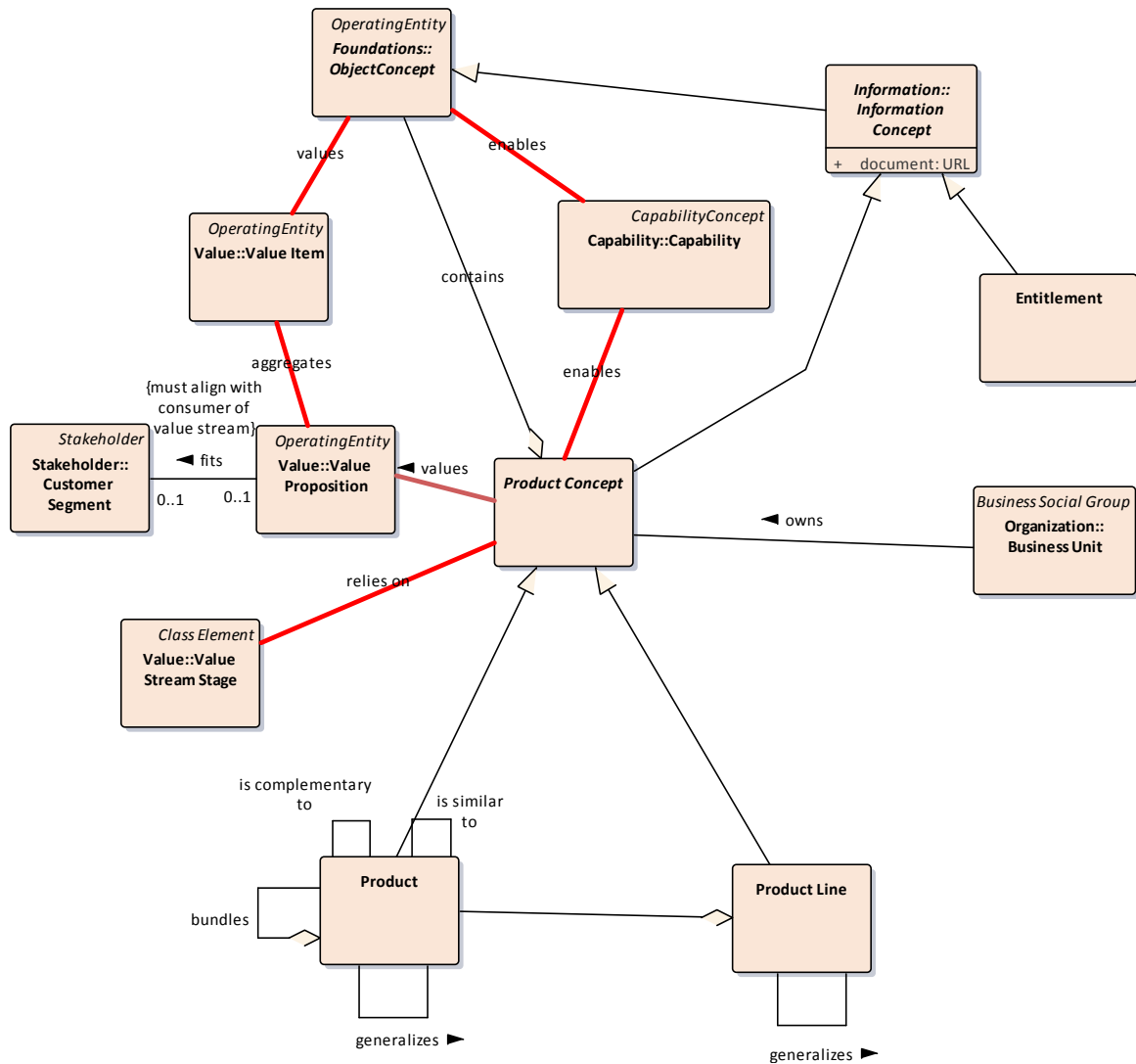


Figure 11: Product

A ProductConcept is an abstract base class for both Product and ProductLine.

Product and ProductLine model instances can be arranged in inheritance hierarchies where appropriate to facilitate the sharing of common concepts.

A Product or ProductLine is owned by a BusinessUnit. It would be typical for a Product to be owned by a BusinessUnit

that is the same or a subordinate of the BusinessUnit that owns the ProductLine containing the Product, but the metamodel does not define this as a constraint.

A Product (or ProductLine) contains BusinessObjects or InformationConcepts. BusinessObjects are tangibles while InformationConcepts are intangibles. An example of a product containing an InformationConcept would be a license to use some specified intellectual property.

Entitlements are benefits that are contained in the Product or ProductLine and can be demanded by the purchaser after the sale. Entitlements are InformationConcepts and can be contained in a Product or ProductLine.

Any component of a Product or ProductLine must be enabled by a Capability in the sense that the Capability must produce Outcomes that create the component or create the benefit represented by the Entitlement. An example entitlement for an auto purchase would be a service contract; the business is obligated by the entitlement to provide an auto service capability to the purchaser, whether by the business itself or through an arrangement with a third party.

When the components of a Product or ProductLine have not yet been modeled, but it is desirable to track the Capabilities that would enable these components, a Capability may enable a Product or ProductLine provided that the Capability also enables some of its components.

A ValueItem may value a component of a Product or ProductLine when the BusinessObject or InformationConcept that is a component of the product is affected by an Outcome that is valued by the same ValueItem (see the Value package for details). This is a high level association that must be consistent with the details specified in the value and capability models.

For convenience in creating a business architecture model or in simplifying it for presentation, a ValueItem may value a Product or ProductLine if that ValueItem values some component of the Product or ProductLine. This association concept is represented by the valuation of an ObjectConcept by a ValueItem, because ProductConcept is a subclass of ObjectConcept.

Likewise, a ValueProposition may be said to value a Product or ProductLine when that ValueProposition contains ValueItems that value the ProductConcept.

7.10.2 Classes:

7.10.2.1 Class Entitlement

in package 'Product'

An entitlement is a right to benefits that is granted especially by law or contract. An entitlement creates an obligation on the part of the seller of the product to provide the benefit defined by the entitlement on the demand by the purchaser and subject to constraints and exclusions defined in the entitlement.

7.10.2.1.1 Superclasses

7.10.2.1.1.1 Class Information::Information Concept

7.10.2.1.2 Subclasses:

7.10.2.1.3 Associations:

7.10.2.1.4 Attributes:

7.10.2.1.5 Methods:

7.10.2.2 Class Product Concept

in package 'Product'

ProductConcept is an abstract base class for Product and ProductLine. It provides conceptual associations that are common to both Product and ProductLine, such as being valued by a ValueProposition, having ObjectConcepts as components, and being owned by a BusinessUnit.

Both Product and ProductLine are conceptual entities that are separate from the ObjectConcept that is contained in the ProductConcept. A physical object, such as a mobile phone, is contained in a Product; the Product determines the offering price and conditions of sale and contains any other other entities or Entitlements.

7.10.2.2.1 Superclasses

7.10.2.2.1.1 Class Information::Information Concept

7.10.2.2.2 Subclasses:

7.10.2.2.2.1 Class Product::Product

7.10.2.2.2.2 Class Foundations::ObjectConcept

7.10.2.2.2.3 Class Product::Product Line

7.10.2.2.3 Associations:

7.10.2.2.3.1.1 Association:relies on Type:Association

Source: Product::Product Concept **Role:** **Cardinality:**

Target: Value::ValueStream **Role:** **Cardinality:**

7.10.2.2.3.1.1.1 Constraints:

7.10.2.2.3.1.2 Association:values Type:Association

Source: Product::Product Concept **Role:** **Cardinality:**

Target: Value::Value Proposition **Role:** **Cardinality:**

7.10.2.2.3.1.2.1 Constraints:

This is a high level association linking a value proposition to a product. If details are added to the business architecture, they must satisfy this association's constraint that the ValueItems added to the ValueProposition must value the Product or ProductLine.

7.10.2.2.3.1.3 Association:owns Type:Association

Source: Organization::Business Unit **Role:** **Cardinality:**

Target: Product::Product Concept **Role:** **Cardinality:**

7.10.2.2.3.1.3.1 Constraints:

This metamodel association represents the concept that a BusinessUnit owns (has responsibility for) a Product or a ProductLine.

7.10.2.2.3.1.4 Association:enables Type:Association

Source: Capability::Capability **Role:** **Cardinality:**

Target: Product::Product Concept **Role:** **Cardinality:**

7.10.2.2.3.1.4.1 Constraints:

This association is a high level abstraction of the "enables" associations of the BusinessObjects and/or InformationConcepts contained in the Product or ProductLine with the Capabilities that produce them. The associations is valuable for aggregating and presenting information about the performance of Capabilities that enable the components of a product. It can also be used when the detailed components of a Product or ProductLine are not yet modeled.

7.10.2.2.3.1.5 Association: Type:Association

Source: BACM_Model::ProductMap **Role:** **Cardinality:**

Target: Product::Product Concept **Role:** **Cardinality:** 0..*

7.10.2.2.3.1.5.1 *Constraints:*

7.10.2.2.3.1.6 Association:evolves Type:Association

Source: Requirements::Requirement **Role:** **Cardinality:**

Target: Product::Product Concept **Role:** **Cardinality:**

7.10.2.2.3.1.6.1 *Constraints:*

Requirements evolve a Product at a conceptual level, which is reflected in one or more Product Instances. Requirements will specify how a product will change via changes to its design.

The "evolves" association is a specialization of the "Changes" association (see Strategy package). In this case, the "before" and "after" ProductConcepts are variants.

BIZBOK® Guide Section 3.8 Reference:

Per Section: Requirements Alignment with Products

Requirement > evolves > Product

7.10.2.2.3.1.7 Association:contains Type:Aggregation

Source: Foundations::ObjectConcept **Role:** **Cardinality:**

Target: Product::Product Concept **Role:** **Cardinality:**

7.10.2.2.3.1.7.1 *Constraints:*

7.10.2.2.3.1.8 Association:relies on Type:Association

Source: Value::Value Stream Stage **Role:** **Cardinality:**

Target: Product::Product Concept **Role:** **Cardinality:**

7.10.2.2.3.1.8.1 *Constraints:*

This association is a high level abstraction of the "enables" association between the ProductConcept and the Capabilities that "support" the ValueStreamStage. A ProductConcept relies on a ValueStreamStage when the ProductConcept contains ObjectConcepts that are enabled by Capabilities that support the ValueStreamStage

7.10.2.2.4 Attributes:

7.10.2.2.5 Methods:

7.10.2.3 Class Product

in package 'Product'

A Product is the overall experience provided by the combination of goods and services to satisfy the customer needs. This metamodel class represents the concept of a product.

7.10.2.3.1 Superclasses

7.10.2.3.1.1 Class Product::Product Concept

7.10.2.3.2 Subclasses:

7.10.2.3.2.1 Class Product::Product

7.10.2.3.3 Associations:

7.10.2.3.3.1.1 Association:is similar to Type:Association

Source: Product::Product **Role:** **Cardinality:**

Target: Product::Product **Role:** **Cardinality:**

7.10.2.3.3.1.1.1 *Constraints:*

Product may be similar to each other.

7.10.2.3.3.1.2 Association:is complementary to Type:Association

Source: Product::Product **Role:** **Cardinality:**

Target: Product::Product **Role:** **Cardinality:**

7.10.2.3.3.1.2.1 *Constraints:*

Product may be seen to be complementary to each other.

7.10.2.3.3.1.3 Association:generalizes Type:Association

Source: Product::Product **Role:** **Cardinality:**

Target: Product::Product **Role:** **Cardinality:**

7.10.2.3.3.1.3.1 *Constraints:*

This association should be interpreted in model instances as asserting that every statement that is true about the more general Product is also true about the more specific Product.

7.10.2.3.3.1.4 Association:bundles Type:Aggregation

Source: Product::Product **Role:** **Cardinality:**

Target: Product::Product **Role:** **Cardinality:**

7.10.2.3.3.1.4.1 *Constraints:*

Products may contain other products. Composed products are often called "bundles". This aggregation represents the non-exclusive containment of some Products by a bundled Product. Note that a bundle may be nothing more than a discount and it should not be taken to mean that the components of a bundle somehow rely on each other.

7.10.2.3.3.1.5 Association: Type:Aggregation

Source: Product::Product **Role:** **Cardinality:**

Target: Product::Product Line **Role:** **Cardinality:**

7.10.2.3.3.1.5.1 *Constraints:*

7.10.2.3.3.1.6 Association:is similar to Type:Association

Source: Product::Product **Role:** **Cardinality:**

Target: Product::Product **Role:** **Cardinality:**

7.10.2.3.3.1.6.1 *Constraints:*

Product may be similar to each other.

7.10.2.3.3.1.7 Association:is complementary to Type:Association

Source: Product::Product **Role:** **Cardinality:**

Target: Product::Product **Role:** **Cardinality:**

7.10.2.3.3.1.7.1 *Constraints:*

Product may be seen to be complementary to each other.

7.10.2.3.3.1.8 Association:generalizes Type:Association

Source: Product::Product **Role:** **Cardinality:**

Target: Product::Product **Role:** **Cardinality:**

7.10.2.3.3.1.8.1 *Constraints:*

This association should be interpreted in model instances as asserting that every statement that is true about the more general Product is also true about the more specific Product.

7.10.2.3.3.1.9 Association:bundles Type:Aggregation

Source: Product::Product **Role:** **Cardinality:**

Target: Product::Product **Role:** **Cardinality:**

7.10.2.3.3.1.9.1 *Constraints:*

Products may contain other products. Composed products are often called "bundles". This aggregation represents the non-exclusive containment of some Products by a bundled Product. Note that a bundle may be nothing more than a discount and it should not be taken to mean that the components of a bundle somehow rely on each other.

7.10.2.3.4 Attributes:

7.10.2.3.5 Methods:

7.10.2.4 Class Product Line

in package 'Product'

A "product line" is defined as a series of different products which form a group, all made by the same company or its business partners.

The ProductLine concept is used to group a family of products together in a meaningful way. A product line may be based on common traits (e.g. life insurance), a targeted buyer (e.g. commercial product line).

A ProductLine should define the rule used to group the products together.

7.10.2.4.1 Superclasses

7.10.2.4.1.1 Class Product::Product Concept

7.10.2.4.2 Subclasses:

7.10.2.4.2.1 Class Product::Product

7.10.2.4.3 Associations:

7.10.2.4.3.1.1 Association:generalizes Type:Association

Source: Product::Product Line **Role:** **Cardinality:**

Target: Product::Product Line **Role:** **Cardinality:**

7.10.2.4.3.1.1.1 *Constraints:*

Model instances of this association should be interpreted as asserting that every statement that is true of the more general ProductLine is also true of the more specific ProductLine.

7.10.2.4.3.1.2 Association:generalizes Type:Association

Source: Product::Product Line **Role:** **Cardinality:**

Target: Product::Product Line **Role:** **Cardinality:**

7.10.2.4.3.1.2.1 *Constraints:*

Model instances of this association should be interpreted as asserting that every statement that is true of the more general ProductLine is also true of the more specific ProductLine.

7.10.2.4.3.1.3 Association: Type:Aggregation

Source: Product::Product **Role:** **Cardinality:**

Target: Product::Product Line **Role:** **Cardinality:**

7.10.2.4.3.1.3.1 *Constraints:*

7.10.2.4.4 Attributes:

7.10.2.4.5 Methods:

7.11 Policy

Package in package 'Model'

7.11.1 Policy diagram

Class diagram in package 'Policy'

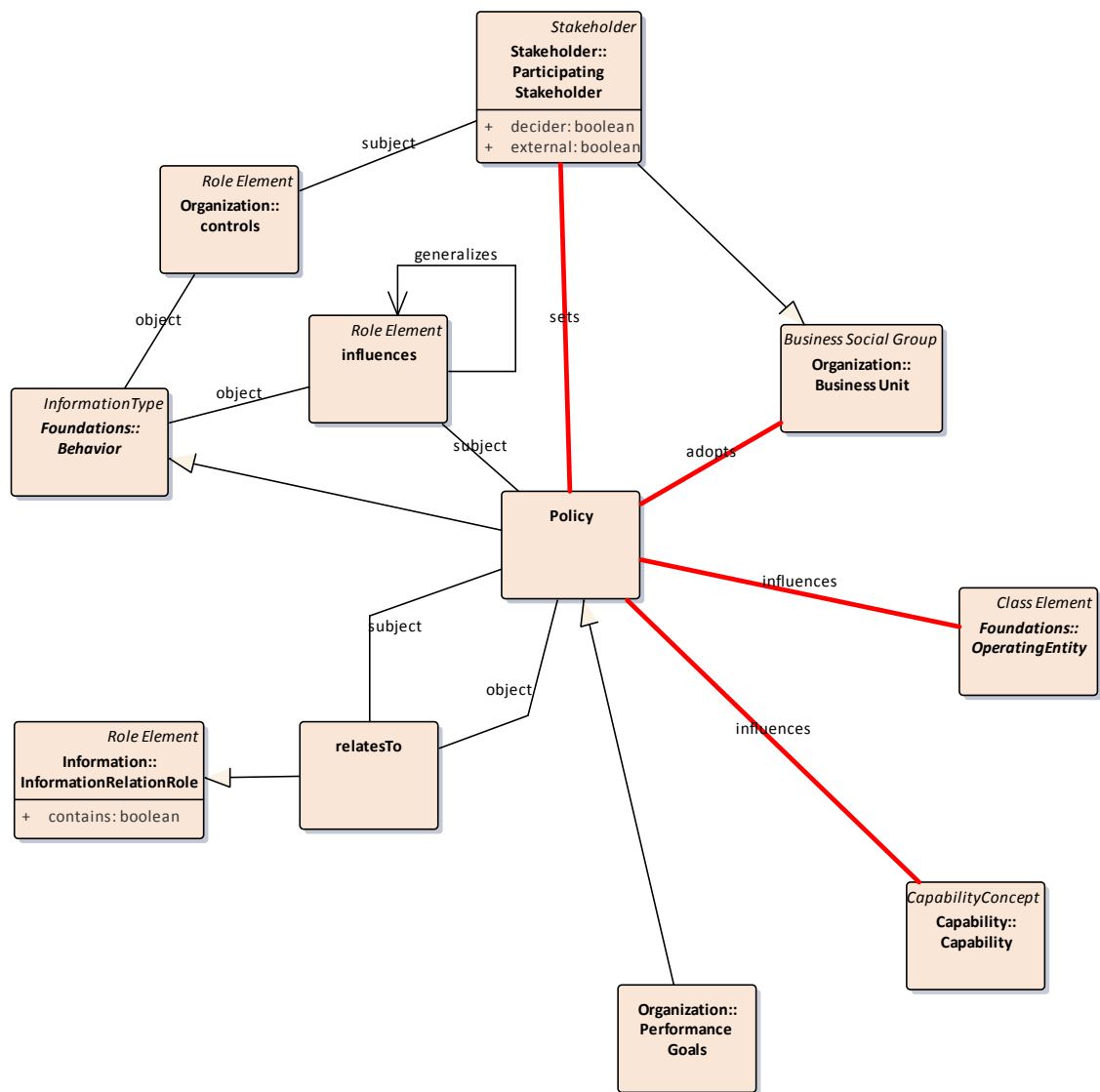


Figure 12: Policy

The Policy diagram shows the generalization and other key relationships in which the Policy class participates. A business policy is a course or principle of action that is adopted by a business.

A Policy is a type of InformationConcept. InformationConcepts are intangibles that frequently have one or more tangible forms. Policy inherits this characteristic from Behavior (see the Foundations package).

Policy is set within a business by a ParticipatingStakeholder (an individual or group with decision making responsibility). Policy may also be set outside of a business by governments, regulatory agencies or market participation agreements. This is represented in the metamodel by the "sets" association between the general notion of a Decider and the Policy.

Policies generally influence Behaviors associated with a Capability. Capability behaviors are represented by the CapabilityBehavior metamodel class. Policy is a component of CapabilityBehavior, along with procedure and activity specifications.

Policies are also said to influence elements of the operating model of a business. While policy directly constrains behavior of people and their automation agents, policy can also constrain behaviors indirectly by controlling the characteristics of Outcomes or the configuration and resources of a business, the metamodel provides a representation of this aspect of Policy through the "influences" association with any OperatingEntity.

To some extent, the influence of a policy overlaps the direct behavior specification through the CapabilityBehavior. The business architect will generally use the "influences" association when the policy constrains behavior indirectly and the CapabilityBehavior association when the policy constrains behavior directly.

A Policy may comprise a number of sub-policies. A particular policy or sub-policy may have both direct and indirect constraints on behavior. Because Policy is a specialization of InformationConcept, Policies may be related to each other by a variety of InformationRelationshipRoles. The metamodel implements one of these roles as the class association "relatesTo".

7.11.2 Classes:

7.11.2.1 Class Policy

in package 'Policy'

Policy - A course or principle of action adopted or proposed by a government, party, business, or individual. High level policies, such as "be responsive to the customer" are often called norms.

Policy is considered to be a type of Behavior. A process or procedure will describe a pattern of behavior; a Policy will constrain that behavior. For example, a sales process may be described as a sequence of activities consisting of receiving the customer's money, transferring possession of the purchased item and recording the sale. However, a Policy may restrict this process for specific types of purchased items according to the customer's age.

7.11.2.1.1 Superclasses

7.11.2.1.1.1 Class Foundations::Behavior

7.11.2.1.2 Subclasses:

7.11.2.1.2.1 Class Organization::Performance Goals

7.11.2.1.3 Associations:

7.11.2.1.3.1.1 Association:influences Type:Association

Source: Policy::Policy **Role:** Cardinality:

Target: Foundations::OperatingEntity **Role:** Cardinality:

7.11.2.1.3.1.1.1 Constraints:

While Policies directly influence behaviors, Policies are often specified as rules about the desired state of one or more OperatingEntities. In many cases, it is not necessary for the architect to directly model the behavior that will be influenced, but it is necessary to record the relationship between the Policy and the OperatingEntities whose state is described in the Policy. This "influences" association allows that relationship to be modeled.

If the influenced Behavior is later added to the model, it must be consistent with the associations to the OperatingEntities affected by or performing that Behavior.

7.11.2.1.3.1.2 Association:object Type:Association

Source: Policy::Policy **Role:** Cardinality:

Target: Policy::relatesTo **Role:** Cardinality:

7.11.2.1.3.1.2.1 Constraints:

7.11.2.1.3.1.3 Association:adopts Type:Association

Source: Policy::Policy **Role:** Cardinality:

Target: Organization::Business Unit **Role:** **Cardinality:**

7.11.2.1.3.1.3.1 *Constraints:*

A BusinessUnit may adopt a Policy. Typically this means that the Decider with responsibility for the BusinessUnit has decided that the Policy will influence the behavior of the BusinessUnit. This association represents the high level concept of both the decision to adopt the policy and the associated notion that the Policy influences the BusinessUnit. It would make little sense for a BusinessUnit to adopt a Policy that had no influence whatever on its behaviors.

7.11.2.1.3.1.4 Association:subject Type:Association

Source: Policy::Policy **Role:** **Cardinality:**

Target: Policy::relatesTo **Role:** **Cardinality:**

7.11.2.1.3.1.4.1 *Constraints:*

7.11.2.1.3.1.5 Association:influences Type:Association

Source: Policy::Policy **Role:** **Cardinality:**

Target: Capability::Capability **Role:** **Cardinality:**

7.11.2.1.3.1.5.1 *Constraints:*

This influences association is an abstraction of the details of a Policy influencing an OperatingEntity, typically a CapabilityBehavior or a CapabilityInstance. Capability is not considered an operating entity because it is an abstract categorization of activities performed by a business and cannot be directly influenced by a policy. The association allows the architect to model the concept that a Policy influences behavior of a Capability without having to model the details of the behavior.

7.11.2.1.3.1.6 Association:subject Type:Association

Source: Policy::Policy **Role:** **Cardinality:**

Target: Policy::influences **Role:** **Cardinality:**

7.11.2.1.3.1.6.1 *Constraints:*

7.11.2.1.3.1.7 Association:sets Type:Association

Source: Stakeholder::Participating Stakeholder **Role:** **Cardinality:**

Target: Policy::Policy **Role:** **Cardinality:**

7.11.2.1.3.1.7.1 *Constraints:*

The "sets" association linking Decider to Policy is a more specific representation of the concept that a "Decider controls behavior" for the case where the Behavior being controlled is a Policy.

7.11.2.1.3.1.8 Association: Type:Association

Source: BACM_Model::PolicyMap **Role:** **Cardinality:**

Target: Policy::Policy **Role:** **Cardinality:** 0..*

7.11.2.1.3.1.8.1 *Constraints:*

7.11.2.1.4 Attributes:

7.11.2.1.5 Methods:

7.11.2.2 Class influences

in package 'Policy'

The Influences class is the core of an association pattern that links a Policy to an OperatingEntity. The metamodel shows this class explicitly in order to also show that the Influences association model instances can be constructed into an inheritance hierarchy that distinguishes various ways that influences is manifested in business terms (e.g. as a regulation, a norm, a business vision).

7.11.2.2.1 Superclasses

7.11.2.2.1.1 Class Foundations::Role Element

7.11.2.2.2 Subclasses:

7.11.2.2.3 Associations:

7.11.2.2.3.1.1 Association:generalizes Type:Association

Source: Policy::influences **Role:** Cardinality:

Target: Policy::influences **Role:** Cardinality:

7.11.2.2.3.1.1.1 Constraints:

Instances of this meta-class link instances of the influences role element and represent generalization. If X and Y are instances of this meta-class and $X \rightarrow Y$, then the set of binary relations specified by Y contains the set specified by X. Each instance network in an instance model must be a directed acyclic graph.

7.11.2.2.3.1.2 Association:object Type:Association

Source: Policy::influences **Role:** Cardinality:

Target: Foundations::Behavior **Role:** Cardinality:

7.11.2.2.3.1.2.1 Constraints:

7.11.2.2.3.1.3 Association:generalizes Type:Association

Source: Policy::influences **Role:** Cardinality:

Target: Policy::influences **Role:** Cardinality:

7.11.2.2.3.1.3.1 Constraints:

Instances of this meta-class link instances of the influences role element and represent generalization. If X and Y are instances of this meta-class and $X \rightarrow Y$, then the set of binary relations specified by Y contains the set specified by X. Each instance network in an instance model must be a directed acyclic graph.

7.11.2.2.3.1.4 Association:subject Type:Association

Source: Policy::Policy **Role:** Cardinality:

Target: Policy::influences **Role:** Cardinality:

7.11.2.2.3.1.4.1 Constraints:

7.11.2.2.4 Attributes:

7.11.2.2.5 Methods:

7.11.2.3 Class relatesTo

in package 'Policy'

The "relatesTo" association is instantiated to relate one policy to another. As Policy is a subclass of InformationConcept, this class association is a specialization of the InformationRelationRole class

7.11.2.3.1 Superclasses

7.11.2.3.1.1 Class Information::InformationRelationRole

7.11.2.3.2 Subclasses:

7.11.2.3.3 Associations:

7.11.2.3.3.1.1 Association:object Type:Association

Source: Policy::Policy **Role:** **Cardinality:**

Target: Policy::relatesTo **Role:** **Cardinality:**

7.11.2.3.3.1.1.1 Constraints:

7.11.2.3.3.1.2 Association:subject Type:Association

Source: Policy::Policy **Role:** **Cardinality:**

Target: Policy::relatesTo **Role:** **Cardinality:**

7.11.2.3.3.1.2.1 Constraints:

7.11.2.3.4 Attributes:

7.11.2.3.5 Methods:

7.11.2.3.6

7.12 Strategy

Package in package 'Model'

The Strategy package (and Initiative package) represent potential or planned change in a business. The Value and Capability packages represent abstract modeling perspectives on the business that are useful for 1) consolidating groups of activities the significantly contribute value to a product, or 2) consolidating similar structures and/or behaviors occurring in multiple parts of the business. The other packages represent business concepts that are often called the "operating model" of the business.

Only the operating model concepts of a business are subject to change resulting from a strategy or an initiative. The Value and Capability concepts are used only for model analysis and presentation; trying to change them will not produce any change in the underlying business.

It does not make sense in a static model (one that is not intended for execution or simulation) to represent a strategy that intends to change itself or another strategy. Having a strategy model that can change itself implies that the meaning of the model cannot be statically determined.

7.12.1 Strategy diagram

Class diagram in package 'Strategy'

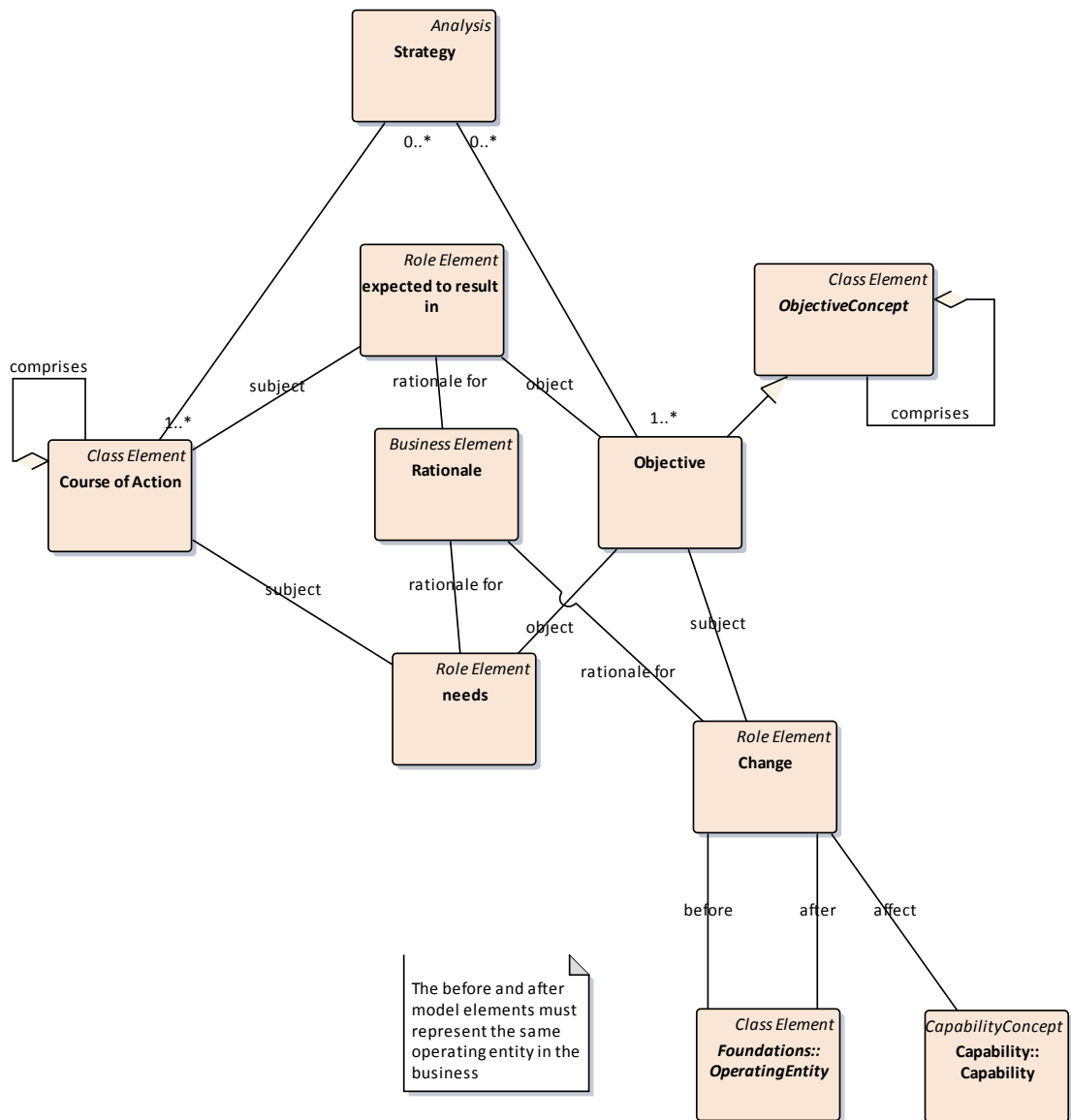


Figure 13: Strategy

Strategies are represented in the metamodel by the concept of some Courses of Action that are expected to result in the achievement of some Objectives. This expectation is supported by a Rationale, an explanation that contains arguments for and against the course of action and its expectation of achievement.

A strategy Objective is usually about a Change in some aspect of the business, represented here by the abstract class OperatingEntity. OperatingEntity has a number of concrete specializations, including ValueProposition, ValueItem and Product. A high level strategy will typically expect objectives that change the value of a business's products and services.

Strategies are rarely simple courses of action and expected objectives, and the metamodel can represent chains of Courses of Action with intermediate Objectives. In such a chain, a Course of Action needs one or more Objectives provided by a preceding Course of Action.

The metamodel also allows for Courses of Action and Objectives to be modeled as aggregations of Courses of Action and Objectives that represent additional details. For example, a Course of Action may aggregate a set of Courses of

Action that are linked in a chain with intermediate Objectives. Some of these intermediate Objectives may be parts of a final Objective.

The metamodel will allow construction of strategy models that do not make sense structurally; the business architect is expected to use good judgement when creating strategy models from this metamodel.

The Change class is associated with before and after OperatingEntity classes. The before and after OperatingEntity class instances in a model should be conceptual versions of the same class instance. This constraint is not enforced by the metamodel and the metamodel has no notion of versions of OperatingEntities.

7.12.2 Classes:

7.12.2.1 Class Change

in package 'Strategy'

The "Change" class represents an initial and a final state of an "Operating Entity". Typically, the initial state would represent the current state of the business concept modeled by the "Operating Entity" and the final state would represent the state of the same business concept envisioned by the strategy "Objective".

This class is a 3-ary association linking the strategy "Objective" to the "before" and "after" versions of the "Operating Entity". The metamodel does not currently support versioning operating entities, but this is an open issue.

7.12.2.1.1 Superclasses

7.12.2.1.1.1 Class Foundations::Role Element

7.12.2.1.2 Subclasses:

7.12.2.1.2.1 Class TOGAF::CapabilityIncrement

7.12.2.1.3 Associations:

7.12.2.1.3.1.1 Association:before Type:Association

Source: Strategy::Change **Role:** **Cardinality:**

Target: Foundations::OperatingEntity **Role:** **Cardinality:**

7.12.2.1.3.1.1.1 Constraints:

7.12.2.1.3.1.2 Association:after Type:Association

Source: Strategy::Change **Role:** **Cardinality:**

Target: Foundations::OperatingEntity **Role:** **Cardinality:**

7.12.2.1.3.1.2.1 Constraints:

7.12.2.1.3.1.3 Association:affect Type:Association

Source: Strategy::Change **Role:** **Cardinality:**

Target: Capability::Capability **Role:** **Cardinality:**

7.12.2.1.3.1.3.1 Constraints:

It will frequently be the case that the business architect has identified a Capability whose CapabilityInstances are not

performing as well as they need to. If the business desires to correct this situation, the architect will create an Objective to improve the performance of the CapabilityInstances. However, at this stage in the development of the strategy, the architect may not be able to identify the specific CapabilityInstances that will be targeted for improvement. In this situation, the architect may create an Objective to change the Capability and later identify the specific CapabilityInstances.

This association allows the architect to record an Objective that changes a Capability. Later, the model may be extended to identify the specific CapabilityInstances (as OperatingEntities) that will be affected by the change.

When this association is present in a model, the before and after associations of the Change must be CapabilityInstances or CapabilityBehaviors.

7.12.2.1.3.1.4 Association:rationale for Type:Association

Source: Strategy::Rationale **Role:** **Cardinality:**

Target: Strategy::Change **Role:** **Cardinality:**

7.12.2.1.3.1.4.1 *Constraints:*

7.12.2.1.3.1.5 Association:subject Type:Association

Source: Strategy::Objective **Role:** **Cardinality:**

Target: Strategy::Change **Role:** **Cardinality:**

7.12.2.1.3.1.5.1 *Constraints:*

7.12.2.1.4 Attributes:

7.12.2.1.5 Methods:

7.12.2.2 Class Course of Action

in package 'Strategy'

A course of action is intended to produce business change that results in accomplishment of strategic objectives. The "Course of Action" class is intended to represent strategic actions that are part of a business strategy.

In practice, once the objectives are determined, the courses of action may not be fully known. As the strategy is elaborated, the course of action may be constructed in greater detail and the details modeled with the self directed aggregation association.

7.12.2.2.1 Superclasses

7.12.2.2.1.1 Class Foundations::Class Element

7.12.2.2.2 Subclasses:

7.12.2.2.2.1 Class Strategy::Course of Action

7.12.2.2.2.2 Class Initiative::Initiative

7.12.2.2.2.3 Class BMM::Analysis

7.12.2.2.3 Associations:

7.12.2.2.3.1.1 Association:comprises Type:Aggregation

Source: Strategy::Course of Action **Role:** **Cardinality:**

Target: Strategy::Course of Action **Role:** **Cardinality:**

7.12.2.2.3.1.1.1 *Constraints:*

7.12.2.2.3.1.1.2 Association:subject Type:Association

Source: Strategy::Course of Action **Role:** **Cardinality:**

Target: Strategy::expected to result in **Role:** **Cardinality:**

7.12.2.2.3.1.2.1 *Constraints:*

7.12.2.2.3.1.3 Association: Type:Association

Source: Strategy::Course of Action **Role:** **Cardinality:** 1..*

Target: Strategy::Strategy **Role:** **Cardinality:** 0..*

7.12.2.2.3.1.3.1 *Constraints:*

7.12.2.2.3.1.4 Association:subject Type:Association

Source: Strategy::Course of Action **Role:** **Cardinality:**

Target: Strategy::needs **Role:** **Cardinality:**

7.12.2.2.3.1.4.1 *Constraints:*

7.12.2.2.3.1.5 Association:comprises Type:Aggregation

Source: Strategy::Course of Action **Role:** **Cardinality:**

Target: Strategy::Course of Action **Role:** **Cardinality:**

7.12.2.2.3.1.5.1 *Constraints:*

7.12.2.2.4 Attributes:

7.12.2.2.5 Methods:

7.12.2.3 Class Objective

in package 'Strategy'

An "Objective" represents a desire to change something. In a strategy setting, typically an objective describes a difference between the way things actually are (the ASIS model) and the way things are desired to be (the TOBE model).

Note: the metamodel does not currently contain a class that refers to particular models that are instances of the metamodel. Such a capability is not typically found in MOF metamodels but could be added by means of a circumlocution - a device such as a text string that is understood to identify a particular model. However, there are details to be worked out about whether the ASIS and TOBE models are separately named models, or versions of a single model (permitted by some modeling tools). In addition, a new type of model element would need to be introduced that is capable of acting as a local proxy for an element that is actually defined elsewhere.

A strategic objective implies a change in some value items, including the creation of new value items and new value streams to provide those value items. (BIZBOK® Guide v6.5, section 2.1)

7.12.2.3.1 Superclasses

7.12.2.3.1.1 Class Strategy::ObjectiveConcept

7.12.2.3.2 Subclasses:

7.12.2.3.2.1 Class BMM::Analysis

7.12.2.3.3 Associations:

7.12.2.3.3.1.1 Association:subject Type:Association

Source: Strategy::Objective **Role:** **Cardinality:**

Target: Strategy::Change **Role:** **Cardinality:**

7.12.2.3.3.1.1.1 *Constraints:*

7.12.2.3.3.1.2 Association: Type:Association

Source: Strategy::Strategy **Role:** **Cardinality:** 0..*

Target: Strategy::Objective **Role:** **Cardinality:** 1..*

7.12.2.3.3.1.2.1 *Constraints:*

7.12.2.3.3.1.3 Association:achieves Type:Association

Source: Initiative::Initiative **Role:** **Cardinality:**

Target: Strategy::Objective **Role:** **Cardinality:**

7.12.2.3.3.1.3.1 *Constraints:*

Constraint: specializes "expected to result in" from Strategy Invariant

The **achieves** association specializes the **expected to result in** association of Strategy. It models the concept that the Objective is more certain to be achieved because of the additional planning and analysis associated with defining the Initiative.

7.12.2.3.3.1.4 Association:object Type:Association

Source: Strategy::expected to result in **Role:** **Cardinality:**

Target: Strategy::Objective **Role:** **Cardinality:**

7.12.2.3.3.1.4.1 *Constraints:*

7.12.2.3.3.1.5 Association:object Type:Association

Source: Strategy::needs **Role:** **Cardinality:**

Target: Strategy::Objective **Role:** **Cardinality:**

7.12.2.3.3.1.5.1 *Constraints:*

7.12.2.3.4 Attributes:

7.12.2.3.5 Methods:

7.12.2.4 Class ObjectiveConcept

in package 'Strategy'

7.12.2.4.1 Superclasses

7.12.2.4.1.1 Class Foundations::Class Element

7.12.2.4.2 Subclasses:

7.12.2.4.2.1 Class Strategy::ObjectiveConcept

7.12.2.4.2.2 Class Strategy::Objective

7.12.2.4.2.3 Class Requirements::Requirement

7.12.2.4.3 Associations:

7.12.2.4.3.1.1 Association:comprises Type:Aggregation

Source: Strategy::ObjectiveConcept **Role:** **Cardinality:**

Target: Strategy::ObjectiveConcept **Role:** **Cardinality:**

7.12.2.4.3.1.1.1 *Constraints:*

7.12.2.4.3.1.2 Association:comprises Type:Aggregation

Source: Strategy::ObjectiveConcept **Role:** **Cardinality:**

Target: Strategy::ObjectiveConcept **Role:** **Cardinality:**

7.12.2.4.3.1.2.1 *Constraints:*

7.12.2.4.4 Attributes:

7.12.2.4.5 Methods:

7.12.2.5 Class Rationale

in package 'Strategy'

The "Rationale" class represents an explanation about why the CourseOfAction is expected to achieve the Objective or why the CourseOfAction needs to have the Objective be the case in order to be taken. The "Rationale" class does not have detail in the metamodel; it is expected that the architect will use this element to record a prose rationale.

7.12.2.5.1 Superclasses

7.12.2.5.1.1 Class Foundations::Business Element

7.12.2.5.2 Subclasses:

7.12.2.5.3 Associations:

7.12.2.5.3.1.1 Association:rationale for Type:Association

Source: Strategy::Rationale **Role:** **Cardinality:**

Target: Strategy::Change **Role:** **Cardinality:**

7.12.2.5.3.1.1.1 *Constraints:*

7.12.2.5.3.1.2 Association:rationale for Type:Association

Source: Strategy::Rationale **Role:** **Cardinality:**

Target: Strategy::needs **Role:** **Cardinality:**

7.12.2.5.3.1.2.1 *Constraints:*

7.12.2.5.3.1.3 Association:rationale for Type:Association

Source: Strategy::Rationale **Role:** **Cardinality:**

Target: Strategy::expected to result in **Role:** **Cardinality:**

7.12.2.5.3.1.3.1 *Constraints:*

7.12.2.5.4 Attributes:

7.12.2.5.5 Methods:

7.12.2.6 Class Strategy

in package 'Strategy'

A Strategy is a collection of Objectives and optionally CoursesofAction that are associated with the Objectives. A Strategy should be coherent in that it contains no irrelevant Objectives and all CoursesofAction are related to some Objectives.

Anticipating that a model may contain several strategy versions, Objectives and CoursesofAction may be shard between Strategy instances.

7.12.2.6.1 Superclasses

7.12.2.6.1.1 Class BMM::Analysis

7.12.2.6.2 Subclasses:

7.12.2.6.3 Associations:

7.12.2.6.3.1.1 Association: Type:Association

Source: Strategy::Strategy **Role:** **Cardinality:** 0..*

Target: Strategy::Objective **Role:** **Cardinality:** 1..*

7.12.2.6.3.1.1.1 *Constraints:*

7.12.2.6.3.1.2 Association: Type:Association

Source: Strategy::Course of Action **Role:** **Cardinality:** 1..*

Target: Strategy::Strategy **Role:** **Cardinality:** 0..*

7.12.2.6.3.1.2.1 *Constraints:*

7.12.2.6.3.1.3 Association: Type:Association

Source: BACM_Model::StrategyMap **Role:** **Cardinality:**

Target: Strategy::Strategy **Role:** **Cardinality:** 0..*

7.12.2.6.3.1.3.1 *Constraints:*

7.12.2.6.4 Attributes:

7.12.2.6.5 Methods:

7.12.2.7 Class expected to result in *in package 'Strategy'*

The "expected to result in" element represents a 3-ary relationship between a "Course of Action", an "Objective", and a "Rationale". The associations connecting the "Class Elements" to the "Role Element" indicate how to read this relationship as prose: "The course of action is expected to result in achievement of the objective because of the rationale."

7.12.2.7.1 Superclasses

7.12.2.7.1.1 Class Foundations::Role Element

7.12.2.7.2 Subclasses:

7.12.2.7.3 Associations:

7.12.2.7.3.1.1 Association:object Type:Association

Source: Strategy::expected to result in **Role:** **Cardinality:**

Target: Strategy::Objective **Role:** **Cardinality:**

7.12.2.7.3.1.1.1 *Constraints:*

7.12.2.7.3.1.2 Association:subject Type:Association

Source: Strategy::Course of Action **Role:** **Cardinality:**

Target: Strategy::expected to result in **Role:** **Cardinality:**

7.12.2.7.3.1.2.1 *Constraints:*

7.12.2.7.3.1.3 Association:rationale for Type:Association

Source: Strategy::Rationale **Role:** **Cardinality:**

Target: Strategy::expected to result in **Role:** **Cardinality:**

7.12.2.7.3.1.3.1 *Constraints:*

7.12.2.7.4 Attributes:

7.12.2.7.5 Methods:

7.12.2.8 Class needs *in package 'Strategy'*

The "needs" class represents a 3-ary relationship between an "Objective", a "Course of Action" and a "Rationale". The labels on the associations connecting the "Class Elements" classes and the "needs" class indicate how the relationship should be read as prose; e.g. "The course of action needs the objective because of the rationale".

The "needs" class specializes the "Role Element", indicating that this class represents a relationship

7.12.2.8.1 Superclasses

7.12.2.8.1.1 Class Foundations::Role Element

7.12.2.8.2 Subclasses:

7.12.2.8.3 Associations:

7.12.2.8.3.1.1 Association:object Type:Association

Source: Strategy::needs **Role:** **Cardinality:**

Target: Strategy::Objective **Role:** **Cardinality:**

7.12.2.8.3.1.1.1 *Constraints:*

7.12.2.8.3.1.2 Association:rationale for Type:Association

Source: Strategy::Rationale **Role:** **Cardinality:**

Target: Strategy::needs **Role:** **Cardinality:**

7.12.2.8.3.1.2.1 *Constraints:*

7.12.2.8.3.1.3 Association:subject Type:Association

Source: Strategy::Course of Action **Role:** **Cardinality:**

Target: Strategy::needs **Role:** **Cardinality:**

7.12.2.8.3.1.3.1 *Constraints:*

7.12.2.8.4 Attributes:

7.12.2.8.5 Methods:

7.13 Initiative

Package in package 'Model'

7.13.1 Initiative diagram

Class diagram in package 'Initiative'

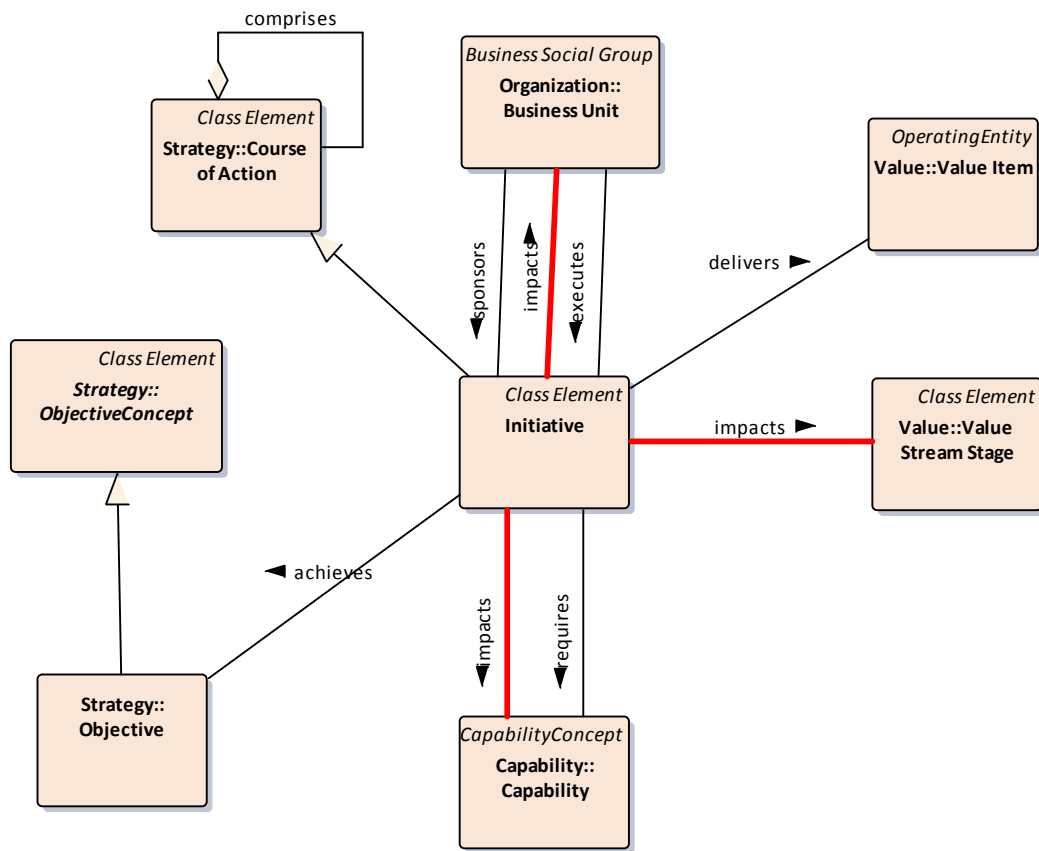


Figure 14: Initiative

Initiatives are courses of actions that have been selected for execution. In addition to having the properties and associations of a "Course of Action", an Initiative is **sponsored** by a BusinessUnit and is **executed** by a BusinessUnit (that may be different from the sponsoring BusinessUnit).

An Initiative will **impact** a BusinessUnit by making changes in the configuration of that BusinessUnit such as job assignments and number and skills of members of the BusinessUnit.

In the planning of an Initiative, it is useful to model the Capabilities that will be **required** to execute the Initiative. These Capabilities should be supplied by the BusinessUnit that will **execute** the Initiative.

7.13.2 Classes:

7.13.2.1 Class Initiative

in package 'Initiative'

Definition: "A course of action that is being executed or has been selected for execution."

Initiative is a specialization of "Course of Action". An initiative differs from a "Course of Action" in having specific details such as budgets, resource assignments and timelines.

Initiatives can be categorized by association with a Category element (see Foundations). Category elements may be a part of a category hierarchy.

7.13.2.1.1 Superclasses

7.13.2.1.1.1 Class Strategy::Course of Action

7.13.2.1.1.2 Class Foundations::Class Element

7.13.2.1.1.2 Subclasses:

7.13.2.1.1.3 Associations:

7.13.2.1.3.1.1 Association:requires Type:Association

Source: Initiative::Initiative **Role:** **Cardinality:**

Target: Capability::Capability **Role:** **Cardinality:**

7.13.2.1.3.1.1.1 *Constraints:*

The **requires** association models the notion that a specific Capability is needed in order to execute the Initiative. This association is used in Initiative planning to establish requirements.

7.13.2.1.3.1.2 Association:impacts Type:Association

Source: Initiative::Initiative **Role:** **Cardinality:**

Target: Capability::Capability **Role:** **Cardinality:**

7.13.2.1.3.1.2.1 *Constraints:*

The **impacts** association generalizes the concept that the Initiative has an Objective to make **changes** in a CapabilityInstance or CapabilityBehavior associated with the target Capability. This association must be consistent with the details when present in a model.

7.13.2.1.3.1.3 Association:delivers Type:Association

Source: Initiative::Initiative **Role:** **Cardinality:**

Target: Value::Value Item **Role:** **Cardinality:**

7.13.2.1.3.1.3.1 *Constraints:*

The **delivers** association models the concept that an Objective of the Initiative delivers a new version of a ValueItem (i.e. the ValueItem is the result **after** the **change** implied by the Objective).

7.13.2.1.3.1.4 Association:impacts Type:Association

Source: Initiative::Initiative **Role:** **Cardinality:**

Target: Value::Value Stream Stage **Role:** **Cardinality:**

7.13.2.1.3.1.4.1 *Constraints:*

The **impacts** association generalizes the concept that the Initiative achieves an Objective of making a **change** to a ValueItem that is produced in a ValueStream (see Strategy and Value packages). Such an association instance must be consistent with model details when such details are present in the model.

7.13.2.1.3.1.5 Association:achieves Type:Association

Source: Initiative::Initiative **Role:** **Cardinality:**

Target: Strategy::Objective **Role:** **Cardinality:**

7.13.2.1.3.1.5.1 *Constraints:*

Constraint: specializes "expected to result in" from Strategy Invariant

The **achieves** association specializes the **expected to result in** association of Strategy. It models the concept that the Objective is more certain to be achieved because of the additional planning and analysis associated with defining the Initiative.

7.13.2.1.3.1.6 Association:executes Type:Association

Source: Organization::Business Unit **Role:** **Cardinality:**

Target: Initiative::Initiative **Role:** **Cardinality:**

7.13.2.1.3.1.6.1 *Constraints:*

7.13.2.1.3.1.7 Association:sponsors Type:Association

Source: Organization::Business Unit **Role:** **Cardinality:**

Target: Initiative::Initiative **Role:** **Cardinality:**

7.13.2.1.3.1.7.1 *Constraints:*

7.13.2.1.3.1.8 Association: Type:Association

Source: BACM_Model::InitiativeMap **Role:** **Cardinality:**

Target: Initiative::Initiative **Role:** **Cardinality:** 0..*

7.13.2.1.3.1.8.1 *Constraints:*

7.13.2.1.3.1.9 Association: Type:Association

Source: TOGAF::CapabilityIncrement **Role:** **Cardinality:**

Target: Initiative::Initiative **Role:** **Cardinality:**

7.13.2.1.3.1.9.1 *Constraints:*

7.13.2.1.3.1.10 Association:impacts Type:Association

Source: Organization::Business Unit **Role:** **Cardinality:**

Target: Initiative::Initiative **Role:** **Cardinality:**

7.13.2.1.3.1.10.1 *Constraints:*

The **impacts** association generalizes the concept that an Objective of the Initiative is to make a change in a Business Unit. Such associations must be consistent with details when details are added to the model.

7.13.2.1.3.1.11 Association:supported by Type:Association

Source: Requirements::Requirement **Role:** **Cardinality:**

Target: Initiative::Initiative **Role:** **Cardinality:**

7.13.2.1.3.1.11.1 *Constraints:*

Requirements define the scope, objectives, and measures for success of an Initiative.

BIZBOK® Guide Reference Section 3.8 Business Architecture and Requirements Alignment:

Per Section: Why Requirements Alignment

Initiative (Project) > supported by > requirement

Per Section: BA and Requirements Alignment

Requirement > links to > initiative

Per Section: BA and Requirements Alignment in Practice

Requirement > links to > Initiative (project scope)

7.13.2.1.4 Attributes:

7.13.2.1.5 Methods:

7.14 Requirements

Package in package 'Model'

7.14.1 Requirements diagram

Class diagram in package 'Requirements'

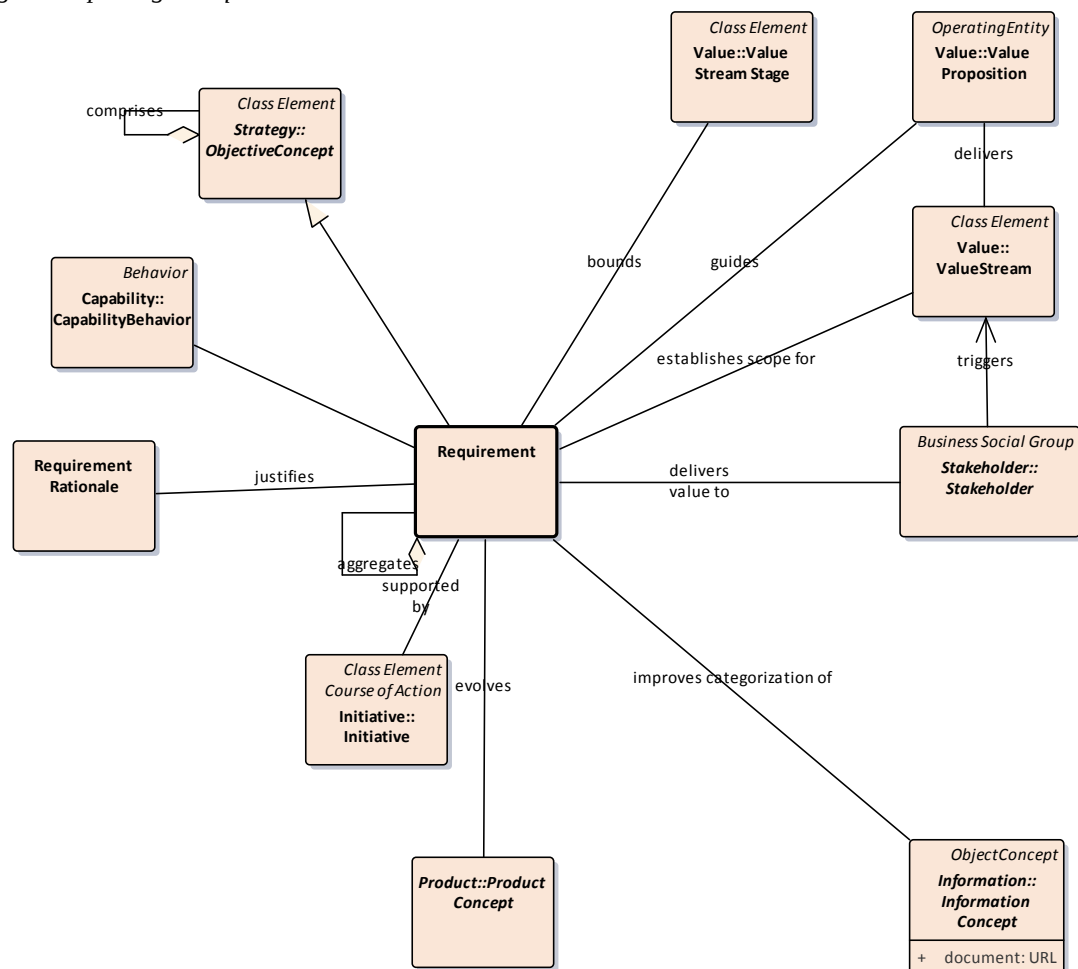


Figure 15: Requirements

Scope:

Focused on a specification only as it pertains to directing changes to a Product, an Object, AND enabling a Capability.

Core Touch points:

- Requirements - requirements aggregate (up & down) into requirements (at various levels of granularity and type)
- Capability - requirements link to Capability Behavior since we are specifying "how" a capability will get delivered
- Product - requirement evolves the Product at a conceptual level, which is reflected in one or more Product Instances. Requirements will specify how a product will change via changes to its design
- Object Concept - In almost every case, information is touched when changes are made, so it's important to influence include both Information Concept & Business Object
- Information - Requirement is framed by the business glossary, which are the terms used to guide the information architecture. Information establishes the scope and categorization for requirements. We are linking at the Information Concept to address the above alignment.
- Stakeholder - requirement delivers value to the stakeholder, while creating clarity tailored to each stakeholder perspective
- Value Stream & Value Stage - Value Stream clarifies how value should be delivered by a requirement. Whereas Value Stage provides the boundary for what is included in a requirement. Value Stream also enables reuse of requirements across initiatives (projects). Value Stream also provides context for requirement.
- Strategy - As strategy is somewhat amorphous, we link to the strategy concept to simplify the alignment and to ensure requirement aligns with the strategic outcomes
- Initiative - Typically referred to as a project (but not limited to a project) defines the scope, objectives, and measures for success to guide the requirements used to specify the design for a solution (waterfall or agile).

Other considerations:

We may want to discuss alignment (via Proxy) to other Meta Models as Requirements will be influenced by one or more of the following:

- Requirement MM from IEEE
- Structured Metrics MM (SMM)
- Business Motivation Model (BMM)

7.14.2 Classes:

7.14.2.1 Class Requirement

in package 'Requirements'

Business requirements are the core expressions for describing what a business needs or wants. A requirement is a condition or capability needed by a stakeholder to solve a problem or achieve an objective. Requirements are met or satisfied by a solution in order to satisfy a contract, standard, specification, or other formally imposed condition. Requirements describe the desired Outcome of a solution whose methods and means are left unspecified in order to promote innovation.

Instances of this class are used to represent requirements. Typically requirements are managed in a tool that is separate from the business architecture modeling tool. Most of the time, these Requirement instances will be proxies for the real requirements managed in a separate model. Requirements are validated by fulfilling some part of a strategy Objective that relates to a ValueProposition and a ValueStream. This validation is done in the context of the Business Architecture model that defines and integrates the ValueStreams, Strategies and Capabilities of the business.

7.14.2.1.1 Superclasses

7.14.2.1.1.1 Class Strategy::ObjectiveConcept

7.14.2.1.2 Subclasses:

7.14.2.1.2.1 Class Requirements::Requirement

7.14.2.1.3 Associations:

7.14.2.1.3.1.1 Association:aggregates Type:Aggregation

Source: Requirements::Requirement **Role:** **Cardinality:**

Target: Requirements::Requirement **Role:** **Cardinality:**

7.14.2.1.3.1.1.1 *Constraints:*

Requirements aggregate (up & down) into requirements (at various levels of granularity and type)

BIZBOK® Guide Section 3.8 Reference:

Per Section: BA and Requirements Alignment in Practice

Requirement > aggregates > Requirement

7.14.2.1.3.1.2 Association:justifies Type:Association

Source: Requirements::Requirement **Role:** **Cardinality:**

Target: Requirements::Requirement Rationale **Role:** **Cardinality:**

7.14.2.1.3.1.2.1 *Constraints:*

7.14.2.1.3.1.3 Association:evolves Type:Association

Source: Requirements::Requirement **Role:** **Cardinality:**

Target: Product::Product Concept **Role:** **Cardinality:**

7.14.2.1.3.1.3.1 *Constraints:*

Requirements evolve a Product at a conceptual level, which is reflected in one or more Product Instances. Requirements will specify how a product will change via changes to its design.

The "evolves" association is a specialization of the "Changes" association (see Strategy package). In this case, the "before" and "after" ProductConcepts are variants.

BIZBOK® Guide Section 3.8 Reference:

Per Section: Requirements Alignment with Products

Requirement > evolves > Product

7.14.2.1.3.1.4 Association:delivers value to Type:Association

Source: Requirements::Requirement **Role:** **Cardinality:**

Target: Stakeholder::Stakeholder **Role:** **Cardinality:**

7.14.2.1.3.1.4.1 *Constraints:*

BIZBOK® Guide Section 3.8 Reference:

Per Section: Benefits of BA and Requirements Alignment

Requirement > delivers value to > stakeholder

Per Section: Principle of BA and Requirements Alignment

Requirements > based on clearly articulated > stakeholder perception

Per Section: BA and Requirements Alignment

Requirement > links to > stakeholder

7.14.2.1.3.1.5 Association:supported by Type:Association

Source: Requirements::Requirement **Role:** **Cardinality:**

Target: Initiative::Initiative **Role:** **Cardinality:**

7.14.2.1.3.1.5.1 *Constraints:*

Requirements define the scope, objectives, and measures for success of an Initiative.

BIZBOK® Guide Section 3.8 Reference:

Per Section: Why Requirements Alignment

Initiative (Project) > supported by > requirement

Per Section: BA and Requirements Alignment

Requirement > links to > initiative

Per Section: BA and Requirements Alignment in Practice

Requirement > links to > Initiative (project scope)

7.14.2.1.3.1.6 Association: Type:Association

Source: Requirements::Requirement **Role:** **Cardinality:**

Target: Capability::CapabilityBehavior **Role:** **Cardinality:**

7.14.2.1.3.1.6.1 *Constraints:*

7.14.2.1.3.1.7 Association:improves categorization of Type:Association

Source: Information::Information Concept **Role:** **Cardinality:**

Target: Requirements::Requirement **Role:** **Cardinality:**

7.14.2.1.3.1.7.1 *Constraints:*

BIZBOK® Guide Section 3.8 Reference:

Per Section: Benefits of BA and Requirements Alignment

information > establishes scope for > requirement

Information > improves categorization of > requirements

Per Section: Requirements Alignment with Information

Requirement > links to > Information (Concepts)

7.14.2.1.3.1.8 Association:guides Type:Association

Source: Value::Value Proposition **Role:** **Cardinality:**

Target: Requirements::Requirement **Role:** **Cardinality:**

7.14.2.1.3.1.8.1 *Constraints:*

A Requirement is intended to (indirectly) affect a ValueProposition in the context of achieving a strategy Objective. The Requirement is said to be "guided" by the new ValueProposition.

BIZBOK® Guide Section 3.8 Reference:

Per Section: Benefits of BA and Requirements Alignment

Requirement > links to > Business (Value) Proposition

7.14.2.1.3.1.9 Association: Type:Association

Source: BACM_Model::RequirementMap **Role:** **Cardinality:**

Target: Requirements::Requirement **Role:** **Cardinality:** 0..*

7.14.2.1.3.1.9.1 *Constraints:*

7.14.2.1.3.1.10 Association:aggregates Type:Aggregation

Source: Requirements::Requirement **Role:** **Cardinality:**

Target: Requirements::Requirement **Role:** **Cardinality:**

7.14.2.1.3.1.10.1 *Constraints:*

Requirements aggregate (up & down) into requirements (at various levels of granularity and type)

BIZBOK® Guide Section 3.8 Reference:

Per Section: BA and Requirements Alignment in Practice

Requirement > aggregates > Requirement

7.14.2.1.3.1.11 Association:establishes scope for Type:Association

Source: Value::ValueStream **Role:** **Cardinality:**

Target: Requirements::Requirement **Role:** **Cardinality:**

7.14.2.1.3.1.11.1 *Constraints:*

A Requirement is intended to assist in fulfilling a strategic Objective. Typically, the Objective is framed in terms of affecting ValuePropositions associated with ValueStreams. Thus, the requirement has a scope established by one or more ValueStreams.

BIZBOK® Guide Section 3.8 Reference:

Per Section: Benefits of BA and Requirements Alignment

Value stream > establishes scope for > requirement

Value Stream > clarifies how value is delivered by > requirement

Value streams > frame the context for > requirements

Value stream > enables reuse of > requirements

Per Section: Requirements Alignment with Value Stream

Requirement > is contextualized by > Value Stream

Per Section: BA and Requirements Alignment in Practice

Requirement > links to > Value Stream

7.14.2.1.3.1.12 Association:bounds Type:Association

Source: Value::Value Stream Stage **Role:** Cardinality:

Target: Requirements::Requirement **Role:** Cardinality:

7.14.2.1.3.1.12.1 *Constraints:*

It should be rare that a Requirement affects changes in Outcomes associated with multiple ValueStreamStages of a ValueStream. The Requirement should be considered for splitting when this situation occurs. The "bounds" association allows the modeler to track the occurrence of such situations.

BIZBOK® Guide Section 3.8 Reference:

Per Section: Principle of BA and Requirements Alignment

Requirements > bounded by > Value Stream Stage

7.14.2.1.4 Attributes:

7.14.2.1.5 Methods:

7.14.2.2 Class Requirement Rationale

in package 'Requirements'

7.14.2.2.1 Superclasses

7.14.2.2.2 Subclasses:

7.14.2.2.3 Associations:

7.14.2.2.3.1.1 Association:justifies Type:Association

Source: Requirements::Requirement **Role:** Cardinality:

Target: Requirements::Requirement Rationale **Role:** Cardinality:

7.14.2.2.3.1.1.1 *Constraints:*

7.14.2.2.4 Attributes:

7.14.2.2.5 Methods:

7.15 Alignment

7.15.1 Alignment diagram

Class diagram in package 'Alignment'

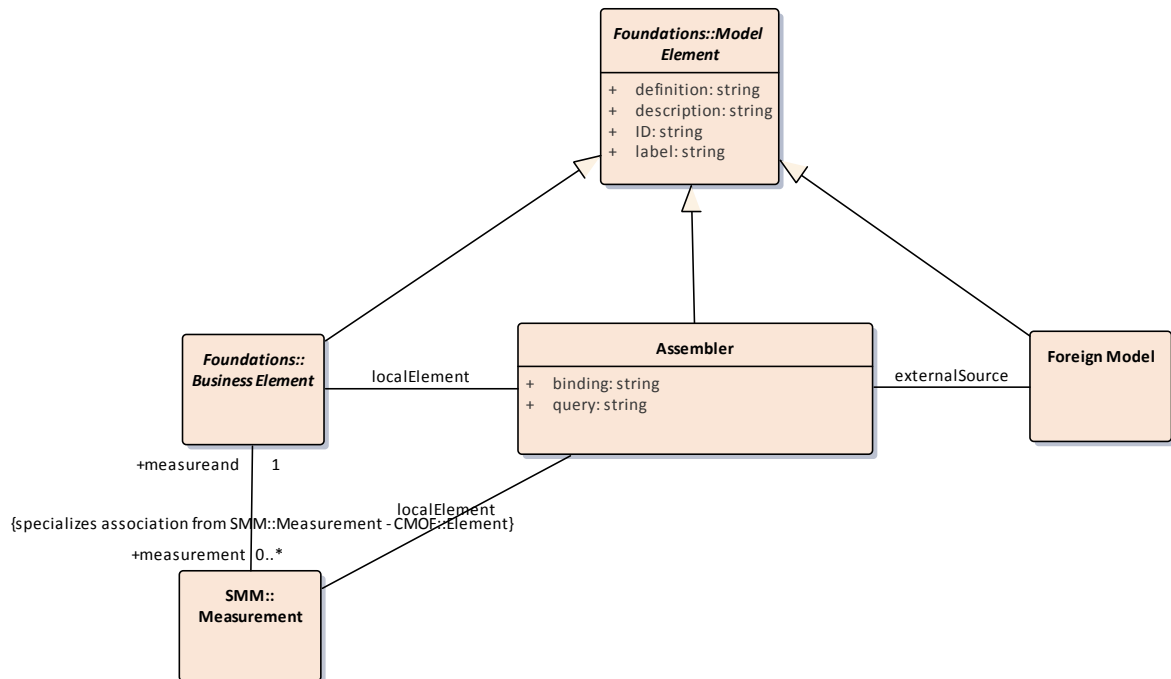


Figure 16: Alignment

When **Assembler** is realized in a model, the modeler can add properties to the **Assembler** object (as one would do for a UML class). The modeler can also specify a query as the default value for the query attribute of the object.

The modeling tool should provide a means for the modeler to invoke the query, retrieve the result set and bind the first member of the result set to the modeler defined attributes. In the absence of a binding string, the modeling tool should match the item names (or aliases) of the query (and result set) to the attribute names. A binding string may be used to customize this binding (the syntax of this string is undefined at present).

An **Assembler** instance may not stand alone in the business architecture model; it must be associated to a **BusinessElement** by means of the "localElement" association.

Because there may be many instances of **Assembler** that query the same external model, the binding of a query to a specific external model is separated into the **ForeignModel** class. Instances of this class behave like database connections.

The **Assembler** instance can be associated with a **BusinessElement** instance or an **SMM::Measurement** instance, but not both. The former pattern is intended to support the retrieval of non-measurement information from a foreign model, while the latter is intended to support the retrieval of measurement information from the foreign model.

7.15.2 Classes:

7.15.2.1 Class Assembler

in package 'Alignment'

The **Assembler** class provides two attributes in every instance: a query attribute for the query default value and a binding attribute for customizing the bindings between the columns of the result set and the modeler defined attributes on the **Assembler** instance object.

The modeler must be able to add typed attributes to instances of the Assembler.

The Assembler includes a method to cause execution of the query and a set of methods to manipulate a cursor on the result set. A side effect of changing the cursor is to update the modeler specified attribute values from the result set cursor position. On successful execution of the query, the binding is set to the first member of the result set. The modeling tool should be prepared to deal with query errors and null result sets.

7.15.2.1.1 Superclasses

7.15.2.1.1.1 Class Foundations::Model Element

7.15.2.1.2 Subclasses:

7.15.2.1.3 Associations:

7.15.2.1.3.1.1 Association:externalSource Type:Association

Source: Alignment::Assembler **Role:** **Cardinality:**

Target: Alignment::Foreign Model **Role:** **Cardinality:**

7.15.2.1.3.1.1.1 Constraints:

The externalSource association binds a query string to a ForeignSource model object allowing the modeling tool to find the connection to a query engine and pass it the query and the identifier of the foreign model to be queried.

7.15.2.1.3.1.2 Association:localElement Type:Association

Source: Alignment::Assembler **Role:** **Cardinality:**

Target: Foundations::Business Element **Role:** **Cardinality:**

7.15.2.1.3.1.2.1 Constraints:

The localElement association links an Assembler model object to a model object instance of any concrete subclass of the BusinessElement class.

7.15.2.1.3.1.3 Association:localElement Type:Association

Source: Alignment::Assembler **Role:** **Cardinality:**

Target: SMM::Measurement **Role:** **Cardinality:**

7.15.2.1.3.1.3.1 Constraints:

This local Element association links an Assembler instance to an instance of a SMM::Measurement. This association allows the Assembler query to retrieve information from a foreign model and expose that information in the business architecture model as an SMM::Measurement.

When using this association, the architect should NOT create a localElement association to the BusinessElement that is the measurand of the SMM::Measurement.

7.15.2.1.3.1.4 Association: Type:Association

Source: BACM_Model::AlignmentLibrary **Role:** **Cardinality:**

Target: Alignment::Assembler **Role:** **Cardinality:** 0..*

7.15.2.1.3.1.4.1 Constraints:

7.15.2.1.4 Attributes:

7.15.2.1.4.1.1 Attribute: Assembler.binding Type: string

A string allowing for customization of the binding between the columns of the result set and the modeler defined attributes of the Assmebler object.

7.15.2.1.4.1.2 Attribute: Assembler.query Type: string

This attribute holds a query string as its default value. The query string is sent to the ForeignModel object for execution against the foreign model, returning a result set if the query is properly formed and the result is not null.

7.15.2.1.5 Methods:

7.15.2.2 Class Foreign Model

in package 'Alignment'

This class is used to represent a foreign model. It contains information needed to access the foreign model and perform the query against its contents (e.g. a filename string for an XML file that serializes a model and can be queries by means of an XQuery processor).

7.15.2.2.1 Superclasses

7.15.2.2.1.1 Class Foundations::Model Element

7.15.2.2.2 Subclasses:

7.15.2.2.3 Associations:

7.15.2.2.3.1.1 Association:externalSource Type:Association

Source: Alignment::Assembler **Role:** **Cardinality:**

Target: Alignment::Foreign Model **Role:** **Cardinality:**

7.15.2.2.3.1.1.1 *Constraints:*

The externalSource association binds a query string to a ForeignSource model object allowing the modeling tool to find the connection to a query engine and pass it the query and the identifier of the foreign model to be queried.

7.15.2.2.3.1.2 Association: Type:Association

Source: BACM_Model::AlignmentLibrary **Role:** **Cardinality:**

Target: Alignment::Foreign Model **Role:** **Cardinality:** 0..*

7.15.2.2.3.1.2.1 *Constraints:*

7.15.2.2.4 Attributes:

7.15.2.2.5 Methods:

8 Alignment Guidelines (non-normative)

Section 8 defines the non-normative guidelines that are focused on alignment to related standards as follows.

Business Motivation Model (BMM) Alignment Package: The BMM alignment package identifies points of interaction between BMM and the business architecture strategy package where objective and course of action are aligned between BMM and the strategy package.

TOGAF Capability-based Planning Alignment Package: This package enables BACM to align to the TOGAF concept of a capability increment to enable capability-based planning.

Value Delivery Modeling Language™ (VDML™) Alignment Package: The VDML alignment package associates aspects of VDML with value related elements within BACM, including value stream, value item, and value proposition, and with stakeholder.

Business Process Modeling Notation (BPMN) Alignment Package: The BPMN alignment package establishes the connection between process and business architecture using the BPMN 2.0 standard.

Case Management Modeling Notation (CMMN) Alignment Package: The CMMN alignment discussion establishes the connection between case management and business architecture using the CMMN standard.

Decision Management Notation (DMN) Alignment Package: The DMN alignment discussion establishes the connection between case management and business architecture using the DMN standard.

8.1 BACM Alignment with BMM

The BACM aligns to the Business Motivation Model (BMM) as describe herein.

8.1.1 BMM diagram

Class diagram in package 'BMM'

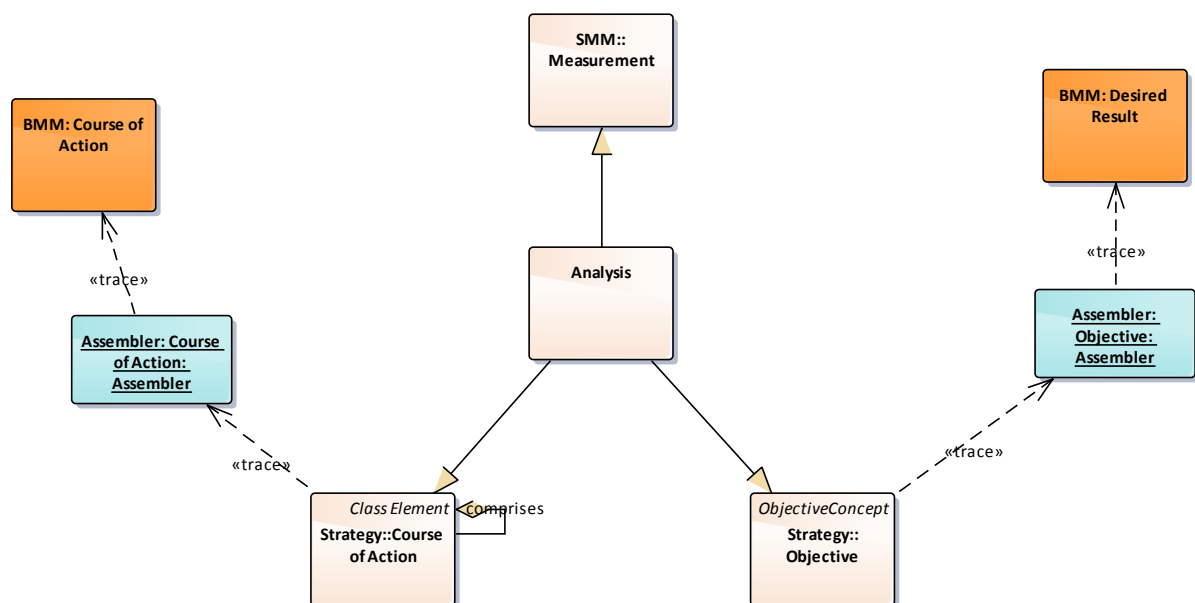


Figure 17: BMM

8.1.2 Classes:

8.1.2.1 Class Analysis

in package 'BMM'

This class is used to address the activities associated with moving from an objective (as part of establishing strategy) to one or more courses of action. The Analysis activities act upon the information (typically in the form of measurements) to guide decision making as to one or more courses of action. In addition, the Analysis can also be used to modify / update a given course of action based on new Measurements and/or changes to existing Measurements.

8.1.2.1.1 Superclasses

8.1.2.1.1.1 *Class Strategy::Objective*

8.1.2.1.1.2 *Class SMM::Measurement*

8.1.2.1.1.3 *Class Strategy::Course of Action*

8.1.2.1.2 Subclasses:

8.1.2.1.2.1 *Class Strategy::Strategy*

8.1.2.1.3 Associations:

8.1.2.1.4 Attributes:

8.1.2.1.5 Methods:

8.1.2.2 Class BMM: Course of Action

in package 'BMM'

8.1.2.3 Class BMM: Desired Result

in package 'BMM'

8.1.3 Assembler: Course of Action

Object in package 'BMM'

This class instance of Assembler is used to form a touch point between the BMM and BACM metamodels via the Course of Action Class (as it relates to the MEANS in the BMM) and the Course of Action Class (as it relates to strategy in the BACM).

CONNECTORS	
 Trace «trace» Source -> Destination	
From:	Assembler: Course of Action : Object, Public
To:	BMM: Course of Action : Class, Public
 Trace «trace» Source -> Destination	
From:	Course of Action : Class, Public
To:	Assembler: Course of Action : Object, Public

8.1.4 Assembler: Objective

Object in package 'BMM'

This class instance of Assembler is used to form a touch point between the BMM and BACM metamodels via the abstraction between a Desired Result (as it relates to the END in the BMM) and the Objective class (as it relates to strategy in the BACM).

CONNECTORS

Trace «trace» Source -> Destination
 From: Assembler: Objective : Object, Public
 To: BMM: Desired Result : Class, Public

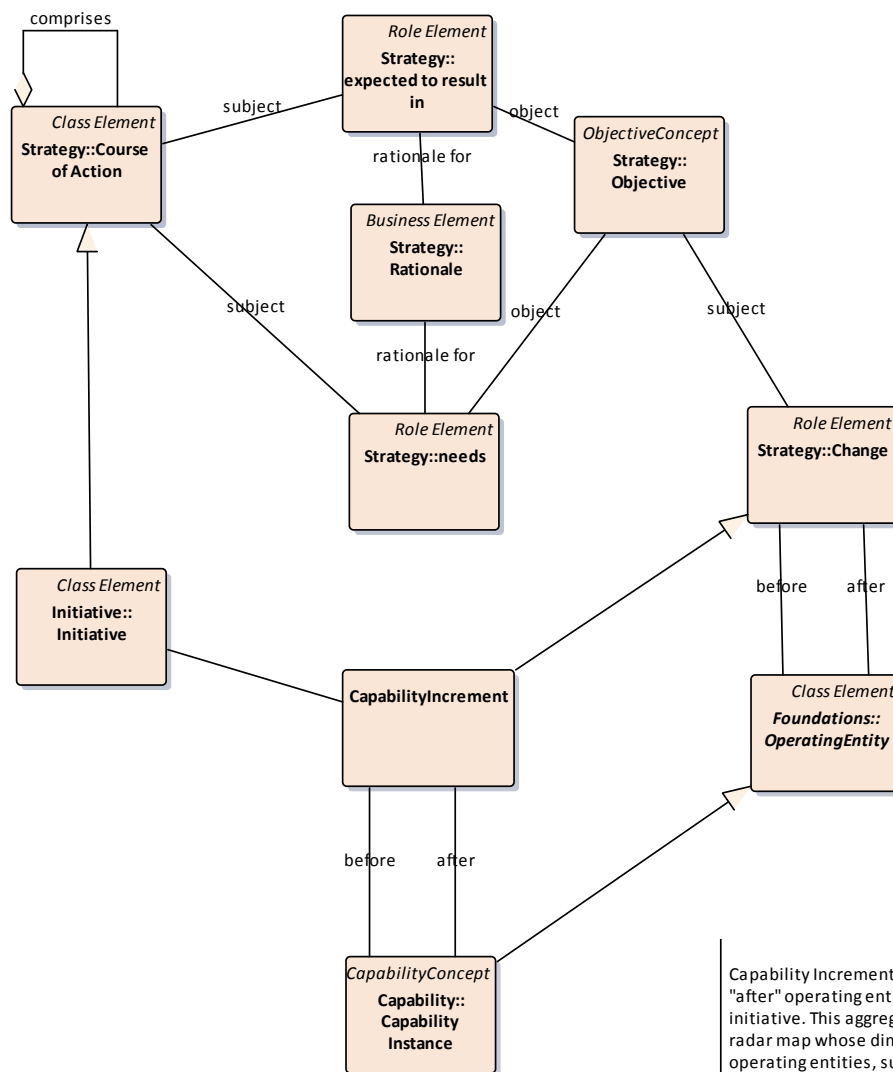
Trace «trace» Source -> Destination
 From: Objective : Class, Public
 To: Assembler: Objective : Object, Public

8.2 BACM Alignment with TOGAF Capability-based Planning

The BACM is aligned to TOGAF for purposes of capability-based planning, which focuses on the “capability increment” specified in TOGAF.

8.2.1 CapabilityIncrement diagram

Class diagram in package 'TOGAF'



Capability Increment represents an aggregation of "after" operating entities expected to result from an initiative. This aggregation is often used to create a radar map whose dimensions are aspects of the operating entities, such as progress in implementing a reengineered business process.

Figure 18: CapabilityIncrement

The CapabilityIncrement diagram illustrates how a business architecture model can be aligned with a TOGAF capability-based planning model. According to the TOGAF standard, the CapabilityIncrement is a version of a capability that represents a change in the performance/maturity of the capability. (Note: the TOGAF standard defines capability as a configuration of people, processes and material; the TOGAF capability aligns with the CapabilityInstance in business architecture.)

The CapabilityIncrement is seen here as a specialization of the Change element defined in the Strategy package. Change defines a relationship between a Strategy::Objective and before and after versions of an OperatingEntity.

8.2.2 Classes:

8.2.2.1 Class CapabilityIncrement

in package 'TOGAF'

In TOGAF, a CapabilityIncrement is a version of a capability that represents a change in the performance/maturity of the capability. A capability, in TOGAF, is a configuration of people, process and material; it aligns with CapabilityInstance in the business architecture model.

8.2.2.1.1 Superclasses

8.2.2.1.1.1 Class Strategy::Change

8.2.2.1.2 Subclasses:

8.2.2.1.3 Associations:

8.2.2.1.3.1.1 Association: Type:Association

Source: TOGAF::CapabilityIncrement **Role:** **Cardinality:**

Target: Initiative::Initiative **Role:** **Cardinality:**

8.2.2.1.3.1.1.1 *Constraints:*

8.2.2.1.3.1.2 Association:before Type:Association

Source: TOGAF::CapabilityIncrement **Role:** **Cardinality:**

Target: Capability::CapabilityBehavior **Role:** **Cardinality:**

8.2.2.1.3.1.2.1 *Constraints:*

8.2.2.1.3.1.3 Association:before Type:Association

Source: TOGAF::CapabilityIncrement **Role:** **Cardinality:**

Target: Capability::Outcome **Role:** **Cardinality:**

8.2.2.1.3.1.3.1 *Constraints:*

8.2.2.1.3.1.4 Association:after Type:Association

Source: TOGAF::CapabilityIncrement **Role:** **Cardinality:**

Target: Capability::Capability Instance **Role:** **Cardinality:**

8.2.2.1.3.1.4.1 *Constraints:*

8.2.2.1.3.1.5 Association:after Type:Association

Source: TOGAF::CapabilityIncrement **Role:** **Cardinality:**

Target: Capability::CapabilityConcept **Role:** **Cardinality:**

8.2.2.1.3.1.5.1 *Constraints:*

8.2.2.1.3.1.6 Association:before Type:Association

Source: TOGAF::CapabilityIncrement **Role:** **Cardinality:**

Target: Capability::Capability Instance **Role:** **Cardinality:**

8.2.2.1.3.1.6.1 *Constraints:*

8.2.2.1.3.1.7 Association:after Type:Association

Source: Capability::Outcome **Role:** **Cardinality:**

Target: TOGAF::CapabilityIncrement **Role:** **Cardinality:**

8.2.2.1.3.1.7.1 *Constraints:*

8.2.2.1.3.1.8 Association:before Type:Association

Source: Capability::CapabilityConcept **Role:** **Cardinality:**

Target: TOGAF::CapabilityIncrement **Role:** **Cardinality:**

8.2.2.1.3.1.8.1 *Constraints:*

8.2.2.1.3.1.9 Association:after Type:Association

Source: Capability::CapabilityBehavior **Role:** **Cardinality:**

Target: TOGAF::CapabilityIncrement **Role:** **Cardinality:**

8.2.2.1.3.1.9.1 *Constraints:*

8.2.2.1.4 Attributes:

8.2.2.1.5 Methods:

8.3 BACM Alignment with VDML

8.3.1 Introduction

The Value Delivery Metamodel (VDML) is an OMG standard modeling language and notation (OMG 2018). According to one of the authors of the standard:

VDML fills a conceptual modeling gap between top-management strategic planning and the operational-level design and management of the business.

VDML does not specify the details of decisions and actions on individual units of production, but, instead, it statistically describes the flow of deliverables through activity networks per unit of production. At the same time, contributions of value are measured

The Business Architecture Core Metamodel (BACM) is intended for use by top-management for strategic planning and business evaluation. It provides a conceptual model of a business that allows multiple perspectives of the business to be integrated to create a business model that is as complete and connected as needed to support strategy formulation and executive decision making.

It is expected that there will be many opportunities to use BACM-based models and VDML-based models together in the overall management of a business. BACM provides an alignment mechanism that effectively allows information from a foreign model to be incorporated into a BACM model. This mechanism can be adopted by other modeling standards and modeling tool providers to support information flows from BACM to other models. This section provides guidance on how to use the alignment mechanism to incorporate information from a VDML-based model into a BACM-based model. It assumes that common principles have been used in the construction of both models; alignment of a BACM-based model and a VDML-based model in the case where the models have been independently developed according to different principles will be challenging, if not impractical.

8.3.2 Value Alignment

BACM distinguishes value desired by a recipient (e.g. the customer) from the value intended or defined by a provider. The first of these is represented by the CustomerSegment class in the BACM metamodel and the second is represented by the ValueProposition class. This separation is consistent with many models of value, including Value Proposition Design, a popular book in the Business Model Canvas series (Osterwalder, et al. 2014).

VDML has a ValueProposition class in its metamodel and the definition corresponds closely to that in the BACM metamodel (BACM::ValueProposition).

The VDML metamodel does not directly include a class corresponding to BACM::CustomerSegment, but VDML::Role is associated with VDML::ValueProposition via VDML::RecipientReceivedProposition and is intended to model the value recipient. The alignment principles recommend that VDML::Role is used in this context to be aligned with BACM::CustomerSegment.

Business Architecture practice defines value streams as abstractions of capabilities required to create a product and its associated value proposition. A value stream is typically represented as a sequence of stages, each of which makes a significant contribution to the final value proposition. The contributions are termed value items.

In VDML, the concept closest to the value stream is the VDML::CapabilityMethod. This class in the metamodel is intended to represent activity networks. The VDML::CapabilityMethod links activities together with VDML::DeliverableFlows. Each VDML::Activity produces a VDML::DeliverableFlow and its associated VDML::BusinessItem. The VDML::DeliverableFlow also connects to the VDML::Activity that accepts the VDML::BusinessItem. VDML::Activities also produce VDML::ValueAdds – a type of VDML element that represents a measure of value. VDML::ValueAdds are aggregated to create VDML::ValuePropositionComponents; these elements represent significant parts of the VDML::ValueProposition.

With this description in mind, one can see that the BACM::ValueStream and its BACM::ValueStreamStages are abstractions of the activity network defined by a VDML::CapabilityMethod. The BACM::ValueItems produced by the BACM::ValueStreamStages will correspond to VDML::ValuePropositionComponents. The VDML::Activities, VDML::DeliverableFlows, VDML::BusinessItems and VDML::ValueAdds provide the details showing how the ValueStream is implemented.

The suggested alignments are shown in the following UML diagram which combines the BACM and VDML metamodels and shows alignments with the UML “trace” association. Figure 19 highlights the value alignment between the BACM and VDML.

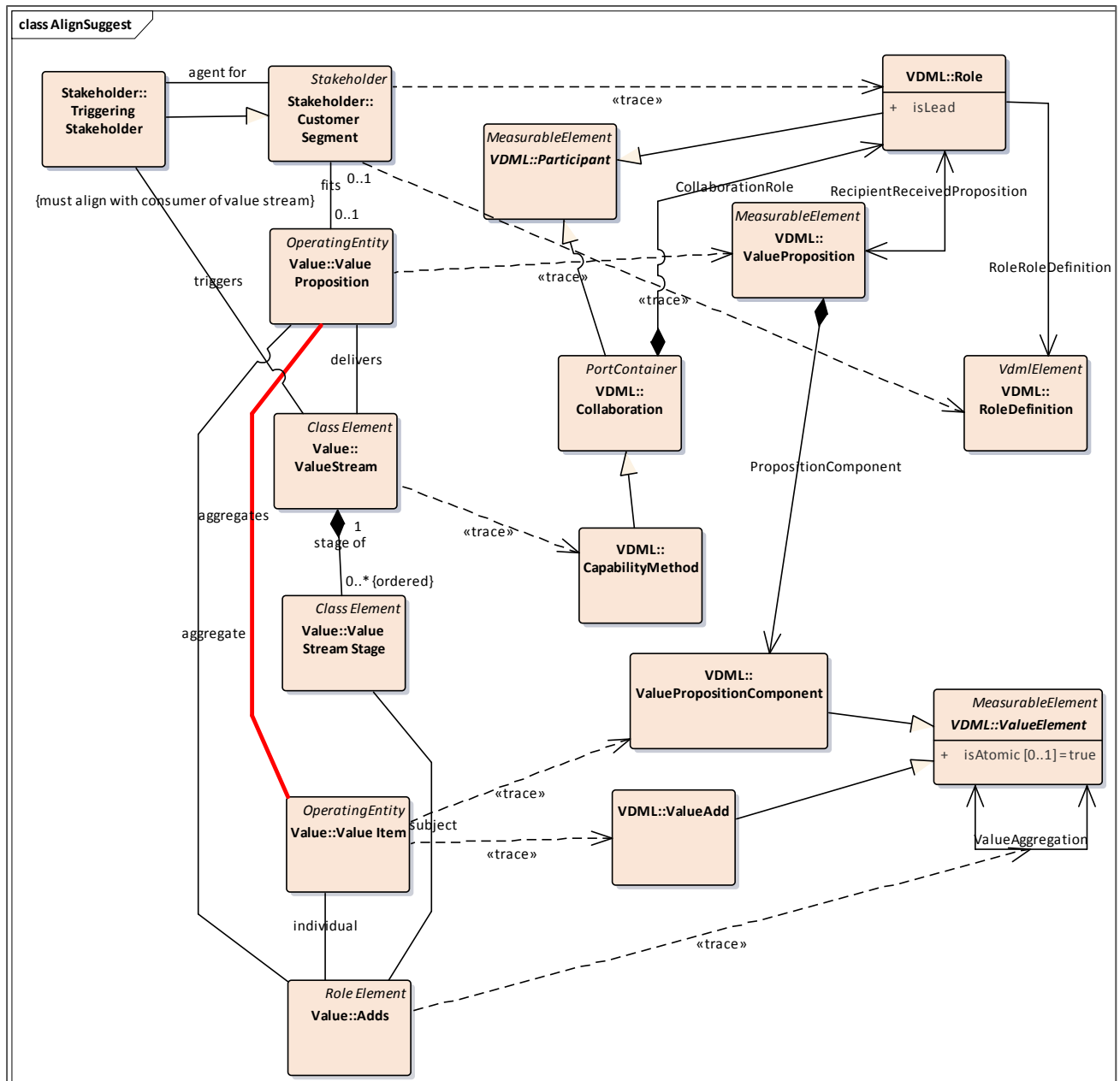


Figure 19: Alignment relationships between the BACM and VDML metamodels

8.3.3 Capability Alignment

The Business Architecture Core Metamodel (BACM) defines capability as “a particular ability or capacity that a business may possess or exchange to achieve a specific purpose or outcome”. Business Architecture treats capabilities as abstractions. Unlike capability-based planning, a capability does not identify a set of resources; it represents what needs to be done to achieve the outcome.

BACM::Capabilities are used to define the set of capacities or abilities that a business of a certain type will need, but they do not prescribe the organization of the business. Matching a BACM::Capability to a business organization and operating model requires the help of two related concepts:

- 1) The BACM::CapabilityInstance which represents a particular set of resources (including staff) configured in roles to carry out the behavior of the BACM::Capability.

- 2) The `BACM::CapabilityBehavior`, which represents a behavior specification (e.g. process definitions, procedures manuals, policies, regulations) for the `BACM::Capability`

Figure 20 depicts the VDMML capability alignment model.

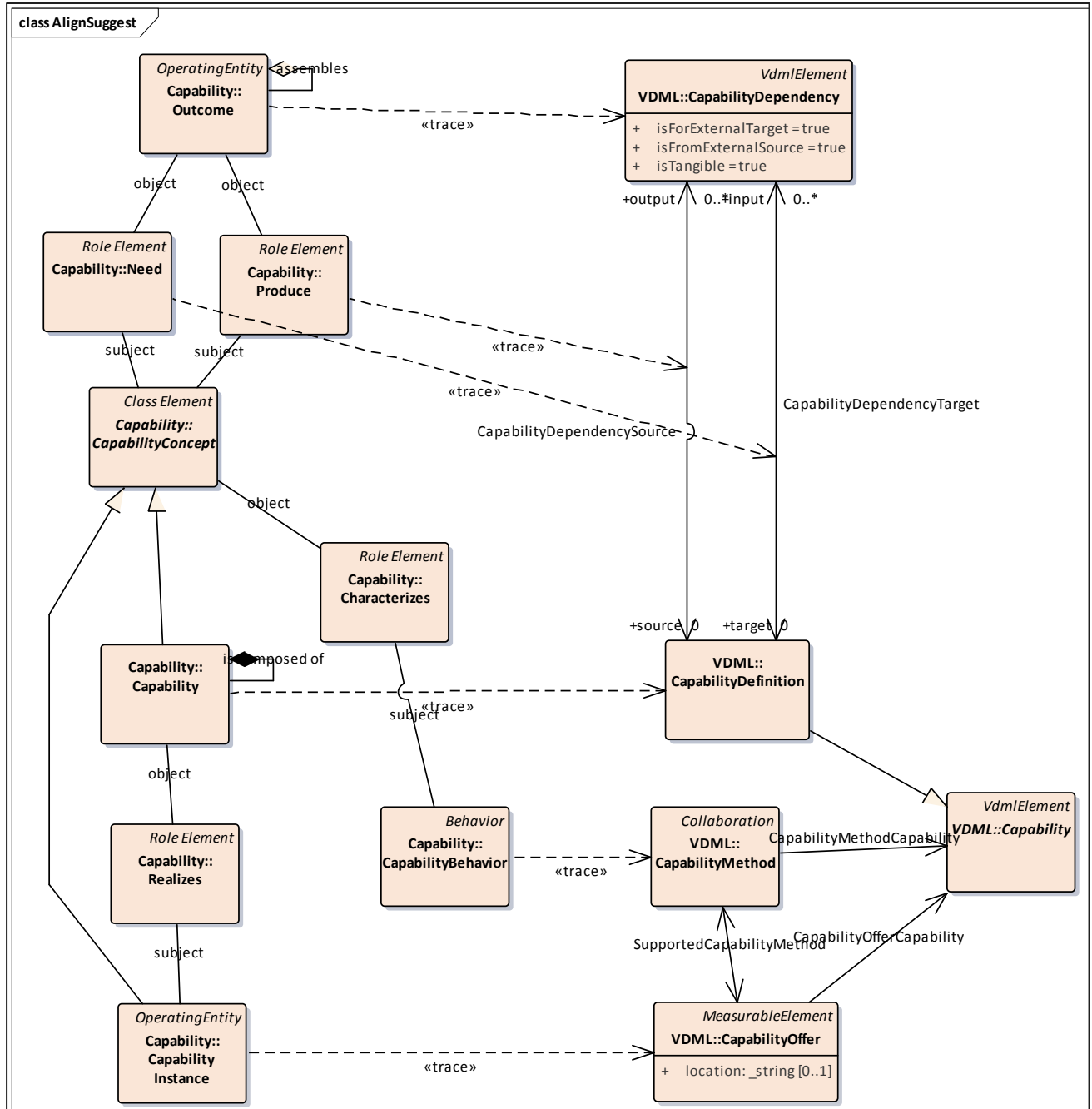


Figure 20: Capability Alignment from BACM metamodel to VDMML metamodel

BACM::Capabilities produce BACM::Outcomes and need BACM::Outcomes produced by other BACM::Capabilities. A BACM::Outcome is typically associated with a BACM::BusinessObject (a BACM::TangibleBusinessObject or a BACM::InformationConcept). The BACM::Capabilities and their BACM::Outcomes form a network.

BACM::Outcomes are associated with BACM::ValueItems, connecting the outcomes of capabilities to a value.

BACM::Capabilities are associated with BACM::ValueStreamStages so that the set of BACM::Capabilities associated

with a BACM::ValueStreamStage eventually result in producing the BACM::Outcomes and BACM::ValueItems associated with each stage in the BACM::ValueStream.

VDML has a concept that is very similar to the BACM::Capability; it is represented in the VDML metamodel by the VDML::CapabilityDefinition. VDML::CapabilityDefinitions can also be configured to create a network by connecting them together with VDML::CapabilityDependencies.

VDML also has two concepts that link VDML::CapabilityDefinitions to organizations:

- 1) The VDML::CapabilityOffer represents a specific set of resources configured in roles that are able to perform the VDML::Capability.
- 2) The VDML::CapabilityMethod (when used to represent a set of processes and policies) describes the behavior associated with a VDML::CapabilityOffer.

The recommended alignment principles from a BACM model to a VDML model are shown in the following UML diagram as “trace” associations connecting the corresponding classes in the two metamodels.

8.4 BACM Alignment with BPMN, CMMN, and DMN

8.4.1 Background

The BACM supports the creation of a business architecture, whose focus is principally at the Business Model level of detail wherein artifacts (models). The collected set of these “describes the rationale of how an organization creates, delivers, and captures value.”²² Out of this level of modeling comes the key concepts of a business architecture, such as a value stream map or a capability map.

Existing OMG standards for process, case, and decision – BPMN²³, CMMN²⁴, and DMN²⁵, respectively – address operational behaviors of an organization that “is an abstract representation of how an organization operates across process, organization and technology domains in order to accomplish its function.”²⁶ Out of this level of modeling comes the corresponding key concepts of Business Process Management (BPM), Case Management (CM, and Decision Management DM).

Each of these Model perspectives provides a different architectural set of views, which define the spectrum of understanding across these views as the modeler (and the model viewer) moves from capturing the “what” of what the organization does towards the “how” of what the organization does. This perspective is expressed in Figure 21: Model Perspective Continuum, along with the various audiences that will consume models created according to the detail called for in this continuum.

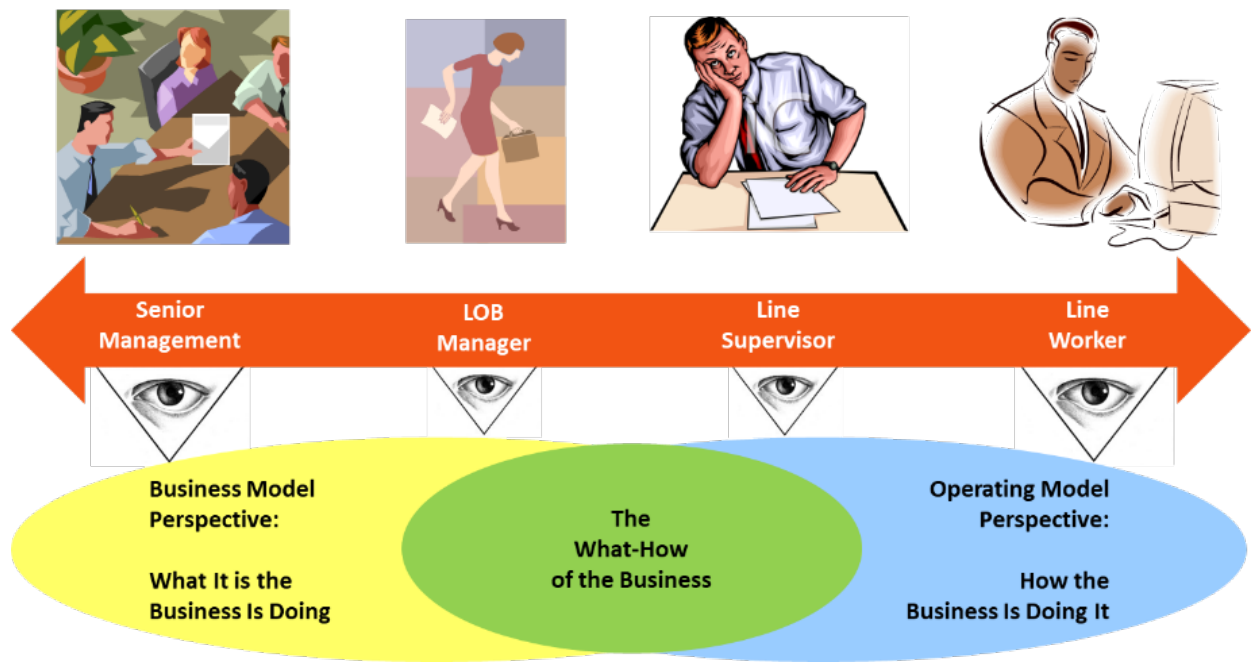


Figure 21: Model Perspective Continuum

The transition from the business architecture level (towards the left) to the BPM/CM/DM level (towards the right) is one of disparate and somewhat incompatible differences in abstraction and granularity. The BACM Solicitation encouraged respondents to propose a workable solution to this transition through Touchpoint Mechanisms that enabled a practical mapping of concepts from one level to the other. Such an approach is presented below that describes how the proposed Touchpoint Mechanisms will work between the proposed BACM concepts and the relevant concepts in BPMN, CMMN, and DMN.

8.4.2 Approach to Touchpoint Mechanisms

Describing this approach begins with following supposition: *at the business architecture level, the concept of “flow” that is needed to understand operating model behavior (as part of execution semantics, automated or otherwise) is not needed to understand the business model behavior (for business functions and results/dependencies).* The semantics of flow and/or dependency concepts are much stronger in BPMN, CMMN, and DMN than is the case in the semantics of flow and/or dependency concepts in any candidate or official business architecture Framework.

Consequently, the central aspect of the proposed “touchpoint mechanism”, which aligns to the concept of touchpoint defined in the BACM RFP, is to shed such associations as contained in the native BPMN, CMMN, and DMN models in favor of mapping relevant elements in these models based on alignment of these elements with either “Activity” as the work done or “State” as the result of the work done. The precise nature of this mapping is prescribed by the modeler or tool vendor based on specific methodologies applied by the modeler or the tool vendor, but the general nature of the touchpoint mechanism is standardized such that the mapping relationships are the same from one application of this approach to another even though the content of the mapping will almost certainly vary. The goal is to avoid Swiftian disputes over who is right or wrong in the specifics of any approach – i.e., all are “right” as long as the standard Touchpoint Mechanism is used.

Beginning with Activity – which is the first-class concept in BPMN, is key in CMMN, and is implicit in DMN – one can see Activity as either a container of other Activities or – at the lowest level – as something with input/output state changes used/produced by the Activity’s execution. Capability has a similar compositional nature but describes the “what” whereas Activity describes the associate “how.”

Note how Activity plays out in this regard across BPMN, CMMN, and DMN metamodels:

- Process in BPMN is really a specialized flow element container, which can (and typically does) include Activities:
 - Process (as a callable activity), sub-process, task (of any type), and call activity in BPMN are all Activities in BPMN
 - Even a Process in BPMN, consisting of just a start event to a sequence flow to an end event, still qualifies (though it does no work because of the absence of an Activity)
- Decision Task in CMMN or its twin business rule task in BPMN is just a special kind of Activity:
 - Decision Model in DMN describes declaratively the decision logic and informational dependencies, but must be invoked by one of these two types of Activity
 - Decisions DO NOT describe an Activity, but the logic that is executed as the Activity is performed, which is not the same thing
- Process Task in CMMN and case task (an increasingly common/accepted extension to BPMN) are an Activity at the lowest level or an Activity as a container of other Activities
- Other Task Types in CMMN are like Task Types in BPMN – i.e., Activities at the lowest level
- Plan Fragment and Stage – both in CMMN, where Stage is a subtype of Plan Fragment – are an Activity at the lowest level or an Activity as a container of other Activities

The proposed Touchpoint Mechanism is an **Activity Assembler** that will populate a proxy-like **Activity**, which will pull concrete elements out of BPMN and CMMN models from the corresponding Foreign Model Container into the **Activity** as the result of the query. Query parameters are set per a methodology or otherwise specified by the modeler or tool vendor architect and pull identified elements along with associated elements (per relationships in the applicable metamodel) as needed.²⁷

The foreign model container could be an XML file (e.g., for the BPMN model or as an RDF version of the BPMN model) or a modeling tool repository (various types), from which the appropriate query mechanism would access the BPMN and CMMN Models. This supports mapping to a Capability Instance or Capability Behavior in the BACM, including flowing up assessments for heat maps by way of the mapping to Capability that shows how more concrete elements (i.e., those from a Process model or its elements for Activity) can inform more abstract elements, specifically a capability by way of the capability instance to which it may be mapped.

Proceeding to State – which is the first-class concept in CMMN, is key in BPMN, and is implicit in DMN – one can see State as either a container of other States or – at the lowest level – as something that is the input to an Activity’s execution or the output of an Activity’s execution. Event and Data in BPMN, CMMN, and DMN can be seen as achieving a State that is either a pre-condition that is required or a post-condition that is produced. Outcome has a similar compositional nature but describes a result that is produced/required by a capability – a “what” concept – whereas State describes a result that is produced/required by an Activity – a “how” concept.

State plays out in this regard across the metamodels for BPMN, CMMN, and DMN as follows:

- For a Process in BPMN as the Activity container, the Input/Output Specification of included Activities and any Catching or Throwing Events achieve States that suggest Outcomes, but relating these Activities within an associated business architecture model is not really understandable without re-introducing “flow”, with the same to be said for Activity as the Activity container or just Activity
- Note that Data Object and Data Store BPMN and Case File Item in CMMN have event-like aspects in that something else occurring in the model is dependent or acted upon them
- “Decision” is a special kind of result (or output, if you will) of the execution of a Decision Task in CMMN or its twin Business Rule Task in BPMN, which achieves a State that suggests an Outcome at the top level of a Decision Model
- Data Input as an input dependency between any decision node and a Knowledge Source or Decision Output from one node as an input dependency with respect to another decision node represent informational

dependencies that can similar be seen as State used or produced

The proposed Touchpoint Mechanism is a State Assembler that will populate a proxy-like State (that can be produced or required), which will pull concrete elements out of BPMN, CMMN, and DMN models from the Foreign Model Container into the State as the result of the query. Query parameters are set per a methodology or otherwise specified by the modeler or tool vendor, and pull identified elements along with associated elements (per relationships in the applicable metamodel) as needed.²⁸

The Foreign Model Container could be an XML file (e.g., for the BPMN model on as an RDF version of the BPMN model) or a modeling tool repository (various types), from which the appropriate query mechanism would access the BPMN, CMMN, and DMN models. This supports mapping to an Outcome (produced or required), including flowing down as expectations for the Activity and Data concepts and Catching or Throwing Events based on the normative characteristics of Outcome (particularly as expressed as Information Objects) that shows how the more abstract (Outcome) can order the more concrete (a Process model or its Activity-level, Data-level, or Event-level elements).

The Assembler provides the extraction from the Foreign Model, while the proxy-like element contains the result set in some grouping called for per the methodology of the modeler or tool vendor, resolving any granularity issues. This element is then mapped to a corresponding element in the business architecture, again per the methodology of the modeler or tool vendor, resolving any viewpoint issues. This is expressed in Figure 22: Touchpoint Mechanisms for BPMN, CMMN, and DMN presented below.

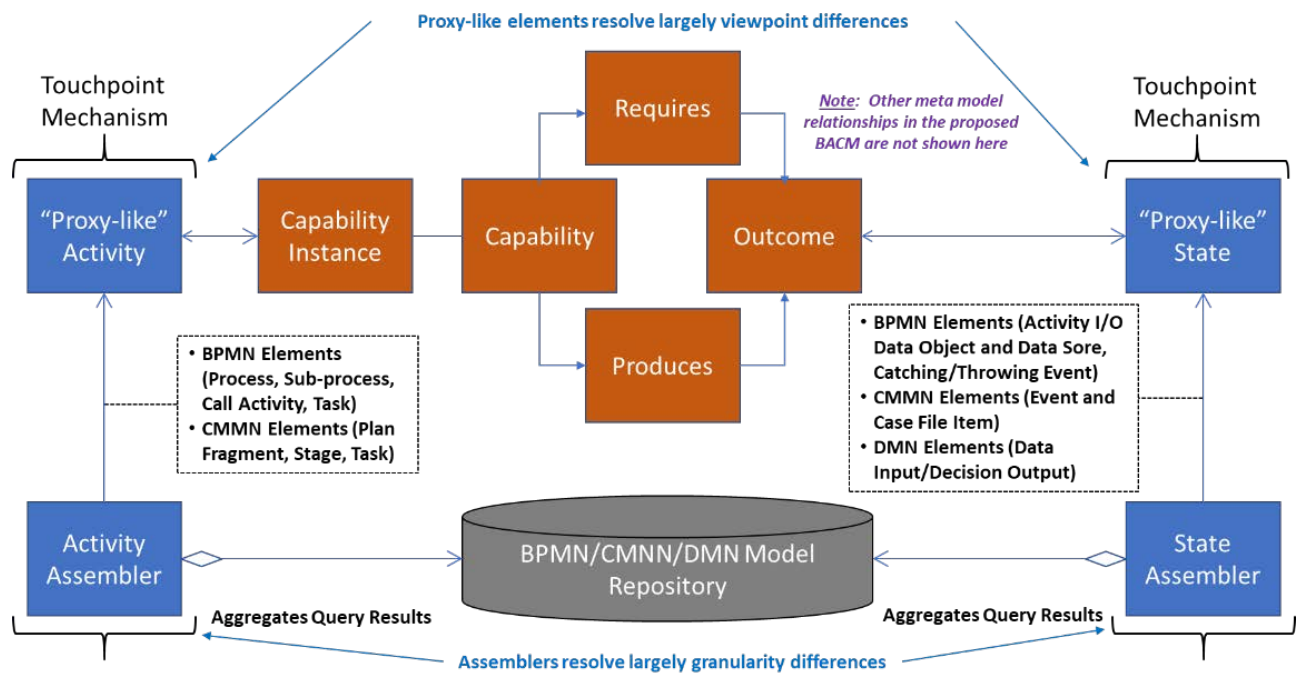


Figure 22: Touchpoint Mechanisms for BPMN, CMMN, and DMN

8.4.3 Mappings from Assemblers to Corresponding Elements

Figure 23: BPMN to Activity Mapping depicts the detailed metamodel relationships for assembling Activity elements out of a BPMN model.

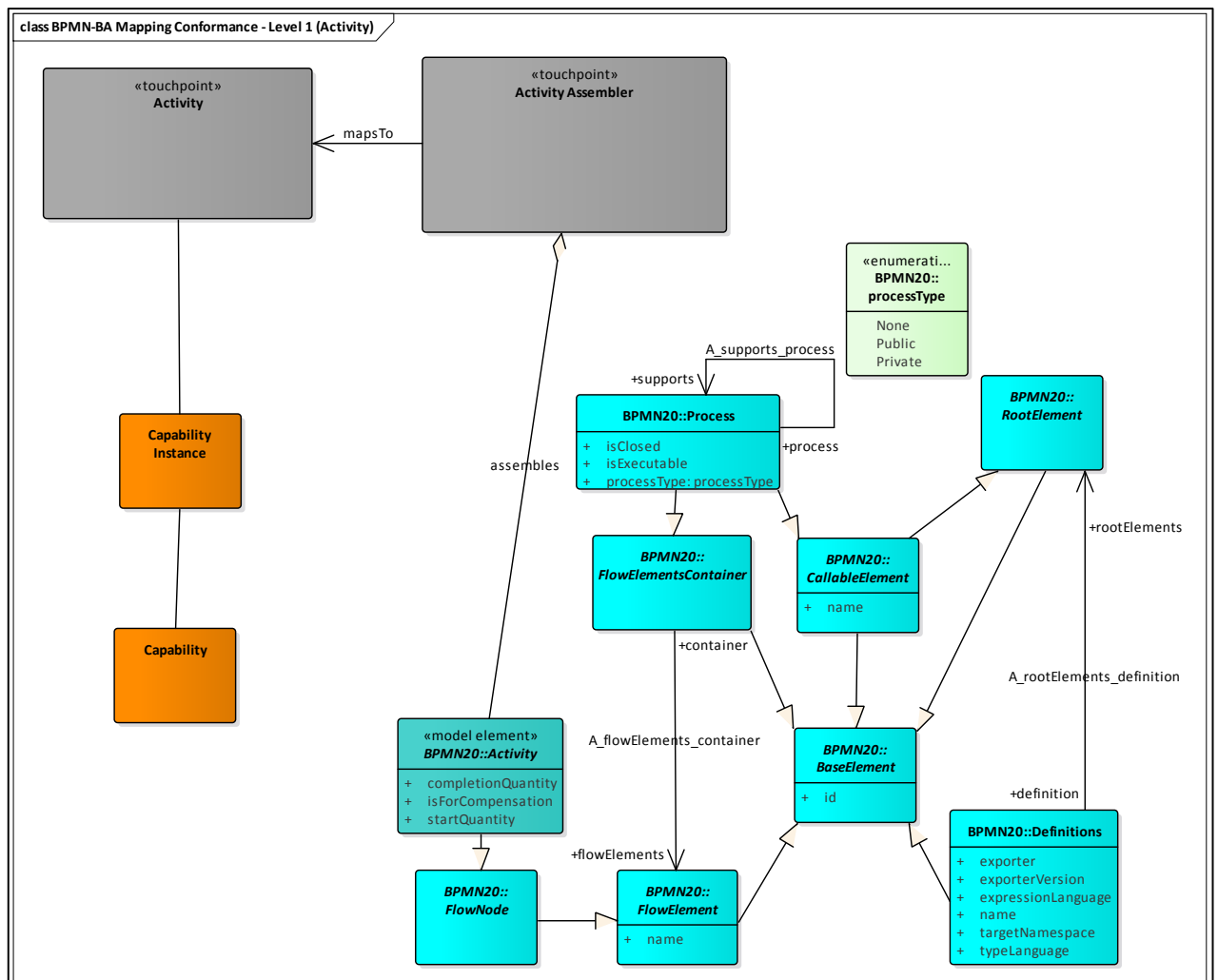


Figure 23: BPMN to Activity Mapping

Figure 24: BPMN to State Mapping (for Events only) depicts the detail metamodel relationships for assembling Event elements out of a BPMN model.

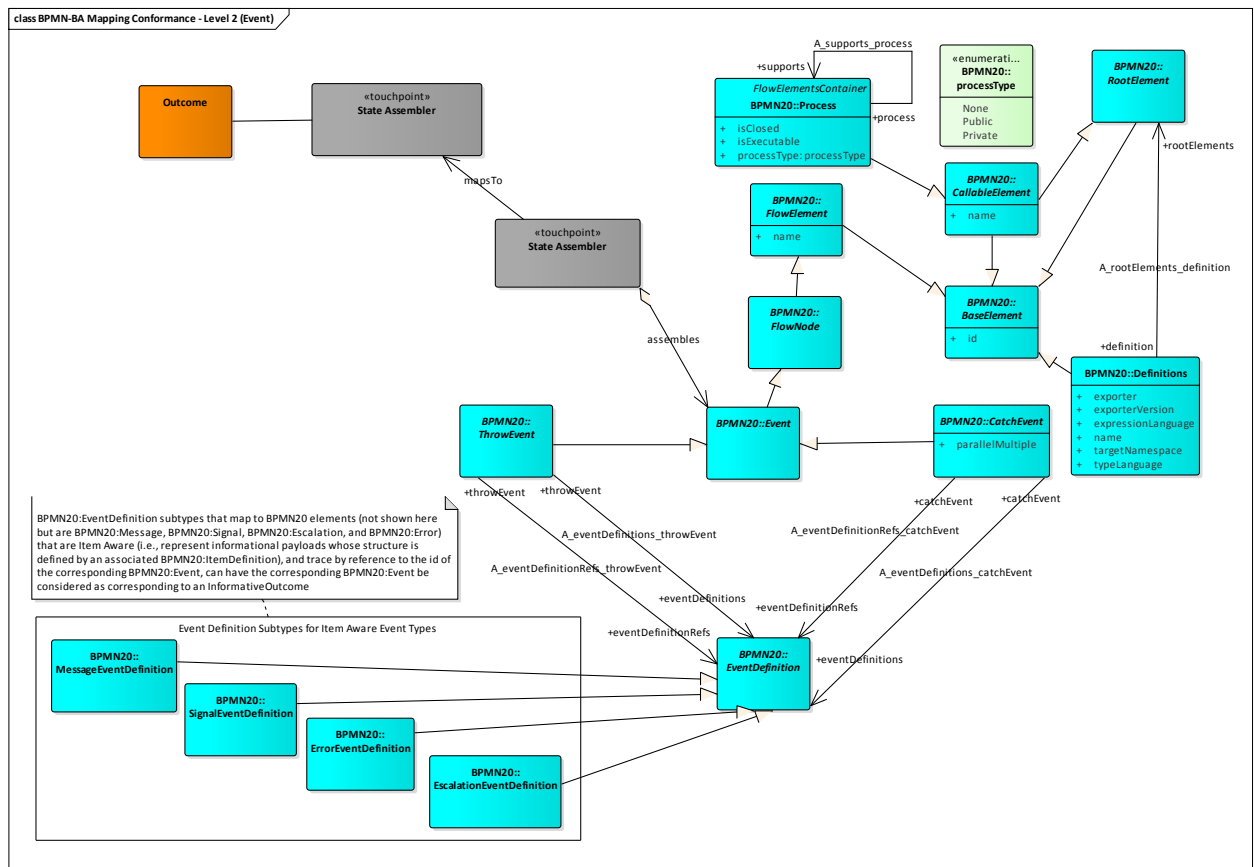


Figure 24: BPMN to State Mapping (for Events only)

Business Architecture Core Metamodel, Version 1.0



Figure 26: DMN to State Mapping depicts the detail metamodel relationships for assembling Input Data and Decision elements out of a DMN model.

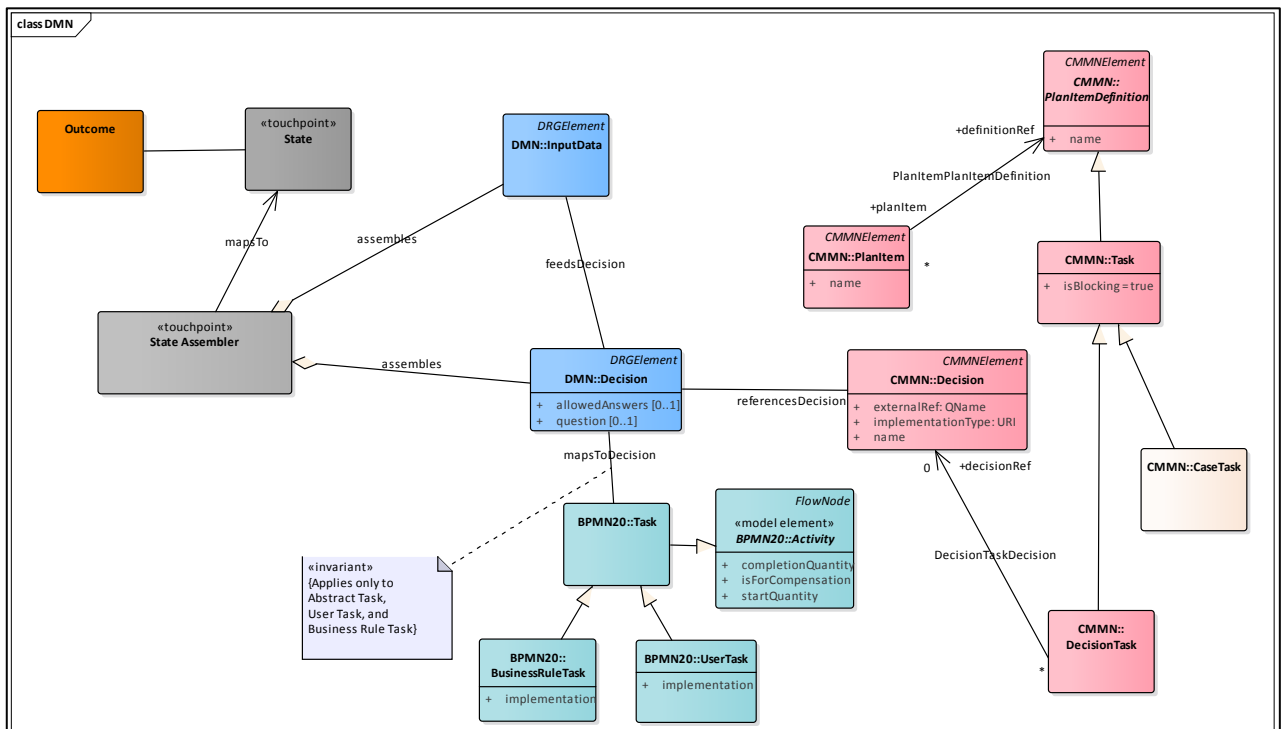


Figure 26: DMN to State Mapping:

8.4.4 Examples of Touchpoint Mechanism at Work per Different Methodologies

Selecting Specific Activities in a Process Model in figure 27 depicts how this might work selecting specific Tasks from a BPMN Process Model and mapping them to capability instance (and then to capability), noting the XML File for the model as the source along with its XML element names/values.

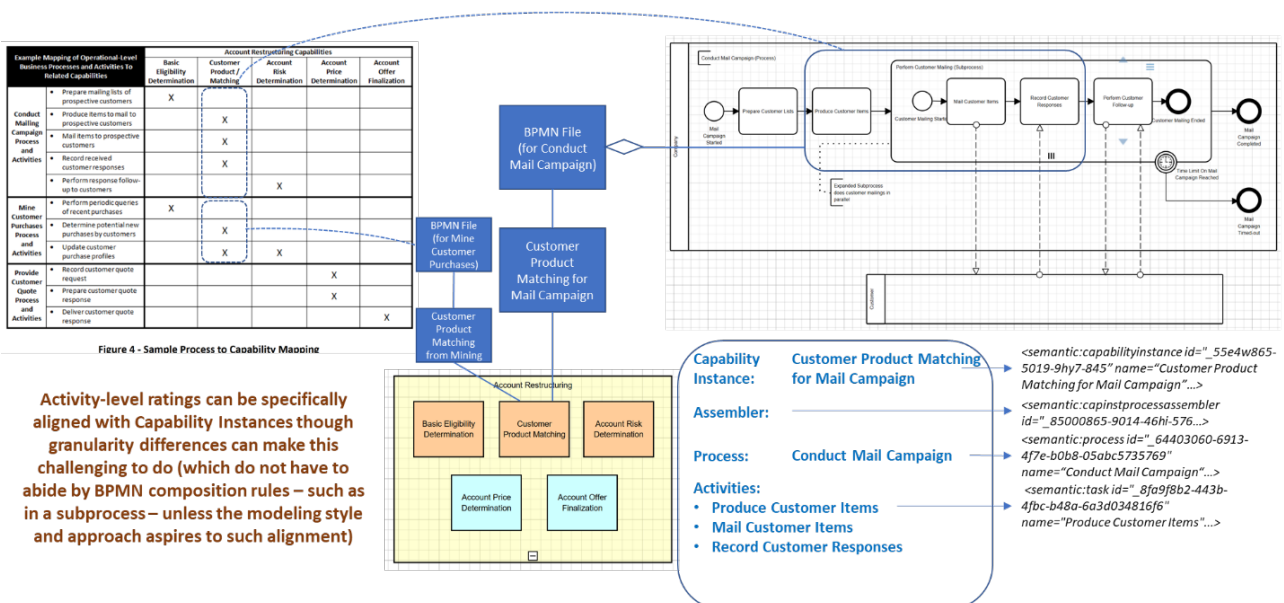
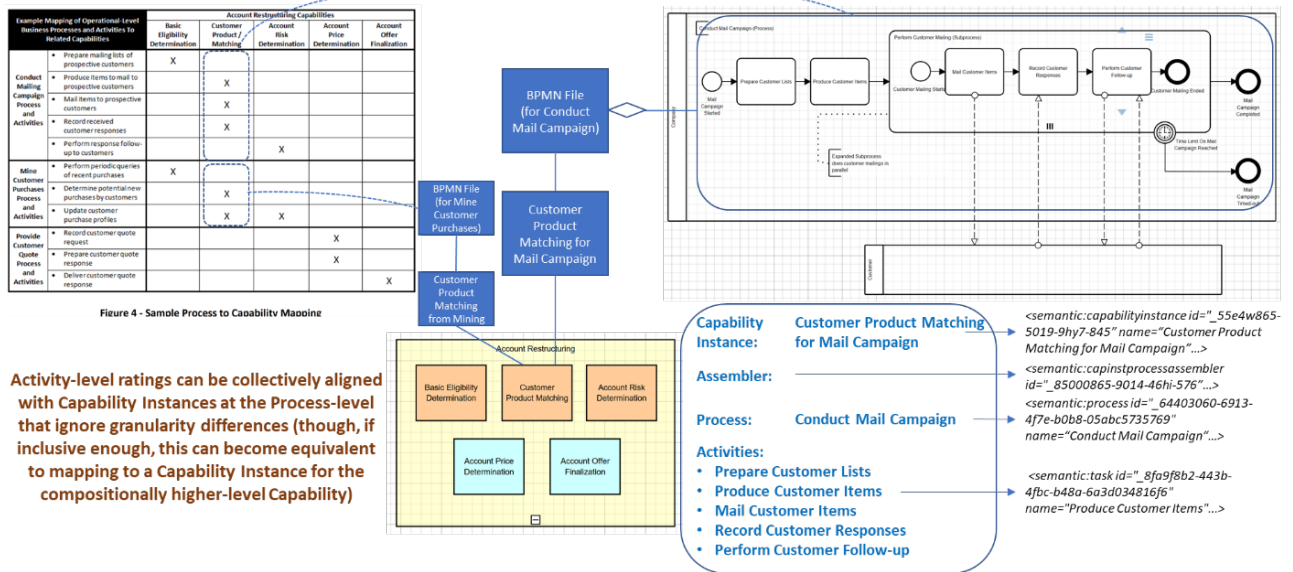


Figure 27: Selecting Specific Activities in a Process Model

Selecting All of the Activities in a Process Model below shows how this might work selecting all of the Tasks from a BPMN Process Model and mapping them to capability instance (and then to capability), noting the XML File for the model as the source along with its XML element names/values, *with the point here being that the mechanism works the same way as the other one.*



8.4.5 Conclusion

This use of Touchpoint Mechanisms to resolve differences in abstraction and granularity between models created from the Business Model perspective and models created from the operating model perspective is proposed as a methodology-agnostic but standard manner by which to create such mappings. It is believed that a standard mechanism has more practical value in the near- and long-term than attempting to come to some final resolution of what the specific mapping should be. This approach leverages well tested theory in semantics and object-oriented programming to come, and is consistent with a MOF-level view of the metamodels for BPMN, CMMN, and DMN relative to BACM.

9 Annexes

Normative annexes are integral parts of the standard. Their presence is optional. An annex's normative status (as opposed to informative) shall be made clear by the way in which it is referred to in the text and under the heading of the annex.

Informative annexes give additional information intended to assist the understanding or use of the standard and shall not contain provisions to which it is necessary to conform in order to be able to claim compliance to the standard. Their presence is optional. An annex's informative status (as opposed to normative) shall be made clear by the way in which it is referred to in the text and under the heading of the annex.

The following Annexes are included:

Annex A: Glossary

Annex B: References

9.1 Annex A: Glossary

All definitions contained in the Annex A: Glossary are sourced where appropriate. Definitions that are not otherwise sourced were derived from the BIZBOK® Guide, v6.5, Part 1: Glossary at www.businessarchitectureguild.org.

Business Architecture

Business architecture represents holistic, multidimensional business views of capabilities, end-to-end value delivery, information, and organizational structure as well as the relationships among these business views and strategies, products, policies, initiatives, and stakeholders.

Source: Architecture definitions adopted by the Federation of Enterprise Architecture Professional Organizations (FEAPO), Taxonomy Working Group, January 14, 2017, after passing a vote by FEAPO Member Organizations. See www.feapo.org and at: <http://feapo.org/wp-content/uploads/2018/05/Taxonomy-The-Federation-of-Enterprise-Architecture-Definitions-.pdf>

Business Architecture Framework

A conceptual view of how business blueprints, business scenarios, and the business architecture knowledgebase interrelate to provide a foundation for establishing the business architecture.

Business Ecosystem

One or more legal entities, in whole or in part, that exist as an integrated community of individuals and assets, or aggregations thereof, interacting as a cohesive whole toward a common mission or purpose.

Business Model

A business model describes the rationale of how an organization creates, delivers, and captures value.

Source: Alexander Osterwalder and Yves Pigneur, *Business Model Generation*, Self-Published, 2010, p. 14.

Business Object

A representation of a thing active in the business domain, including at least its business name and definition, attributes, behavior, relationships and constraints, that may represent, for example, a person, place, or concept.

Source: Business Object Designer, <http://docs.imis.com/15.2.0/index.htm?url=whatisabusinessobject2.htm>

Business Process

A series of logically related activities or tasks (such as planning, production, or sales) performed together to produce a defined set of results.

Source: www.BusinessDictionary.com

Business Process Management (BPM)

Discipline involving any combination of modeling, automation, execution, control, measurement, and optimization of business activity flows, in support of enterprise goals, spanning systems, employees, customers, and partners within and beyond the enterprise boundaries.

Source: <http://social-biz.org/2014/01/27/one-common-definition-for-bpm/3>

This definition is based on discussions on or with Linked-In's BPM Guru Group, BPM.COM's Forum, Workflow Management Coalition (WfMC) Members, and the Association of BPM Professionals (ABPMP) Forum.

Business / IT Architecture Alignment

A state in which business information, capabilities, and value streams are appropriately represented and deployed from an IT automation perspective.

Business Unit

A logical element or segment of a company (such as accounting, production, marketing) representing a specific business function, and a definite place on the organizational chart, under the domain of a manager. Also called department, division, or functional area.

Source: www.BusinessDictionary.com

Capability

A particular ability or capacity that a business may possess or exchange to achieve a specific purpose or outcome.

Source: Ulrich Homann, “A Business-Oriented Foundation for Service Orientation”, 2006.

Capability Level

A number that indicates the depth of decomposition for a given capability.

Capability Instance

A specific realization of a capability, as it exists or is envisioned to exist, in the context of a given business unit, value stream stage, or other situational context.

Capability Map

A diagrammatic or other means or media used to represent capabilities for a business.

Source: BIZBOK® Guide v7.0, Part 1: Glossary.

Case Management

A method or practice of coordinating work by organizing all of the relevant pieces into one place – called a case.

Source: Keith D. Swenson, “Adaptive Case Management”, Tampa: MK Press, 2011.

Collaborative Team

A named group or unit created by two or more internal or external business units that has a defined set of shared principles and common goals.

Decision Model

Technique for identifying the set of decisions that must be made to support a gate along a gated process for approving an initiative.

Information Concept

Way in which to represent business terms and semantics within the context of business architecture.

Initiative

A course of action that is being executed or has been selected for execution.

Objective

A quantitative, measurable result that defines strategy.

Source: Forrester Research

Operating Model

An operating model is an abstract representation of how an organization operates across a range of domains in order to accomplish its function.

Source: Marne de Vries et al. “A Method for Identifying Process Reuse Opportunities to Enhance the Operating Model”, IEEE International Conference on Industrial Engineering and Engineering Management, 2011.
en.wikipedia.org/wiki/Operating_model.

Organization

A social unit of people, systematically structured and managed to meet a need or to pursue collective goals on a continuing basis.

Source: www.BusinessDictionary.com

Organization Map

A business blueprint that depicts organizational decomposition by organization unit along with additional aspects of the business where appropriate.

Outcome

An end result or final product that is a consequence of an event, action, or a series of events/actions.

Participating Stakeholder

A category of stakeholder that has a defined role or responsibility within a value stream stage by contributing to outcomes of the capabilities associated with that value stream stage.

Policy

A course or principle of action adopted or proposed by a government, party, business, or individual.

Source: Oxford Dictionaries, Oxford University Press, s.v., “Policy”.

Product

The overall experience provided by the combination of goods and services to satisfy the customer’s needs.

Source: Greg Geracie and Steven Eppinger, *The Guide to the Product Management and Marketing Body of Knowledge® (ProdBOK®)*, Product Marketing and Management, 2013.

Scenario

A program-oriented, template-based application of business architecture that addresses a specific, yet common requirement that organizations face.

Service

Intangible products such as accounting, banking, cleaning, consultancy, education, insurance, expertise, medical treatment, or transportation.

Source: www.BusinessDictionary.com

Stakeholder

An internal or external individual or organization with a vested interest in achieving value through a particular outcome.

Strategy

The pattern or plan that integrates an organization’s major goals, policies, and action sequences into a cohesive whole.

Source: J. B. Quinn, *Strategies for Change: Logical Incrementalism* (Homewood: Richard D. Irwin, Inc., 1980).

Strategy Map

A business blueprint that “displays the cause-effect relationships among the objectives that make up a strategy”.

Source: J. B. Quinn, *Strategies for Change: Logical Incrementalism* (Homewood: Richard D. Irwin, Inc., 1980).

Triggering Stakeholder

A category of stakeholder that initiates a value stream for purposes of achieving a stated value proposition.

Value

The benefit that is derived by an organization’s stakeholder while interacting with that organization.

Value Item

The judgment of worth, made by an individual or organization, attached to something tangible or intangible and attained in the course of a particular interaction with one or more other parties.

Value Proposition

An innovation, service, or feature intended to make a company, product, or service attractive to customers or related stakeholders.

Value Stream

An end-to-end collection of activities that create a result for a customer, who may be the ultimate customer or an internal end-user of the value stream.

Source: Ralph Whittle and Conrad Myrick, *Enterprise Business Architecture: The Formal Link between Strategy and Results* (Boca Raton: Auerbach, 2005).

Value Stream Stage

A distinct, identifiable phase or step within a value stream that has a unique name, entrance criteria, exit criteria, and identifiable participating stakeholder(s).

9.2 Annex B: References

The following reflect various industry references for terms, standards, and related concepts used within the BACM.

-
- ¹ Business Architecture Guild®, www.businessarchitectureguild.org
 - ² BIZBOK® Guide, Part 1: Introduction, <http://www.businessarchitectureguild.org/?page=002>.
 - ³ Federation of Enterprise Architecture Professional Organizations (FEAPO), www.feapo.org.
 - ⁴ FEAPO definition of business architecture, FEAPO Publication Resources, <http://feapo.org/wp-content/uploads/2018/05/Taxonomy-The-Federation-of-Enterprise-Architecture-Definitions-.pdf>
 - ⁵ FEAPO member listing, <http://feapo.org/members/>.
 - ⁶ Business Architecture Quick Guide, MK Press, January 2018, Business Architecture Guild.
 - ⁷ Marne de Vries et al. “A Method for Identifying Process Reuse Opportunities to Enhance the Operating Model”, IEEE International Conference on Industrial Engineering and Engineering Management, 2011. en.wikipedia.org/wiki/Operating_model.
 - ⁸ Business Architecture: Putting “Business” into Enterprise Architecture, R. Soley and W. Ulrich, CXO Insights, May 09, 2016, http://cymcdn.com/sites/www.businessarchitectureguild.org/resource/resmgr/docs/putting_business_into_busine.pdf.
 - ⁹ “A Business-Oriented Foundation for Service Orientation”, Homann, Ulrich, 2006, <http://bit.ly/2faqT67>.
 - ¹⁰ BIZBOK® Guide, Appendix A, Glossary, <http://www.businessarchitectureguild.org/?page=002>.
 - ¹¹ BIZBOK® Guide, Part 1: Introduction, <http://www.businessarchitectureguild.org/?page=002>.
 - ¹² BIZBOK® Guide, Appendix A, Glossary, <http://www.businessarchitectureguild.org/?page=002>.
 - ¹³ Source: Greg Geracie and Steven Eppinger, *The Guide to the Product Management and Marketing Body of Knowledge® (ProdBOK®)*, Product Marketing and Management, 2013.
 - ¹⁴ Oxford Dictionaries, Oxford University Press.
 - ¹⁵ J. B. Quinn, *Strategies for Change: Logical Incrementalism* (Homewood: Richard D. Irwin, Inc., 1980).
 - ¹⁶ BIZBOK® Guide, Appendix A, Glossary, <http://www.businessarchitectureguild.org/?page=002>.
 - ¹⁷ A Guide to the Business Analysis Body of Knowledge (BABOK) v2.0, International Institute of Business Analysis
 - ¹⁸ Structured Metrics Metamodel (SMM) standard, <https://www.omg.org/spec/SMM/About-SMM/>.
 - ¹⁹ “A Business-Oriented Foundation for Service Orientation”, Homann, Ulrich, 2006, https://cdn.ymaws.com/www.businessarchitectureguild.org/resource/resmgr/homann_article_on_capabilityi.pdf.
 - ²⁰ *Enterprise Business Architecture: The Formal Link between Strategy and Results*, Ralph Whittle and Conrad Myrick, Boca Raton: Auerbach, 2005.
 - ²¹ “The Great Transition”, Martin, James, 1995, AMACOM, ISBN-10: 0814403158.
 - ²² Source: Alexander Osterwalder and Yves Pigneur, *Business Model Generation*, Self-Published, 2010, Page 14
 - ²³ BPMN 2.0 Standard, <http://www.omg.org/bpmn/>.
 - ²⁴ CMMN Standard, <https://www.omg.org/spec/CMMN/1.0/About-CMMN/>.

²⁵ DMN Standard, <https://www.omg.org/spec/DMN/About-DMN/>.

²⁶ Marne de Vries et al. “A Method for Identifying Process Reuse Opportunities to Enhance the Operating Model”, IEEE International Conference on Industrial Engineering and Engineering Management, 2011. en.wikipedia.org/wiki/Operating_model.

²⁷ For example, an activity element will be associated with the flow container in which it resides (e.g., a process), but this may or may not be information needed per the methodology applied by the modeler or the tool vendor.

²⁸ For example, a data object element will be associated with the activity with which it has a data association, but this may or may not be information needed per the methodology applied by the modeler or the tool vendor.